



MINERVAMESH: UMA PLATAFORMA WEB DE CÓDIGO ABERTO PARA
SIMULAÇÕES NUMÉRICAS NO ENSINO DE ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

Projeto de Graduação apresentado ao Curso de Engenharia
Mecânica da Escola Politécnica, Universidade Federal do Rio
de Janeiro, como parte dos requisitos necessários à obtenção
do título de Engenheiro.

Orientador: Gustavo Rabello dos Anjos

Rio de Janeiro

Junho de 2026



UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

Departamento de Engenharia Mecânica

DEM/POLI/UFRJ



MINERVAMESH: UMA PLATAFORMA WEB DE CÓDIGO ABERTO PARA
SIMULAÇÕES NUMÉRICAS NO ENSINO DE ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

PROJETO FINAL SUBMETIDO AO CORPO DOCENTE DO DEPARTAMENTO
DE ENGENHARIA MECÂNICA DA ESCOLA POLITÉCNICA DA
UNIVERSIDADE FEDERAL DO RIO DE JANEIRO COMO PARTE
DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO GRAU DE
ENGENHEIRO MECÂNICO.

Aprovada por:

Prof. Gustavo Rabello dos Anjos, Ph.D.

Prof. Examinador Um, D.Sc.

Prof. Examinador Dois, D.Sc.

RIO DE JANEIRO, RJ – BRASIL

JUNHO DE 2026

Farias, Luiz Gustavo Santos

MinervaMesh: uma plataforma web de código aberto para simulações numéricas no ensino de Engenharia Mecânica/ Luiz Gustavo Santos Farias. – Rio de Janeiro: UFRJ/Escola Politécnica, 2026.

XVIII, 120 p.: il.; 29,7cm.

Orientador: Gustavo Rabello dos Anjos

Projeto de Graduação – UFRJ/ Escola Politécnica/ Curso de Engenharia Mecânica, 2026.

Referências Bibliográficas: p. 87 – 92.

1. Método dos Elementos Finitos. 2. Método das Diferenças Finitas. 3. Simulação Web. 4. Mecânica Computacional. 5. Ensino de Engenharia. I. Anjos, Gustavo Rabello dos. II. Universidade Federal do Rio de Janeiro, UFRJ, Curso de Engenharia Mecânica. III. MinervaMesh: uma plataforma web de código aberto para simulações numéricas no ensino de Engenharia Mecânica.

A Deus, Senhor da minha vida, que sustentou cada etapa desta caminhada.

À minha esposa Aretha, companheira fiel em todos os momentos.

À minha filha Marina, motivação diária para não desistir.

À minha mãe, Daniela, cuidado, suporte e incentivo de todas as horas.

Ao meu pai, Silvio, que mesmo de longe nunca deixou de apoiar meus projetos e trabalhou duro para que nada me faltasse.

Às minhas irmãs, Yngrid e Yasmim, cumplicidade que a distância não desfaz.

À minha avó Jussa, colo e sabedoria de todas as gerações.

Comecei este caminho como estudante. Concluo como engenheiro, marido, pai e servo de Cristo.

Agradecimentos

Agradeço primeiramente a Deus, por sustentar cada etapa desta caminhada e por conceder sabedoria e perseverança para chegar até aqui.

Ao meu orientador, Prof. Gustavo Rabello dos Anjos, pela orientação cuidadosa, confiança e incentivo intelectual, fundamentais para o desenvolvimento deste trabalho e para o amadurecimento da proposta do MinervaMesh.

À Universidade Federal do Rio de Janeiro e ao corpo docente do curso de Engenharia Mecânica, pela formação técnica e científica que tornou possível a realização deste projeto.

Ao meu grande amigo e padrinho de casamento, Michel Silva Bonifácio, pelo apoio durante a graduação e pela presença constante em momentos decisivos da minha trajetória acadêmica.

Ao meu amigo Eliab Araújo, pelo incentivo para a conclusão do curso e pela motivação contínua no desenvolvimento deste trabalho.

Aos colegas, amigos e a todos que contribuíram direta ou indiretamente para minha formação como engenheiro.

Resumo do Projeto de Graduação apresentado à Escola Politécnica/UFRJ como parte dos requisitos necessários para a obtenção do grau de Engenheiro Mecânico

MINERVAMESH: UMA PLATAFORMA WEB DE CÓDIGO ABERTO PARA SIMULAÇÕES NUMÉRICAS NO ENSINO DE ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

Junho/2026

Orientador: Gustavo Rabello dos Anjos

Programa: Engenharia Mecânica

A simulação numérica é ferramenta central de análise e projeto em Engenharia Mecânica, mas seu uso no ensino esbarra em duas dificuldades recorrentes: as ferramentas mais capazes exigem licenciamento, infraestrutura e configuração de ambiente, e raramente expõem de forma transparente a formulação matemática e o código de cada simulação. O problema atacado é reduzir a barreira de acesso a um ambiente de simulação didático, sem abrir mão do rigor técnico nem da transparência da implementação. Como resposta, apresenta-se o desenvolvimento do *Minerva-Mesh*, plataforma web de código aberto que resolve Equações Diferenciais Parciais de interesse em Engenharia Mecânica pelo Método dos Elementos Finitos (MEF) e pelo Método das Diferenças Finitas (MDF). A contribuição articula dois aspectos: (i) a entrega das simulações integralmente no navegador, em arquitetura *client-side* sobre *WebAssembly* e *PyScript*, sem instalação e com código-fonte Python inspecionável; e (ii) a implementação e verificação dos métodos numéricos, confrontadas com soluções analíticas e, nos casos sem solução fechada, com *benchmarks* consagrados da literatura. Os resultados mostram que as equações discretizadas reproduzem as soluções de referência e que a plataforma é adequada ao ensino, à prototipagem e a demonstrações de mecânica computacional, podendo ser utilizada por alunos da própria instituição e de outras.

Abstract of Undergraduate Project presented to POLI/UFRJ as a partial fulfillment of the requirements for the degree of Mechanical Engineer

MINERVAMESH: AN OPEN-SOURCE WEB PLATFORM FOR NUMERICAL SIMULATIONS IN MECHANICAL ENGINEERING EDUCATION

Luiz Gustavo Santos Farias

June/2026

Advisor: Gustavo Rabello dos Anjos

Department: Mechanical Engineering

Numerical simulation is a core tool for analysis and design in Mechanical Engineering, yet teaching runs into two recurring obstacles: the most capable tools require licensing, infrastructure, and environment setup, and they seldom expose the underlying mathematical formulation and source code of each simulation. The problem addressed is to lower the barrier of access to a didactic simulation environment, without sacrificing technical rigor or implementation transparency. As a response, this work presents the development of *MinervaMesh*, an open-source web platform that solves Partial Differential Equations of interest in Mechanical Engineering with the Finite Element Method (FEM) and the Finite Difference Method (FDM). The contribution combines two aspects: (i) the delivery of these simulations fully in the browser, using a *client-side* architecture built on *WebAssembly* and *PyScript*, with no installation and with inspectable Python source code; and (ii) the implementation and verification of the numerical methods, assessed against analytical solutions and, for cases without a closed-form solution, against well-established benchmarks. Results show that the discretized equations reproduce the reference solutions and that the platform is suitable for teaching, rapid prototyping, and demonstrations in computational mechanics, ready to be used by students at the author's institution and beyond.

Sumário

Lista de Figuras	xiii
Lista de Tabelas	xvi
Lista de Abreviaturas	xviii
1 Introdução	1
1.1 Introdução	1
1.2 Objetivos	2
1.3 Justificativa	3
1.4 Organização do Trabalho	4
2 Revisão Bibliográfica	5
2.1 Introdução ao Capítulo	5
2.2 Estado da Arte	5
2.2.1 Problemas Térmicos e de Escoamento	5
2.2.2 Métodos Numéricos para EDPs	6
2.2.3 Consolidação Histórica do MEF e da CFD	6
2.2.4 Computação Científica no Navegador	8
2.3 Trabalhos Relacionados	8
2.3.1 Transferência de Calor e Condução em Componentes	8
2.3.2 Escoamento e CFD pela Formulação Vorticidade-Corrente	9
2.3.3 Comparação de Métodos e Fundamentação Variacional	10
2.4 Posicionamento do Trabalho e Contribuições	10
3 Fundamentação Teórica	12
3.1 Introdução ao Capítulo	12

3.2	Equações Governantes	12
3.2.1	Condução e Convecção Térmica	13
3.2.2	Escoamento de Fluidos (Navier-Stokes)	13
3.3	Transição Discreta: O Método das Diferenças Finitas (MDF)	14
3.4	O Método dos Elementos Finitos (MEF)	14
3.4.1	Da Forma Forte à Forma Fraca	15
3.4.2	Condições de Contorno	16
3.4.3	O Método de Galerkin	18
3.4.4	Elementos Triangulares Lineares	19
3.4.5	Matrizes Elementares	20
3.4.6	Montagem Global	21
3.5	Formulação Vorticidade-Corrente	21
3.5.1	Definições	21
3.5.2	Equação de Transporte da Vorticidade	22
3.5.3	Sistema Acoplado e Solução Iterativa	22
3.5.4	Matriz de Convecção e Estabilização SUPG	22
3.6	Integração Temporal	23
3.6.1	Método θ Generalizado	23
3.6.2	Esquema Semi-Implicito Adotado	24
3.6.3	Condição de Contorno de Vorticidade na Parede	24
3.7	Síntese do Capítulo	25
4	A Plataforma MinervaMesh	26
4.1	Visão Geral da Plataforma	26
4.2	Justificativa Tecnológica	28
4.2.1	PyScript e WebAssembly	28
4.2.2	Computação <i>Client-Side</i>	29
4.2.3	Comparação com Alternativas	29
4.2.4	Análise Computacional	30
4.3	Arquitetura de Software	33
4.3.1	Organização de Diretórios	33
4.3.2	Fluxo de Execução de uma Simulação	34
4.4	Limitações da Plataforma	34

4.5	Síntese do Capítulo	37
5	Implementação dos Módulos Numéricos	38
5.1	Módulos de Simulação MEF	39
5.1.1	Condução Permanente 1D	39
5.1.2	Condução Transiente 1D com $k(x)$ Variável	40
5.1.3	Convecção-Difusão 1D com Estabilização SUPG	41
5.1.4	Viga 1D — Análise de Flexão (Euler-Bernoulli)	42
5.1.5	Vibração 1D — Problema de Autovalor	43
5.1.6	Transferência de Calor 2D	44
5.1.7	Escoamento Potencial 2D	45
5.1.8	Navier-Stokes 2D (Vorticidade-Corrente)	46
5.2	Módulos Complementares em MDF	48
5.3	Síntese do Capítulo	49
6	Resultados e Validação	50
6.1	Introdução	50
6.2	Condução Permanente 1D	52
6.2.1	Formulação do Problema	52
6.2.2	Solução Analítica	52
6.2.3	Solução MEF e Comparação	53
6.2.4	Análise de Erro	53
6.3	Condução Transiente 1D com $k(x)$ Variável	55
6.3.1	Formulação do Problema	55
6.3.2	Solução Analítica de Regime Permanente	55
6.3.3	Solução MEF e Comparação	56
6.3.4	Análise de Erro	57
6.4	Convecção-Difusão 1D com Estabilização SUPG	58
6.4.1	Formulação do Problema	58
6.4.2	Solução Analítica	59
6.4.3	Solução MEF com SUPG	59
6.4.4	Análise de Erro	60
6.5	Viga 1D — Flexão de Euler-Bernoulli	62

6.5.1	Formulação do Problema	62
6.5.2	Solução Analítica	63
6.5.3	Solução MEF e Comparação	63
6.5.4	Análise de Erro	64
6.6	Vibração 1D — Problema de Autovalor	66
6.6.1	Formulação do Problema	66
6.6.2	Solução Analítica	66
6.6.3	Solução MEF e Comparação	67
6.6.4	Análise de Erro	67
6.7	Transferência de Calor Transiente 2D	69
6.7.1	Formulação do Problema	70
6.7.2	Solução Analítica de Regime Permanente	71
6.7.3	Solução MEF e Comparação	71
6.7.4	Análise de Erro	71
6.8	Escoamento Potencial 2D em Torno de Cilindro	73
6.8.1	Formulação do Problema	74
6.8.2	Solução Analítica em Domínio Infinito	74
6.8.3	Solução MEF e Comparação	75
6.8.4	Análise de Erro	75
6.9	Navier-Stokes 2D — Cavidade com Tampa Móvel	77
6.9.1	Formulação do Problema	78
6.9.2	Benchmark de Referência	78
6.9.3	Solução MEF e Comparação	79
6.9.4	Análise de Erro	80
7	Conclusões	83
7.1	Conclusões	83
7.2	Trabalhos Futuros	85
	Referências Bibliográficas	87
A	Listagens Representativas dos Módulos de Simulação	93
A.1	Módulo 1D Completo: Condução Permanente	94
A.2	Núcleo Numérico dos Módulos 2D	98

B	Scripts de Benchmark Computacional	104
B.1	Solver de Condução Permanente 1D Adaptado	104
B.2	Solver de Transcalor 2D Adaptado	106
B.3	Biblioteca Auxiliar de Cronometragem	110
B.4	Runner Local (CPython)	113
B.5	Runner no Navegador (PyScript)	116

Lista de Figuras

4.1	Tela inicial do <i>MinervaMesh</i> com o menu de simulações disponíveis. .	28
4.2	Escalabilidade do tempo total do solver com o número de nós, em escala log-log, comparando CPython nativo (linha sólida) e Pyodide no navegador (linha tracejada). À esquerda, condução 1D MEF (denso) com inclinação compatível com $\mathcal{O}(N^3)$; à direita, transcalor 2D MEF (esparso, 50 passos) com inclinação próxima de $\mathcal{O}(N^{1,2})$	32
5.1	Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.	40
5.2	Evolução temporal do campo de temperatura com condutividade variável $k(x)$	41
5.3	Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.	42
5.4	Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carregamento, propriedades, momento fletor e esforço cortante.	43
5.5	Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.	44
5.6	Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).	45
5.7	Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.	46
5.8	Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.	48

6.1	Condução permanente 1D: comparação entre a solução MEF (pontos azuis, $n_e = 10$ elementos lineares) e a solução analítica (linha tracejada vermelha). A sobreposição exata confirma a propriedade de reprodução polinomial dos elementos lineares para fontes uniformes.	54
6.2	Condução transiente 1D com condutividade descontínua ($k_1 = 5$, $k_2 = 1$): solução MEF no instante final $t = 0,2$ s (pontos azuis) comparada ao perfil analítico de regime permanente (linha tracejada vermelha). A curva verde tracejada no eixo secundário indica a variação de $k(x)$ com a descontinuidade em $x = L/2$. A malha de 40 elementos lineares com $\theta = 1/2$ reproduz fielmente o <i>kink</i> de condutividade, com $T(L/2) \approx 1/6$	57
6.3	Convecção-difusão 1D com SUPG ativo em $Pe_e = 0,5$. Painel superior: temperatura $T(x)$ — pontos azuis para o MEF e linha tracejada vermelha para a solução analítica, visualmente coincidentes em toda a malha. Painel inferior: erro absoluto $ T^{\text{MEF}} - T^{\text{ana}} $ em escala logarítmica, com máximo $4,44 \times 10^{-16}$ °C — precisão de máquina.	60
6.4	Convecção-difusão 1D com Galerkin clássico (SUPG desativado), mesmos parâmetros da figura anterior. O painel superior mostra ainda concordância visual em escala linear; o painel inferior de erro revela, entretanto, crescimento exponencial do desvio em direção à camada-limite ($x \rightarrow L$), atingindo máximo $3,39 \times 10^{-2}$ °C próximo ao contorno direito.	61
6.5	Viga simplesmente apoiada com carga uniforme: (i) deflexão $w(x)$ com MEF (pontos azuis) e analítica (linha vermelha) visualmente coincidentes; (ii) perfil da carga distribuída constante $q = 10$ kN/m; (iii) propriedades materiais E e I uniformes ao longo do vão; (iv) momento fletor $M(x)$ parabólico com máximo $qL^2/8 = 5,0$ kN·m no centro; (v) esforço cortante $V(x)$ linear decrescente de $+qL/2 = +10$ kN no apoio esquerdo a -10 kN no direito. M e V são obtidos por pós-processamento da deflexão nodal.	65

6.6	Cinco primeiros modos de vibração axial da barra livre-livre. Tabela superior: frequências MEF e analíticas com erro relativo. Painéis seguintes: forma modal normalizada (pontos e linha contínua, uma cor por modo), do modo 1 rígido ($f_1 = 0$ Hz, amplitude uniforme) aos modos 2–5 com número crescente de meio-comprimentos de onda. Painel inferior: propriedades materiais (E e ρ) constantes ao longo do vão.	68
6.7	Transferência de calor transiente 2D com os <i>defaults</i> do módulo, em $t = 0,049$ s: o campo $T(x, y)$ mostra a perturbação térmica propagando-se a partir da parede aquecida (topo, $T_{\text{top}} = 100^\circ\text{C}$) e da parede morna (base, $T_{\text{bot}} = 30^\circ\text{C}$) para o interior da placa, ainda inicialmente em $T = 0^\circ\text{C}$. O <i>colormap</i> jet varre a faixa de temperatura instantânea e exhibe a natureza bidimensional da difusão: a frente de calor do topo penetra mais rapidamente que a da base devido ao maior gradiente ΔT	72
6.8	Escoamento potencial em torno de cilindro, três painéis empilhados: (i) função de corrente $\psi(x, y)$ com a malha triangular sobreposta, destacando o obstáculo circular no centro; (ii) linhas de corrente contornando o cilindro, coloridas pela magnitude de velocidade (<i>colormap</i> viridis) — simetria aproximada do escoamento visível; (iii) distribuição de $C_p(\theta)$ sobre a superfície do cilindro, comparando a solução numérica (azul) à teoria potencial $C_p = 1 - 4 \sin^2 \theta$ (vermelho tracejado). Os pontos de estagnação frontal ($\theta = 0$) e traseiro ($\theta \approx \pm\pi$) são reproduzidos com pequeno desvio; os polos de pressão mínima próximos de $\theta = \pm 90^\circ$ exibem assimetria devida ao confinamento lateral do domínio ($2r/H = 30\%$) e ao viés da malha estruturada. . .	76
6.9	Cavidade com tampa móvel a $\text{Re} = 100$: campos ψ , ω e linhas de corrente.	81
6.10	Perfil $u(y)$ em $x = 0,5$ comparado aos valores tabulados por Ghia et al. para a cavidade a $\text{Re} = 100$	82

Lista de Tabelas

3.1	Classificação variacional e tratamento numérico das condições de contorno no MEF.	17
4.1	Comparação entre plataformas de simulação numérica para ensino. . .	30
4.2	Tempo total do solver (mediana, com MAD reportado abaixo de 1% em todas as células) em CPython nativo e Pyodide (navegador), na mesma máquina (Apple M1 Max). t_{mont} é o tempo de montagem (incluindo geração de malha quando aplicável) e t_{slv} é o tempo da resolução linear (<code>np.linalg.solve</code> no caso 1D denso, ou o total dos passos de <code>spsolve</code> nos casos 2D esparsos). “CP” = CPython nativo; “Py” = Pyodide.	31
6.1	Visão geral dos casos de validação apresentados neste capítulo.	51
6.2	Parâmetros da simulação de condução permanente 1D.	52
6.3	Parâmetros da simulação de condução transiente 1D com $k(x)$ variável. 56	
6.4	Parâmetros da simulação de convecção-difusão 1D.	59
6.5	Parâmetros da simulação de flexão de viga 1D.	63
6.6	Parâmetros da simulação de vibração axial 1D (barra de aço bi-engastada).	67
6.7	Frequências naturais: MEF vs. analítica para barra livre-livre.	69
6.8	Parâmetros da simulação de transferência de calor transiente 2D. . . .	70
6.9	Parâmetros da simulação de escoamento potencial 2D em torno de cilindro.	74
6.10	Coefficiente de pressão C_p na superfície do cilindro: MEF vs. teoria potencial.	77

6.11	Parâmetros da simulação de Navier-Stokes 2D — cavidade com tampa móvel.	79
A.1	Localização de cada módulo de simulação no repositório do <i>MinervaMesh</i> . Caminhos relativos ao diretório <code>simulations/</code> na raiz do repositório.	94

Lista de Abreviaturas

CFD	<i>Computational Fluid Dynamics</i> (Dinâmica dos Fluidos Computacional), p. 1
MDF	Método das Diferenças Finitas, p. 1
MEF	Método dos Elementos Finitos, p. 1
SUPG	<i>Streamline Upwind/Petrov-Galerkin</i> , p. 1
UFRJ	Universidade Federal do Rio de Janeiro, p. 1
WASM	WebAssembly, p. 1

Capítulo 1

Introdução

1.1 Introdução

A simulação numérica consolidou-se como instrumento central da engenharia contemporânea, complementando abordagens teóricas e experimentais na investigação de fenômenos físicos complexos. No contexto da Engenharia Mecânica, a Dinâmica dos Fluidos Computacional (*Computational Fluid Dynamics* – CFD) e a modelagem de transferência de calor viabilizam a análise de problemas governados por Equações Diferenciais Parciais (EDPs), muitas vezes inviáveis por solução analítica direta ou por experimentação em escala de laboratório [1]. Essa capacidade preditiva tem papel relevante no dimensionamento, na otimização e na verificação de desempenho de sistemas térmicos e fluidodinâmicos.

Apesar dos avanços metodológicos, o acesso a ambientes de simulação permanece limitado por barreiras econômicas e operacionais. Soluções comerciais amplamente utilizadas exigem licenciamento oneroso, infraestrutura computacional específica e configuração de ambiente com elevado custo de entrada, sobretudo em cenários de ensino e pesquisa inicial. Como consequência, a formação em métodos numéricos frequentemente fica condicionada à disponibilidade de laboratório e suporte técnico especializado, o que reduz a reprodutibilidade e a acessibilidade das atividades acadêmicas [2].

Neste contexto, este trabalho ataca o problema de oferecer um ambiente de simulação numérica simultaneamente **acessível**, **didático** e **transparente** para o

ensino de Engenharia Mecânica. A resposta proposta é o *MinervaMesh*, uma plataforma de código aberto executada integralmente no navegador, com arquitetura *client-side*, que combina Python científico com tecnologias web modernas — notadamente *WebAssembly* e *PyScript* — para executar localmente rotinas de MEF e MDF sem qualquer instalação de dependências pelo usuário: o PyScript [3] executa, no próprio navegador, a pilha científica do Python, incluindo o NumPy [4]. A proposta articula dois aspectos sob esse mesmo objetivo: de um lado, a *implementação e a verificação* dos métodos numéricos, confrontadas com soluções analíticas e *benchmarks* de referência; de outro, a *disponibilização* dessas simulações na web, com formulação matemática explícita e código-fonte inspecionável em cada caso. Os dois aspectos não constituem trabalhos paralelos, mas meios complementares para reduzir a barreira de acesso ao aprendizado de métodos numéricos — a correteza dos métodos dá fundamento à plataforma, e a entrega na web a torna efetivamente utilizável por terceiros.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver e validar o **MinervaMesh**, uma plataforma web de código aberto¹ para solução de EDPs de interesse em Engenharia Mecânica. A plataforma executa as simulações diretamente no navegador, operando integralmente no lado do cliente (*client-side*), e preserva a transparência dos algoritmos e o caráter didático de cada simulação, de modo a poder ser utilizada por alunos desta instituição e de outras.

Os objetivos específicos são:

1. Implementar, como núcleo numérico da plataforma, métodos clássicos — com ênfase no Método dos Elementos Finitos (MEF) e uso complementar do Método das Diferenças Finitas (MDF) —, empregando bibliotecas científicas Python no ambiente web [4].
2. Desenvolver uma interface interativa para pré-processamento, solução e pós-processamento, incluindo parametrização de malha e visualização de campos

¹Disponível em <https://github.com/lgsfarias/minervamesh>, sob licença MIT.

de interesse.

3. Validar a plataforma, prioritariamente, por meio de casos com solução analítica conhecida e, em seguida, demonstrar sua aplicação em casos adicionais de engenharia, com foco em mecânica dos fluidos e transferência de calor.
4. Consolidar um fluxo de trabalho reprodutível para problemas de:
 - Escoamentos potenciais;
 - Problemas de advecção-difusão (com estabilização SUPG) [5];
 - Equações de Navier-Stokes para escoamentos laminares incompressíveis.

1.3 Justificativa

A relevância do trabalho reside na articulação entre rigor numérico e acessibilidade computacional. Ao viabilizar simulações diretamente no navegador, reduz-se o atrito inicial de aprendizado e amplia-se o potencial de uso em disciplinas de graduação, monitorias e estudos independentes. Adicionalmente, a explicitação da formulação matemática e de sua implementação computacional fortalece o caráter acadêmico da proposta, permitindo que a plataforma seja utilizada tanto como ferramenta de simulação quanto como objeto de estudo em métodos numéricos [6].

No espectro de ferramentas disponíveis para o ensino e a prototipagem de métodos numéricos, o *MinervaMesh* ocupa um nicho distinto das alternativas predominantes. Bibliotecas robustas de MEF em Python, como o FEniCS, e plataformas especializadas em métodos espectrais, como o Dedalus, oferecem grande flexibilidade e rigor, mas exigem instalação local de ambientes científicos e familiaridade com suas linguagens de domínio; suítes comerciais como COMSOL Multiphysics e ANSYS entregam interfaces gráficas de alto nível, porém seu código é proprietário e seu licenciamento restringe o uso acadêmico autônomo; e demonstrações web em *p5.js* ou JavaScript puro, embora acessíveis, raramente apresentam formulação matemática explícita ou código inspecionável em linguagem científica consolidada. A contribuição original do *MinervaMesh* reside na combinação simultânea de três atributos: execução integralmente no navegador sem qualquer instalação, código-fonte Python aberto e ins-

peçonável em cada simulação diretamente na página, e portfólio unificado cobrindo três domínios clássicos da Engenharia Mecânica em uma única interface consistente. A distribuição sob licença livre (MIT) completa esse posicionamento: o estudo, a modificação e a extensão da plataforma ficam franqueados a estudantes e docentes de qualquer instituição, sem custo de licenciamento.

1.4 Organização do Trabalho

Este trabalho está estruturado da seguinte forma:

- O **Capítulo 2** (Revisão Bibliográfica) apresenta o estado da arte dos temas que sustentam o trabalho, os trabalhos relacionados do mesmo grupo de pesquisa e o posicionamento deste trabalho com suas contribuições.
- O **Capítulo 3** (Fundamentação Teórica) apresenta o embasamento matemático necessário para a discretização de EDPs via MEF.
- O **Capítulo 4** (A Plataforma MinervaMesh) apresenta a plataforma enquanto software de entrega: a arquitetura *client-side*, as tecnologias PyScript e *WebAssembly*, a análise de desempenho e as limitações do paradigma.
- O **Capítulo 5** (Implementação dos Módulos Numéricos) detalha como cada módulo de simulação traduz as formulações do Capítulo 3 em código executado no navegador.
- O **Capítulo 6** (Resultados e Validação) apresenta a verificação numérica dos métodos implementados, comparando as soluções discretizadas com soluções analíticas conhecidas e *benchmarks* consagrados da literatura.
- O **Capítulo 7** (Conclusão) resume as contribuições e propõe trabalhos futuros.

Capítulo 2

Revisão Bibliográfica

2.1 Introdução ao Capítulo

Este capítulo apresenta a revisão bibliográfica que fundamenta o desenvolvimento do *MinervaMesh*, organizada em três blocos: o **estado da arte** dos temas que sustentam o trabalho — problemas térmicos e de escoamento, métodos numéricos para EDPs, sua consolidação histórica e a emergência da computação científica no navegador; os **trabalhos relacionados** do mesmo grupo de pesquisa, que estabelecem a tradição de implementação em Python científico na qual este trabalho se insere; e, por fim, o **posicionamento do trabalho**, com a lacuna identificada e a enumeração explícita das contribuições.

2.2 Estado da Arte

2.2.1 Problemas Térmicos e de Escoamento

A base física dos problemas tratados pela plataforma é a da transferência de calor e da mecânica dos fluidos. A condução é descrita pela Lei de Fourier e pela equação da difusão térmica — que em regime estacionário se reduz às equações de Laplace e Poisson —, com tratamento de referência em engenharia consolidado por Incropera et al. [7] e, em registro mais introdutório, por Çengel e Ghajar [8]. Quando há interação sólido-fluido, a convecção passa a governar a taxa de remoção de calor e exige o acoplamento entre as equações de energia e de escoamento: os fundamentos físicos desse acoplamento vão dos conceitos introdutórios de Fox et al. [9] à análise

de escoamentos viscosos de White [10]. A solução numérica conjunta de calor e fluidos é desenvolvida por Maliska [1] e apresentada didaticamente por Versteeg e Malalasekera [11]. A radiação térmica, relevante no balanço global em aplicações de alta temperatura, permanece fora do escopo deste trabalho e é considerada extensão natural para desenvolvimentos futuros.

2.2.2 Métodos Numéricos para EDPs

Dois métodos de discretização concentram a literatura de interesse. O MDF aproxima derivadas por expansões de Taylor sobre malhas estruturadas e, por sua implementação direta, é frequentemente adotado em validações iniciais e na discretização temporal de problemas transientes [12]; em problemas dominados por difusão, esquemas implícitos como o de Crank-Nicolson são preferidos pela robustez numérica [11]. O MEF, por sua vez, oferece flexibilidade geométrica por meio de malhas não estruturadas e de uma formulação variacional, característica decisiva em domínios complexos; seu desenvolvimento e fundamentação rigorosa são consolidados no tratado de referência de Zienkiewicz et al. [13]. A dedução da formulação de Galerkin — que transforma a EDP na forma fraca e conduz à montagem das matrizes globais de rigidez, massa e convecção — é apresentada em detalhe por Reddy [14]; a aplicação do método a problemas acoplados de transferência de calor e escoamento é tratada por Lewis et al. [15]; e textos introdutórios como o de Fish e Belytschko [6] mostram que, para elementos triangulares lineares, a implementação resultante é eficiente e adequada a plataformas computacionais didáticas. O desenvolvimento matemático desses métodos, na forma empregada pela plataforma, é objeto do Capítulo 3.

2.2.3 Consolidação Histórica do MEF e da CFD

O desenvolvimento do MEF resulta de uma sucessão de contribuições distintas, e não de um marco único. Galerkin [16] introduziu o método dos resíduos ponderados que leva seu nome, no qual a solução aproximada é obtida impondo a ortogonalidade do resíduo às próprias funções de base, dispensando a existência prévia de um princípio variacional. Courant [17] formalizou o emprego de funções de interpolação contínuas por partes sobre subdomínios triangulares na solução de problemas de equilíbrio e vibração, antecipando o conceito de elemento. Hrennikoff [18], por sua

vez, propôs o *framework method*, no qual o meio contínuo elástico é discretizado por uma treliça de barras com propriedades equivalentes — um precursor direto da ideia de discretização do domínio. A estruturação do MEF moderno aplicado à engenharia veio com Turner et al. [19], que formularam a análise matricial de rigidez de estruturas aeronáuticas complexas a partir de elementos triangulares e retangulares, e com Clough [20], a quem se atribui a cunhagem do próprio termo *método dos elementos finitos*.

No tratamento do transporte convectivo e das equações de Navier-Stokes, a literatura desenvolveu formulações estabilizadas para suprimir as oscilações espúrias da formulação de Galerkin padrão em regime de convecção dominante. Christie et al. [21] propuseram funções de peso assimétricas (*upwind*) para equações de segunda ordem com termos de primeira derivada significativos, um dos primeiros tratamentos consistentes do problema. Brooks e Hughes [5] generalizaram essa ideia na formulação SUPG (*Streamline Upwind/Petrov-Galerkin*), que adiciona difusão artificial apenas na direção das linhas de corrente e tornou-se referência para escoamentos incompressíveis. Donea [22] introduziu o método Taylor-Galerkin, que deriva o esquema estabilizado a partir de uma expansão de Taylor no tempo, obtendo baixa difusão numérica em problemas de advecção transiente. Löhner et al. [23] estenderam essa abordagem a sistemas hiperbólicos não lineares, realizando a discretização temporal antes da espacial para recuperar uma forma auto-adjunta compatível com o processo de Galerkin.

A validação de solucionadores de escoamento apoia-se em casos de referência consagrados, cada um associado a um regime distinto. Ghia et al. [24] produziram, por método multigrid, soluções de alta resolução para a cavidade com tampa móvel (*lid-driven cavity*) em números de Reynolds de até 10.000, estabelecendo o *benchmark* mais difundido para esse problema; Marchi et al. [25] refinaram posteriormente esse padrão com soluções em malha 1024×1024 e extrapolação de Richardson. Para o degrau regressivo (*backward-facing step*), Armaly et al. [26] forneceram medições experimentais por anemometria laser-Doppler acompanhadas de análise numérica, enquanto Thomas et al. [27] apresentaram uma das primeiras análises dessa mesma geometria por elementos finitos. No campo dos algoritmos, Pironneau [28] formali-

zou o método de transporte-difusão baseado em características, com aplicação direta às equações de Navier-Stokes. Também se destaca a relevância das ferramentas de geração de malha na qualidade da solução, com destaque para o Gmsh em fluxos de trabalho acadêmicos e de pesquisa [29].

2.2.4 Computação Científica no Navegador

Mais recentemente, a maturidade do *WebAssembly* abriu caminho para a execução de código científico diretamente no navegador, sem instalação local. O projeto PyScript [3] disponibiliza em ambiente web o interpretador CPython e parte de sua pilha científica — incluindo o NumPy, cujo modelo de programação vetorial é descrito por Harris et al. [4]. Ferramentas consolidadas de pré-processamento começam a percorrer o mesmo caminho: o `triangle-wasm` [30] demonstra a viabilidade de portar geradores de malha clássicos para *WebAssembly*. No contexto de formação em engenharia, o material didático de Anjos [2] sistematiza o emprego do Python científico na solução de problemas de engenharia, evidenciando tanto o potencial pedagógico dessa pilha quanto a barreira de configuração de ambiente que este trabalho se propõe a eliminar.

2.3 Trabalhos Relacionados

Em projetos de graduação recentes do mesmo grupo de pesquisa, observa-se forte convergência para a implementação dos métodos numéricos em Python científico aplicada a problemas térmicos e de escoamento. Em vez de agrupá-los, os trabalhos mais próximos são resumidos individualmente a seguir, pois cada um aborda um problema físico e uma estratégia de implementação distintos.

2.3.1 Transferência de Calor e Condução em Componentes

Aguiar [31] desenvolveu uma rotina em Python para o dimensionamento térmico preliminar de discos de freio, resolvendo a equação axissimétrica de condução de calor por MEF e validando o código por convergência de malha e confronto com soluções analíticas. Alves [32] estendeu o problema do disco de freio para a equação tridimensional transiente do calor, também por MEF, com ênfase em oferecer alternativa de

código aberto aos pacotes comerciais para comparar materiais e geometrias. Ferreira [33] aplicou o MEF em Python à análise térmica de processadores computacionais, contrastando o comportamento térmico sob diferentes discretizações em condições críticas de operação. Pinheiro [34] tratou a condução de calor em componentes eletrônicos de material heterogêneo, modelando a dissipação resistiva como geração interna e resolvendo o problema bidimensional por elementos finitos.

Capitanio [35] combinou MEF (Galerkin no espaço) e MDF (no tempo) para o estudo paramétrico de dissipadores de processadores, validando a metodologia por solução manufaturada, com erro quadrático médio da ordem de 10^{-4} a 10^{-6} conforme o refino de malha. Miranda [36] investigou geometrias de canais em placas frias para arrefecimento de eletrônicos, acoplando transferência de calor e escoamento pela formulação função-corrente-vorticidade e validando contra o escoamento de Poiseuille. Teixeira [37] resolveu a equação de Laplace tridimensional para a temperatura em um bloco de motor, empregando uma cadeia de ferramentas integralmente de código aberto (FreeCAD, Gmsh, Python/Julia) e comparando fluidos de arrefecimento. Oliveira [38] otimizou o número de aletas de um dissipador de calor de cobre por MEF em Python, validando o solver contra solução analítica com erro relativo da ordem de 10^{-11} .

2.3.2 Escoamento e CFD pela Formulação Vorticidade-Corrente

Na linha de mecânica dos fluidos, há forte convergência para a formulação vorticidade-corrente das equações de Navier-Stokes implementada em Python. Florentino [39] aplicou essa formulação, com estabilização Taylor-Galerkin, ao arrefecimento de eletrônicos, validando o código em casos clássicos (Poiseuille, cilindro em canal) antes de simular canais aletados. Santos [40] empregou a mesma formulação para investigar os campos de velocidade, vorticidade e função corrente em escoamento livre ao redor de diferentes geometrias, com a verificação apoiada nos escoamentos de Poiseuille e na cavidade com tampa móvel. Silva [41] acoplou a formulação vorticidade-corrente a uma descrição arbitrária lagrangiana-euleriana (ALE) para simular as oscilações de um cilindro sob escoamento transversal, abor-

dando a interação fluido-estrutura. Amaral [42] e Gomes [43] estenderam a formulação ao transporte convectivo-difusivo de espécie química em artérias coronárias — com ênfase, respectivamente, nas diferentes geometrias de *stent* farmacológico e nas configurações de restrição de fluxo —, evidenciando a necessidade de esquemas estabilizados em regime de convecção dominante.

2.3.3 Comparação de Métodos e Fundamentação Variacional

Dois trabalhos tratam diretamente da comparação entre métodos e da fundamentação matemática do MEF. Araujo [44] comparou solução analítica, MDF e MEF em problemas bidimensionais de condução de calor de complexidade crescente — até o caso transiente com geração de calor, sem solução fechada — caracterizando o comportamento do erro com o refinamento de malha; é a referência mais próxima do contraste $\text{MDF} \times \text{MEF}$ retomado neste trabalho. Gomes [45] apresentou a análise variacional da equação de Poisson em transferência de calor, estabelecendo existência e unicidade da solução fraca pelo teorema de Lax-Milgram e validando a discretização de Galerkin (MEF em Python) por solução manufaturada, com erro relativo de aproximadamente 0,46%.

Em conjunto, esses trabalhos consolidam, no mesmo grupo de pesquisa, uma tradição de implementação dos métodos numéricos em Python científico para problemas térmicos e de escoamento. Todos, contudo, pressupõem a instalação local do ambiente Python e a execução em *desktop*; nenhum disponibiliza as simulações de forma interativa na web — lacuna sobre a qual o *MinervaMesh* se posiciona.

2.4 Posicionamento do Trabalho e Contribuições

A revisão indica que: (i) há base teórica consolidada para problemas térmicos e de escoamento; (ii) MDF e MEF são métodos complementares, com o MEF mais adequado para geometrias complexas; e (iii) existe demanda por ferramentas didáticas acessíveis para implementação e validação desses métodos. Os trabalhos relacionados confirmam a maturidade da implementação desses métodos em Python ci-

entífico dentro do próprio grupo de pesquisa, mas nenhum deles elimina a barreira de instalação e configuração de ambiente: as simulações permanecem confinadas ao *desktop* de quem as desenvolveu.

É exatamente nessa lacuna que este trabalho se enquadra: levar a tradição de implementação aberta em Python científico do grupo para um meio de entrega sem barreira de acesso — o navegador —, apoiando-se na maturidade recente do *WebAssembly* descrita na Seção 2.2.4. Frente a esse panorama, as contribuições deste trabalho são:

- o desenvolvimento da plataforma *MinervaMesh*, ambiente web de código aberto para simulação numérica em Engenharia Mecânica;
- a execução integral das simulações no navegador, via *WebAssembly* e *PyScript*, sem instalação nem licenciamento;
- a disponibilização de código Python aberto e inspecionável em cada simulação, preservando a transparência dos algoritmos;
- a implementação de módulos baseados em MEF e em MDF, cobrindo os domínios térmico, estrutural e fluidodinâmico;
- a validação dos módulos frente a soluções analíticas e a *benchmarks* clássicos da literatura;
- a aplicação voltada ao ensino de Engenharia Mecânica, para uso por alunos desta instituição e de outras.

Capítulo 3

Fundamentação Teórica

3.1 Introdução ao Capítulo

Este capítulo apresenta a formulação matemática e os métodos numéricos que fundamentam as análises realizadas no *MinervaMesh*. O objetivo central é fornecer uma base sólida, típica da Engenharia Mecânica, para a conversão dos modelos físicos contínuos em sistemas algébricos discretos resolvidos computacionalmente.

Inicialmente, revisam-se as equações diferenciais parciais (EDPs) governantes da condução de calor e do escoamento de fluidos de forma conceitual. Em seguida, após uma breve fundamentação do Método das Diferenças Finitas (MDF), o capítulo se aprofunda extensamente no Método dos Elementos Finitos (MEF). O desenvolvimento abrange desde a formulação de Galerkin, passando pela discretização com elementos triangulares lineares, até a consolidação da formulação em Vorticidade-Corrente adotada para a dinâmica de fluidos.

3.2 Equações Governantes

A modelagem térmica e fluidodinâmica sustenta-se em princípios de conservação: massa, quantidade de movimento e energia. Estas leis físicas, contínuas no espaço e no tempo, são expressas por EDPs que determinam o comportamento termofluidodinâmico do meio contínuo abordado. Adota-se ao longo deste trabalho a convenção de denotar grandezas vetoriais em negrito (e.g., $\mathbf{v}$, $\mathbf{f}_b$) e suas componentes escalares

em itálico (e.g., $\mathbf{v} = (u, v)$, com u e v associadas às direções x e y , respectivamente).

3.2.1 Condução e Convecção Térmica

A transferência de calor em meios sólidos contínuos é regida pela equação da difusão de calor (Lei de Fourier acoplada à conservação da energia térmica). Para um meio estacionário, a distribuição de temperatura $T(\mathbf{x}, t)$ é ditada por [7]:

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + q''' \quad (3.1)$$

onde ρ é a massa específica, c_p é o calor específico a pressão constante, k é a condutividade térmica do material, e q''' representa a taxa volumétrica de geração de energia térmica. Para regimes permanentes e propriedades constantes, a Equação 3.1 se simplifica na clássica equação de Poisson (ou Laplace, se $q''' = 0$).

Quando existe escoamento (*convecção*), adiciona-se à equação o transporte advectivo promovido pelo campo de velocidades $\mathbf{v}$:

$$\rho c_p \left(\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T \right) = \nabla \cdot (k \nabla T) + q''' \quad (3.2)$$

Esta é a equação de convecção-difusão, paradigma central para o estudo da transferência de calor conjugada ou escoamentos não-isotérmicos.

3.2.2 Escoamento de Fluidos (Navier-Stokes)

O escoamento de fluidos incompressíveis newtonianos é regido pela restrição de continuidade (conservação de massa) e pelas Equações de Navier-Stokes (conservação do momento linear) [10]:

$$\nabla \cdot \mathbf{v} = 0 \quad (3.3)$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \mu \nabla^2 \mathbf{v} + \mathbf{f}_b \quad (3.4)$$

onde p representa o campo de pressão estática, μ a viscosidade dinâmica do fluido e $\mathbf{f}_b$ eventuais forças de campo (como as forças de empuxo gravitacional). Esta formulação em variáveis primitivas (velocidade-pressão) apresenta consideráveis desafios numéricos devido ao acoplamento embutido pela restrição de continuidade

[1]. Consequentemente, abordagens numéricas como a Vorticidade-Corrente são frequentemente adotadas em 2D para contornar a incógnita da pressão diretamente.

3.3 Transição Discreta: O Método das Diferenças Finitas (MDF)

Antes do extensivo emprego de formulações variacionais associadas a malhas arbitrárias, a conversão de EDPs a um sistema matricial utilizou maciçamente as propriedades da expansão da série de Taylor. O Método das Diferenças Finitas (MDF) consiste na substituição direta dos operadores diferenciais exatos pelas respectivas aproximações algébricas discretizadas em nós de uma malha tipicamente ortogonal e estruturada [2].

Considere a representação da derivada parcial aproximada pelo método da diferença adiantada e atrasada para aproximações de derivadas espaciais e temporais:

$$\left. \frac{\partial u}{\partial x} \right|_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta x} \quad (\text{Diferença Central}) \quad (3.5)$$

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2} \quad (3.6)$$

Embora de inegável valor histórico e de extrema eficácia para discretizações explícitas no tempo e problemas de topologia regular simples, o MDF apresenta entraves práticos quando restrições de domínios geométricos altamente complexos exigem representações intrincadas de fronteiras, resultando em aproximações em escada (*stair-step*) dos contornos físicos do domínio fluido ou de sólidos.

Para permitir tratamento sistemático de geometrias complexas e de condições de contorno de Neumann ou mistas, o trabalho adota a seguir a formulação integral característica do Método dos Elementos Finitos (MEF).

3.4 O Método dos Elementos Finitos (MEF)

O Método dos Elementos Finitos constitui a abordagem numérica central adotada neste trabalho. Diferentemente do MDF, o MEF parte de uma formulação

integral (variacional ou de resíduos ponderados) da EDP, permitindo o uso de malhas não estruturadas que se adaptam naturalmente a geometrias complexas [13]. Esta seção desenvolve a formulação desde a forma forte até a obtenção do sistema algébrico global.

3.4.1 Da Forma Forte à Forma Fraca

Considere, para fins de generalidade, uma EDP elíptica em um domínio $\Omega \subset \mathbb{R}^d$ com fronteira $\Gamma = \partial\Omega$. De maneira genérica, a *forma forte* do problema visa encontrar $u(\mathbf{x})$ tal que:

$$\mathcal{L}(u) = f \quad \text{em } \Omega \quad (3.7)$$

sujeita às condições de contorno:

$$u = \bar{u} \quad \text{em } \Gamma_D \quad (\text{Dirichlet}) \quad (3.8)$$

$$k \frac{\partial u}{\partial n} = \bar{q} \quad \text{em } \Gamma_N \quad (\text{Neumann}) \quad (3.9)$$

onde $\mathcal{L}$ é um operador diferencial (por exemplo, $-\nabla \cdot (k \nabla(\cdot))$ para condução), f é o termo fonte, $\bar{u}$ é o valor prescrito na fronteira de Dirichlet Γ_D , $\bar{q}$ é o fluxo prescrito na fronteira de Neumann Γ_N , e $\Gamma_D \cup \Gamma_N = \Gamma$.

A *forma fraca* (ou variacional) é obtida multiplicando-se a Equação 3.7 por uma *função-teste* (ou função de ponderação) $w \in \mathcal{V}_0$, pertencente a um espaço funcional adequado, e integrando-se sobre o domínio Ω :

$$\int_{\Omega} w \mathcal{L}(u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.10)$$

A vantagem fundamental desta operação reside na possibilidade de aplicar o Teorema de Green (integração por partes generalizada), transferindo uma ordem de derivação do operador diferencial $\mathcal{L}$ para a função-teste w . Considere o caso concreto da equação de Poisson estacionária ($-\nabla \cdot (k \nabla u) = f$). Multiplicando por w e integrando:

$$-\int_{\Omega} w \nabla \cdot (k \nabla u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.11)$$

Aplicando o Teorema de Green ao lado esquerdo:

$$\int_{\Omega} k \nabla w \cdot \nabla u \, d\Omega - \int_{\Gamma_N} w k \frac{\partial u}{\partial n} \, d\Gamma = \int_{\Omega} w f \, d\Omega \quad (3.12)$$

Substituindo a condição de Neumann (Eq. 3.9) no termo de contorno, obtém-se a *forma fraca*: encontrar $u \in \mathcal{V}$ tal que, para todo $w \in \mathcal{V}_0$:

$$\boxed{\int_{\Omega} k \nabla w \cdot \nabla u \, d\Omega = \int_{\Omega} w f \, d\Omega + \int_{\Gamma_N} w \bar{q} \, d\Gamma} \quad (3.13)$$

Esta expressão é de importância central: ela reduz os requisitos de regularidade sobre u (de C^2 para H^1), incorpora naturalmente as condições de Neumann no funcional, e constitui a base sobre a qual o MEF opera.

3.4.2 Condições de Contorno

A unicidade da solução de uma EDP elíptica ou parabólica depende da prescrição de informações apropriadas em $\Gamma = \partial\Omega$. No contexto da formulação variacional do MEF, as condições de contorno classificam-se em dois grupos: as *essenciais*, que restringem o espaço de soluções admissíveis $\mathcal{V}$, e as *naturais*, que emergem espontaneamente do termo de contorno gerado pela integração por partes (Eq. 3.12) [14]. Os três tipos canônicos — Dirichlet, Neumann e Robin — abrangem a quase totalidade dos problemas de engenharia tratados pelo MEF.

Dirichlet (essencial). Já apresentada na Eq. 3.8, a condição de Dirichlet prescreve diretamente o valor da incógnita em Γ_D — por exemplo, temperatura fixa em parede aquecida, ou condição de não-deslizamento $\mathbf{v} = \mathbf{0}$ em paredes sólidas. Por restringir o espaço admissível $\mathcal{V} = \{u \in H^1(\Omega) : u = \bar{u} \text{ em } \Gamma_D\}$, a função-teste w deve anular-se em Γ_D . Numericamente, sua imposição é feita após a montagem global, zerando-se as linhas e colunas da matriz de rigidez global $\mathbf{K}$ correspondentes aos nós em Γ_D e ajustando o vetor de carga global $\mathbf{F}$ para incorporar o valor prescrito [13] — a construção formal do sistema $\mathbf{KU} = \mathbf{F}$ é desenvolvida na Seção 3.4.3.

Neumann (natural). A condição de Neumann (Eq. 3.9) prescreve o fluxo $k \partial u / \partial n = \bar{q}$ em Γ_N . Sua designação como “natural” reflete o fato de que ela aparece de forma direta no termo de contorno da forma fraca após a integração por partes: basta substituir $\bar{q}$ no integrando $\int_{\Gamma_N} w \bar{q} \, d\Gamma$ (Eq. 3.13) para incorporá-la no vetor de

carga global. O caso particular mais comum é a parede adiabática ($\bar{q} = 0$), em que a integral simplesmente se anula, dispensando qualquer manipulação adicional.

Robin (mista). As condições de Robin (também chamadas de mistas ou de terceiro tipo) combinam o valor da incógnita e seu fluxo na fronteira em uma relação linear:

$$\alpha u + \beta \frac{\partial u}{\partial n} = \gamma \quad \text{em } \Gamma_R \quad (3.14)$$

com α, β, γ coeficientes que parametrizam o tipo de interação física na fronteira. O caso canônico em transferência de calor é a convecção newtoniana, em que o fluxo por condução é igualado ao fluxo convectivo trocado com o meio externo a temperatura T_∞ via coeficiente convectivo h [7]:

$$-k \frac{\partial T}{\partial n} = h (T - T_\infty) \quad \text{em } \Gamma_R \quad (3.15)$$

Substituindo a relação acima no termo de contorno $\int_{\Gamma_R} w k \partial u / \partial n d\Gamma$ da forma fraca, surgem duas contribuições simultâneas: uma na matriz de rigidez — $\int_{\Gamma_R} h N_i N_j d\Gamma$ — e outra no vetor de carga — $\int_{\Gamma_R} h T_\infty N_i d\Gamma$. Robin é, portanto, a única dentre as três que altera simultaneamente $\mathbf{K}$ e $\mathbf{F}$ [14].

A Tabela 3.1 sintetiza a classificação variacional e o impacto numérico de cada tipo.

Tabela 3.1: Classificação variacional e tratamento numérico das condições de contorno no MEF.

Tipo	Forma	Classificação	Impacto em $\mathbf{K}, \mathbf{F}$
Dirichlet	$u = \bar{u}$	Essencial	Modifica $\mathbf{K}$ e $\mathbf{F}$
Neumann	$k \partial u / \partial n = \bar{q}$	Natural	Modifica apenas $\mathbf{F}$
Robin	$\alpha u + \beta \partial u / \partial n = \gamma$	Natural mista	Modifica $\mathbf{K}$ e $\mathbf{F}$

Os módulos da plataforma documentados no Capítulo 5 fazem uso predominante de condições de Dirichlet (temperaturas e velocidades prescritas) e, de forma implícita, de Neumann homogêneas em paredes adiabáticas e fronteiras livres ($\bar{q} = 0$). Condições de Robin não são empregadas nos módulos atuais; sua incorporação — particularmente no módulo de transferência de calor 2D, para representar

troca convectiva com o meio externo — configura-se como extensão natural prevista nas considerações finais do Capítulo 7.

3.4.3 O Método de Galerkin

O método de Galerkin consiste em restringir o problema variacional contínuo (Eq. 3.13) a um subespaço de dimensão finita $\mathcal{V}^h \subset \mathcal{V}$, parametrizado por uma malha de elementos finitos. A solução aproximada u^h é expressa como uma combinação linear de N funções de base $\{N_j\}_{j=1}^N$:

$$u^h(\mathbf{x}) = \sum_{j=1}^N U_j N_j(\mathbf{x}) \quad (3.16)$$

onde U_j são os coeficientes nodais (incógnitas do sistema discreto) e $N_j(\mathbf{x})$ são as funções de forma associadas a cada nó j da malha.

A escolha fundamental do método de Galerkin é adotar as *mesmas* funções de base como funções-teste: $w^h = N_i$, $i = 1, \dots, N$. Substituindo na forma fraca (Eq. 3.13):

$$\sum_{j=1}^N U_j \int_{\Omega} k \nabla N_i \cdot \nabla N_j d\Omega = \int_{\Omega} N_i f d\Omega + \int_{\Gamma_N} N_i \bar{q} d\Gamma \quad \forall i = 1, \dots, N \quad (3.17)$$

Esta expressão define um sistema linear $\mathbf{KU} = \mathbf{F}$, onde:

$$K_{ij} = \int_{\Omega} k \nabla N_i \cdot \nabla N_j d\Omega \quad (\text{Matriz de Rigidez}) \quad (3.18)$$

$$F_i = \int_{\Omega} N_i f d\Omega + \int_{\Gamma_N} N_i \bar{q} d\Gamma \quad (\text{Vetor de Carga}) \quad (3.19)$$

A resolução deste sistema algébrico fornece os coeficientes nodais $\mathbf{U}$, a partir dos quais o campo aproximado u^h é reconstituído via Equação 3.16. A qualidade da aproximação depende diretamente da riqueza do espaço $\mathcal{V}^h$, controlada pelo refinamento da malha e pela ordem polinomial das funções de base [6].

De maneira compacta, o procedimento de Galerkin converte o problema contínuo em:

$$\boxed{\mathbf{K}\mathbf{U} = \mathbf{F}} \quad (3.20)$$

A construção efetiva das matrizes $\mathbf{K}$ e do vetor $\mathbf{F}$ depende da escolha dos elementos e das funções de forma, desenvolvida na seção seguinte.

3.4.4 Elementos Triangulares Lineares

A discretização do domínio Ω é realizada por uma malha de N_e elementos triangulares, cujos vértices coincidem com os N nós globais da malha. Cada elemento e possui três nós locais $(1, 2, 3)$, com coordenadas (x_i, y_i) , $i = 1, 2, 3$.

As funções de forma lineares N_i^e para o elemento triangular linear são definidas de modo que $N_i^e(\mathbf{x}_j) = \delta_{ij}$ (propriedade de partição da unidade nodal). Em coordenadas cartesianas, estas funções são expressas por [14]:

$$N_i^e(x, y) = \frac{1}{2A^e}(a_i + b_i x + c_i y), \quad i = 1, 2, 3 \quad (3.21)$$

onde A^e é a área do elemento triangular e os coeficientes são dados por permutação cíclica dos índices dos vértices:

$$a_i = x_j y_k - x_k y_j, \quad b_i = y_j - y_k, \quad c_i = x_k - x_j \quad (3.22)$$

com (i, j, k) em ordem cíclica $(1, 2, 3)$, $(2, 3, 1)$, $(3, 1, 2)$. A área do elemento é calculada por:

$$A^e = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)| \quad (3.23)$$

Os gradientes das funções de forma são constantes dentro de cada elemento linear:

$$\nabla N_i^e = \frac{1}{2A^e} \begin{pmatrix} b_i \\ c_i \end{pmatrix} \quad (3.24)$$

Esta propriedade simplifica consideravelmente as integrais elementares, uma vez que os integrandos envolvendo $\nabla N_i \cdot \nabla N_j$ tornam-se constantes no interior do elemento.

3.4.5 Matrizes Elementares

A partir das funções de forma lineares, as integrais da formulação de Galerkin (Eqs. 3.18 e 3.19) são calculadas elemento a elemento e, em seguida, assembradas no sistema global.

Matriz de Rigidez Elementar

Para o problema de difusão com condutividade k constante por elemento, a matriz de rigidez elementar $\mathbf{K}^e$ de dimensão 3×3 é:

$$K_{ij}^e = \int_{\Omega^e} k \nabla N_i^e \cdot \nabla N_j^e d\Omega = \frac{k}{4A^e} (b_i b_j + c_i c_j) \quad (3.25)$$

Matriz de Massa Elementar

Em problemas transientes, a discretização temporal introduz um termo de acumulação que requer a *matriz de massa*. Para elementos lineares, a matriz de massa consistente é [13]:

$$M_{ij}^e = \int_{\Omega^e} \rho c_p N_i^e N_j^e d\Omega = \frac{\rho c_p A^e}{12} \begin{cases} 2 & \text{se } i = j \\ 1 & \text{se } i \neq j \end{cases} \quad (3.26)$$

Alternativamente, a *matriz de massa concentrada* (*lumped*) distribui a massa uniformemente nos nós, resultando em uma matriz diagonal:

$$M_{ij}^{e,\text{lump}} = \frac{\rho c_p A^e}{3} \delta_{ij} \quad (3.27)$$

Vetor de Carga Elementar

Para um termo fonte f uniforme no elemento, o vetor de carga elementar resulta em:

$$F_i^e = \int_{\Omega^e} N_i^e f d\Omega = \frac{f A^e}{3} \quad (3.28)$$

3.4.6 Montagem Global

A construção do sistema global $\mathbf{KU} = \mathbf{F}$ é realizada pelo procedimento de *assembly*, que consiste em acumular as contribuições de cada elemento nas posições globais correspondentes. Para cada elemento e , a conectividade local-global é definida por um mapa de índices $\text{IEN}(a, e) \rightarrow I$, que associa o nó local a do elemento e ao nó global I .

A montagem é expressa como:

$$K_{IJ} = \sum_{e=1}^{N_e} K_{\text{IEN}(a,e), \text{IEN}(b,e)}^e, \quad F_I = \sum_{e=1}^{N_e} F_{\text{IEN}(a,e)}^e \quad (3.29)$$

Após a montagem, as condições de contorno de Dirichlet são impostas por modificação das linhas correspondentes do sistema. A resolução do sistema linear resultante fornece os valores nodais $\mathbf{U}$.

3.5 Formulação Vorticidade-Corrente

A resolução direta das equações de Navier-Stokes em variáveis primitivas (velocidade-pressão) apresenta dificuldades numéricas associadas ao acoplamento pressão-velocidade e à satisfação da restrição de continuidade (Eq. 3.3). Em problemas bidimensionais, a formulação em *Vorticidade-Corrente* (ω - ψ) oferece uma alternativa eficaz, eliminando o campo de pressão das incógnitas e reduzindo o sistema a duas equações escalares [1].

3.5.1 Definições

Para um escoamento bidimensional, define-se a *função corrente* $\psi(x, y, t)$ tal que o campo de velocidades satisfaça identicamente a equação da continuidade:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x} \quad (3.30)$$

A *vorticidade*, componente escalar no plano 2D, é definida como o rotacional do campo de velocidades:

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \quad (3.31)$$

Substituindo as definições da função corrente na expressão da vorticidade, obtém-se a equação de Poisson para a função corrente:

$$\nabla^2 \psi = -\omega \quad (3.32)$$

3.5.2 Equação de Transporte da Vorticidade

Aplicando o rotacional às equações de Navier-Stokes (Eq. 3.4), o gradiente de pressão é eliminado e obtém-se a equação de transporte da vorticidade:

$$\frac{\partial \omega}{\partial t} + u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} = \nu \nabla^2 \omega \quad (3.33)$$

onde $\nu = \mu/\rho$ é a viscosidade cinemática. Esta equação possui a estrutura clássica de convecção-difusão, onde o campo de velocidades (u, v) transporta a vorticidade ω enquanto a difusão viscosa a dissipa.

3.5.3 Sistema Acoplado e Solução Iterativa

O sistema ω - ψ é resolvido iterativamente. A cada passo de tempo (ou iteração estacionária), o procedimento consiste em:

1. Resolver a equação de Poisson (Eq. 3.32) para ψ , dado ω .
2. Calcular o campo de velocidades (u, v) a partir de ψ (Eq. 3.30).
3. Resolver a equação de transporte (Eq. 3.33) para ω , dado (u, v) .
4. Verificar a convergência e repetir, se necessário.

3.5.4 Matriz de Convecção e Estabilização SUPG

A discretização via MEF do operador convectivo $\mathbf{v} \cdot \nabla \phi$ conduz à *matriz de convecção* elementar, de dimensão 3×3 para elementos lineares:

$$C_{ij}^e = \int_{\Omega^e} N_i (\mathbf{v} \cdot \nabla N_j) d\Omega = \frac{A^e}{3} \left(\frac{u_e b_j + v_e c_j}{2A^e} \right) \quad (3.34)$$

onde u_e e v_e são as componentes de velocidade avaliadas no centroide do elemento.

Quando o campo de velocidades é intenso em relação à difusão (número de Péclet elevado), a formulação de Galerkin padrão produz oscilações espúrias. Para contornar essa limitação, emprega-se a técnica *Streamline Upwind Petrov-Galerkin* (SUPG), que adiciona uma difusão artificial orientada na direção do escoamento. A formulação SUPG foi proposta por Brooks e Hughes [5], generalizando as funções de peso assimétricas (*upwind*) introduzidas anteriormente por Christie et al. [21].

Na formulação SUPG, a função-teste é modificada para $w_i = N_i + \tau_{\text{SUPG}} \mathbf{v} \cdot \nabla N_i$, com o parâmetro de estabilização:

$$\tau_{\text{SUPG}} = \frac{h_e}{2\|\mathbf{v}\|} \left(\coth \text{Pe}_e - \frac{1}{\text{Pe}_e} \right) \quad (3.35)$$

onde $\text{Pe}_e = \|\mathbf{v}\| h_e / (2\alpha)$ é o número de Péclet elementar.

No *MinervaMesh*, a estabilização SUPG é empregada no caso unidimensional de convecção-difusão, onde o efeito do número de Péclet é mais crítico. No solver bidimensional de Navier-Stokes, a convecção da vorticidade utiliza a formulação de Galerkin padrão (Eq. 3.34), sendo a estabilidade controlada pela escolha adequada do passo de tempo e do número de Reynolds.

3.6 Integração Temporal

Para problemas transientes, a discretização temporal converte a EDP semidiscreta (após a discretização espacial por MEF) em um sistema algébrico a cada intervalo de tempo Δt . No caso geral, o sistema semidiscreto assume a forma:

$$\mathbf{M} \frac{d\mathbf{U}}{dt} + \mathbf{K}\mathbf{U} = \mathbf{F}(t) \quad (3.36)$$

3.6.1 Método θ Generalizado

A família de métodos θ parametriza a ponderação temporal entre os instantes t^n e t^{n+1} :

$$\mathbf{M} \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{\Delta t} + \mathbf{K} [\theta \mathbf{U}^{n+1} + (1 - \theta) \mathbf{U}^n] = \theta \mathbf{F}^{n+1} + (1 - \theta) \mathbf{F}^n \quad (3.37)$$

Os casos particulares mais relevantes são:

- $\theta = 0$: Euler explícito (condicionalmente estável);
- $\theta = 1/2$: Crank-Nicolson (incondicionalmente estável, segunda ordem);
- $\theta = 1$: Euler implícito (incondicionalmente estável, primeira ordem).

3.6.2 Esquema Semi-Implícito Adotado

No solver de Navier-Stokes do *MinervaMesh*, a equação de transporte da vorticidade (Eq. 3.33) é discretizada com um esquema *semi-implícito*: o termo difusivo é tratado implicitamente ($\theta = 1$), enquanto o termo convectivo é avaliado explicitamente no instante t^n . Isso resulta no sistema:

$$\left(\mathbf{M} + \frac{\Delta t}{\text{Re}} \mathbf{K} \right) \boldsymbol{\omega}^{n+1} = \mathbf{M} \boldsymbol{\omega}^n - \Delta t \mathbf{C}^n \boldsymbol{\omega}^n \quad (3.38)$$

onde $\mathbf{C}^n$ é a matriz de convecção assembrada a partir do campo de velocidades no instante atual e $\text{Re} = \rho UL/\mu$ é o número de Reynolds. O tratamento implícito da difusão garante estabilidade incondicional para esse operador, enquanto a convecção explícita impõe restrições sobre Δt associadas ao número de Courant.

Para problemas puramente difusivos (condução transiente), utiliza-se o método θ generalizado (Eq. 3.37), permitindo ao usuário selecionar entre Euler e Crank-Nicolson conforme o compromisso desejado entre precisão e estabilidade.

3.6.3 Condição de Contorno de Vorticidade na Parede

Na formulação ω - ψ , a vorticidade nas paredes sólidas não é conhecida *a priori* e deve ser calculada indiretamente a partir do campo de função corrente. Aplicando uma expansão de Taylor de ψ na direção normal à parede, obtém-se uma aproximação de primeira ordem para ω_w , conhecida como condição de Thom [46]:

$$\omega_w = -\frac{2(\psi_{\text{nb}} - \psi_w)}{h^2} - \frac{2u_{\text{tan}}}{h} \quad (3.39)$$

onde ψ_w e ψ_{nb} são os valores da função corrente na parede e no nó vizinho mais próximo na direção normal (apontando para o interior do domínio), respectivamente; h é a distância entre esses nós; e u_{tan} é a componente tangencial da velocidade da parede, consistente com a convenção $u = \partial\psi/\partial y$ adotada nesta formulação ($u_{\text{tan}} =$

0 para paredes estacionárias e $u_{\text{tan}} = U_{\text{lid}}$ para a tampa móvel da cavidade com velocidade U_{lid} na direção $+x$).

3.7 Síntese do Capítulo

Este capítulo estabeleceu o arcabouço matemático completo empregado pelo *MinervaMesh*. A formulação parte das EDPs governantes para fenômenos térmicos e fluidodinâmicos, transita pela introdução conceitual do MDF e se aprofunda no MEF via Galerkin. A discretização com elementos triangulares lineares permite a construção explícita das matrizes elementares de rigidez, massa e convecção, enquanto a formulação Vorticidade-Corrente viabiliza a solução de escoamentos 2D sem a necessidade de resolver diretamente o campo de pressão. A integração temporal semi-implícita e a aproximação de primeira ordem para a vorticidade na parede completam o ferramental necessário para a simulação dos casos apresentados no Capítulo 6.

Capítulo 4

A Plataforma MinervaMesh

Este capítulo apresenta o produto central deste trabalho: o *MinervaMesh*, plataforma web de código aberto para simulação numérica em Engenharia Mecânica. O foco aqui é a plataforma enquanto *software de entrega*: a motivação e a visão geral, a justificativa das tecnologias adotadas — *PyScript*, *WebAssembly* e a arquitetura *client-side* —, a análise quantitativa do custo computacional da execução no navegador, a organização do código e as limitações do paradigma. A implementação numérica de cada módulo de simulação, isto é, a tradução das equações diferenciais discretizadas em código, é tratada separadamente no Capítulo 5.

4.1 Visão Geral da Plataforma

O *MinervaMesh* é uma plataforma de simulação numérica projetada para operar integralmente no navegador web, sem dependência de servidores de processamento remoto nem instalação de software por parte do usuário. A plataforma reúne implementações dos métodos numéricos apresentados no Capítulo 3 em uma interface interativa que permite a parametrização do problema, a execução da simulação e a visualização dos resultados em uma única página. Todo o código é distribuído como software livre, sob licença MIT, em repositório público¹ — o que permite a qualquer instituição de ensino hospedar, estudar, modificar e estender a plataforma.

¹<https://github.com/lgsfarias/minervamesh>

A motivação central do projeto reside na eliminação das barreiras de acesso associadas às ferramentas de simulação tradicionais. Ambientes como ANSYS, COMSOL Multiphysics ou mesmo distribuições locais de Python científico exigem licenciamento, infraestrutura computacional dedicada e configuração de ambiente — requisitos que frequentemente inviabilizam o uso autônomo em disciplinas de graduação, monitorias e estudos independentes [2]. O *MinervaMesh* contorna essas limitações ao executar todo o processamento no dispositivo do próprio usuário, utilizando o navegador como ambiente de execução.

A plataforma abrange três domínios da Engenharia Mecânica:

- **Transferência de Calor:** condução estacionária e transiente, convecção-difusão com estabilização SUPG, transferência de calor 2D;
- **Mecânica dos Fluidos:** escoamento potencial, Navier-Stokes 2D via formulação Vorticidade-Corrente;
- **Mecânica Estrutural:** flexão de vigas (Euler-Bernoulli) e análise modal de vibração.

A Figura 4.1 apresenta a tela inicial da plataforma, cujo menu organiza as simulações disponíveis por método numérico.

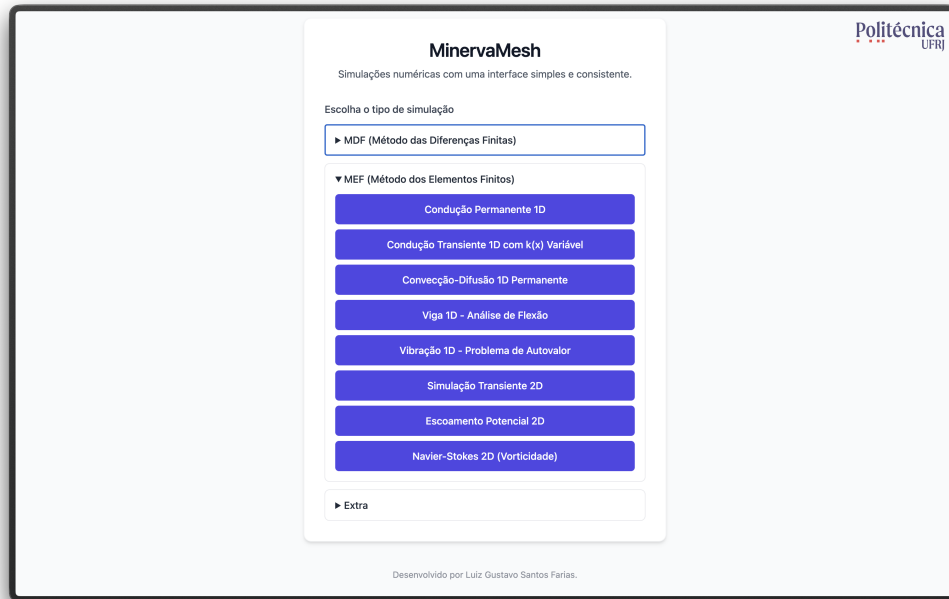


Figura 4.1: Tela inicial do *MinervaMesh* com o menu de simulações disponíveis.

4.2 Justificativa Tecnológica

4.2.1 PyScript e WebAssembly

A escolha do PyScript como runtime da plataforma fundamenta-se em três critérios: portabilidade, familiaridade da linguagem e manutenibilidade.

O PyScript é um *framework* de código aberto que permite a execução de programas Python diretamente no navegador, apoiando-se no Pyodide — uma compilação do interpretador CPython para WebAssembly (Wasm) [3]. O WebAssembly é um formato binário padronizado pelo W3C que executa código de baixo nível em velocidade próxima à nativa, dentro da *sandbox* de segurança do navegador. A cadeia de execução é:

1. O navegador carrega a página HTML contendo a tag `<script type="py">`.
2. O *runtime* do PyScript inicializa o Pyodide, que instancia uma máquina virtual CPython compilada para WebAssembly.
3. As dependências declaradas em `config.toml` (NumPy, SciPy, Matplotlib) são carregadas como pacotes pré-compilados para Wasm.

4. O código Python da simulação é interpretado pelo Pyodide, com acesso direto ao DOM do navegador para leitura de parâmetros e renderização de resultados.

A escolha de Python como linguagem de implementação é estratégica para o contexto educacional: a linguagem apresenta sintaxe próxima à notação matemática e dispõe de um ecossistema científico consolidado — NumPy para álgebra linear, SciPy para solvers esparsos, Matplotlib para visualização [4]. Ao manter a implementação em Python puro, o *MinervaMesh* permite que alunos inspecionem, compreendam e modifiquem o código das simulações, exibido com realce de sintaxe na própria página de cada módulo.

4.2.2 Computação *Client-Side*

A arquitetura *client-side* implica que todo o processamento numérico ocorre no dispositivo do usuário. Esta decisão tem impactos diretos em quatro dimensões:

1. **Custo de infraestrutura:** o *MinervaMesh* é um site estático. Pode ser hospedado gratuitamente em plataformas como GitHub Pages ou Netlify, sem necessidade de servidor *backend*, banco de dados ou infraestrutura de *cloud computing*. O custo operacional é efetivamente zero.
2. **Escalabilidade:** não existe gargalo de servidor central. Cada usuário utiliza o poder computacional de sua própria máquina, eliminando filas de processamento e limites de usuários simultâneos.
3. **Privacidade:** nenhum dado de simulação transita pela rede. Parâmetros, malhas e resultados permanecem integralmente no dispositivo do usuário.
4. **Disponibilidade:** após o carregamento inicial dos *assets*, a plataforma pode operar sem conexão com a internet, viabilizando o uso em ambientes com conectividade limitada.

4.2.3 Comparação com Alternativas

A Tabela 4.1 posiciona o *MinervaMesh* frente às alternativas mais comuns no contexto de ensino de simulação numérica.

Tabela 4.1: Comparação entre plataformas de simulação numérica para ensino.

Critério	MinervaMesh	MATLAB	Jupyter	COMSOL
Custo de licença	Gratuito	Pago	Gratuito	Pago
Instalação necessária	Não	Sim	Sim	Sim
Execução no navegador	Sim	Não	Parcial	Não
Código-fonte acessível	Sim	Parcial	Sim	Não
Infraestrutura de servidor	Não	Não	Sim	Sim

A principal limitação da abordagem *client-side* reside na capacidade computacional: problemas com malhas muito refinadas (dezenas de milhares de nós) podem apresentar tempo de execução elevado, uma vez que o desempenho do WebAssembly, embora próximo ao nativo, ainda é inferior ao de implementações compiladas em C/Fortran. Para os casos de interesse deste trabalho, envolvendo malhas de centenas a poucos milhares de nós, essa limitação não se mostrou restritiva. Uma análise quantitativa desse custo é apresentada na Seção 4.2.4, e a Seção 4.4 sistematiza as limitações da plataforma.

4.2.4 Análise Computacional

Para quantificar o custo computacional da execução em ambiente *client-side* e o impacto da camada *WebAssembly*, foi conduzido um *benchmark* cruzado entre dois ambientes: *Pyodide* no navegador (mesmo cenário em que a plataforma opera para o usuário final) e *CPython* nativo, ambos rodando na mesma máquina (Apple M1 Max, 64 GB de RAM, macOS), com idênticas versões de NumPy e SciPy. Foram instrumentados dois solvers MEF representativos do espectro de complexidade da plataforma: a condução permanente 1D (sistema denso resolvido por `np.linalg.solve`) e a transferência de calor 2D transiente (sistema esparsa CSR com 50 passos de `spsolve`). As medições reportam mediana e MAD (*median absolute deviation*) de cinco rodadas (três para os tamanhos maiores), descartando uma rodada de *warm-up*. As fases de montagem da matriz e resolução do sistema são cronometradas separadamente, com toda a etapa de pós-processamento (geração de gráficos, escrita no DOM) excluída da medição. As listagens completas dos *scripts* de *benchmark* constam no Apêndice B.

Tabela 4.2: Tempo total do solver (mediana, com MAD reportado abaixo de 1% em todas as células) em CPython nativo e Pyodide (navegador), na mesma máquina (Apple M1 Max). t_{mont} é o tempo de montagem (incluindo geração de malha quando aplicável) e t_{slv} é o tempo da resolução linear (`np.linalg.solve` no caso 1D denso, ou o total dos passos de `spsolve` nos casos 2D esparsos). “CP” = CPython nativo; “Py” = Pyodide.

Caso	N	t_{mont} CP	t_{slv} CP	t_{tot} CP	t_{tot} Py	Py/CP
Cond. 1D MEF	101	0,15 ms	0,06 ms	0,21 ms	1,6 ms	7,1×
Cond. 1D MEF	1 001	1,56 ms	9,89 ms	11,5 ms	16,0 ms	1,4×
Cond. 1D MEF	10 001	23,8 ms	4 887 ms	4 911 ms	1 394 ms	0,3×
Transcalor 2D MEF	400	56,8 ms	25,1 ms	81,9 ms	244 ms	3,0×
Transcalor 2D MEF	1 600	236 ms	143 ms	379 ms	1 138 ms	3,0×
Transcalor 2D MEF	6 400	994 ms	849 ms	1 843 ms	5 814 ms	3,2×

A Figura 4.2 ilustra a escalabilidade dos dois solvers em escala logarítmica. No caso 1D, o crescimento aproximadamente cúbico do tempo de `solve` (de 0,06 ms para 4,89 s ao multiplicar N por cem) reflete a complexidade $\mathcal{O}(N^3)$ do solver direto denso adotado, que se torna proibitivo já em torno de $N = 10\,000$ nós — precisamente o regime em que se justificaria a migração para um solver iterativo esparsos. No caso transcalor 2D, a inclinação observada é substancialmente mais branda: o tempo total cresce aproximadamente com $N^{1,2}$, comportamento típico de fatorações esparsas em malhas estruturais 2D, e mantém-se em poucos segundos mesmo com $N \approx 6\,400$ nós e 50 passos temporais. Os dois casos cobrem, portanto, os dois caminhos de código distintos da plataforma: solver denso em problemas 1D e solver esparsos com avanço temporal nos problemas 2D. A simulação de Navier-Stokes 2D, embora mais pesada por exigir remontagem da matriz de convecção a cada passo, segue qualitativamente o mesmo padrão da transcalor; a validação do caso de cavidade contra Ghia *et al.* [24], apresentada no Capítulo 6, é executada na própria plataforma com tempos compatíveis com uso interativo (poucos minutos no navegador para malha 41×41).

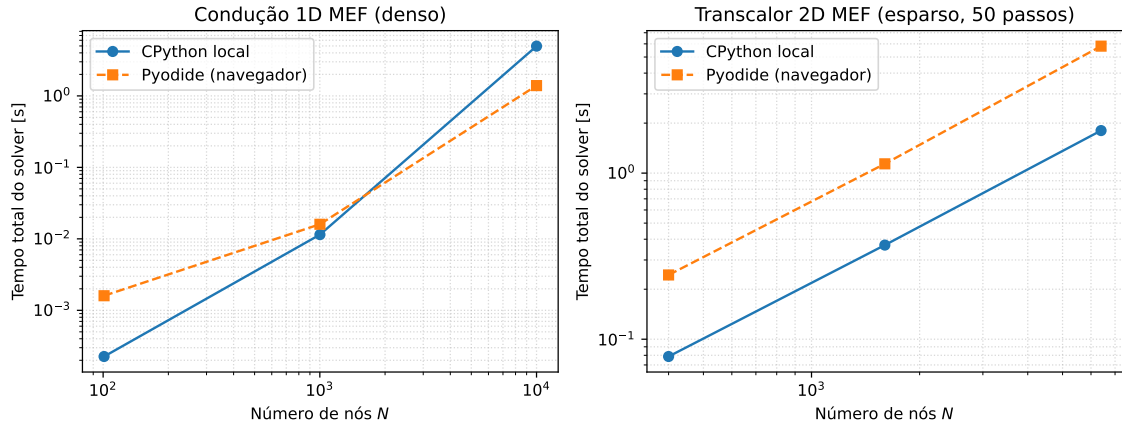


Figura 4.2: Escalabilidade do tempo total do solver com o número de nós, em escala log-log, comparando CPython nativo (linha sólida) e Pyodide no navegador (linha tracejada). À esquerda, condução 1D MEF (denso) com inclinação compatível com $\mathcal{O}(N^3)$; à direita, transcalor 2D MEF (esparso, 50 passos) com inclinação próxima de $\mathcal{O}(N^{1,2})$.

A leitura da Tabela 4.2 revela três regimes. Quando o tempo total é dominado por loops Python sobre elementos — caso da transcalor 2D em todos os tamanhos — a razão Pyodide/CPython mantém-se aproximadamente constante em torno de 3,0–3,2 $\times$, refletindo o fato de que o *WebAssembly* executa o interpretador CPython compilado para Wasm com penalidade modesta, mas *não* acelera o *bytecode* interpretado em si. Em problemas muito pequenos — como a condução 1D com $N = 101$ — a razão sobe para cerca de 7 $\times$, refletindo o sobrecusto fixo de inicialização das chamadas Python que se torna dominante quando o tempo absoluto é da ordem de milissegundos. Já no caso grande, em que o tempo migra para a fatoração linear — a condução 1D em $N = 10\,001$, com a etapa de `np.linalg.solve` respondendo por cerca de 99% do total —, a Tabela 4.2 registra o Pyodide mais rápido que o CPython nativo (razão 0,3 $\times$). Esse ponto, contudo, não indica uma vantagem de desempenho do navegador: trata-se de um solver denso aplicado a uma matriz tridiagonal armazenada de forma densa (quase toda nula), fora do regime representativo de uso. O Apple Accelerate, biblioteca BLAS empregada pelo NumPy nativo, executa a fatoração LU densa completa, de custo $\mathcal{O}(N^3)$, independentemente do preenchimento, ao passo que o OpenBLAS portado para Wasm realiza muito menos operações sobre os blocos nulos; a aparente inversão mede, portanto, o volume de trabalho de cada

biblioteca sobre uma matriz degenerada, e não a velocidade relativa dos ambientes. Em síntese, enquanto o gargalo está em *loops* Python ou na montagem das matrizes — situação das cargas representativas da plataforma —, a penalidade de execução no navegador é um pequeno fator multiplicativo (entre 1,4 e 3,2×), compatível com a interatividade esperada em uso didático; em álgebra densa de grande porte a execução em Wasm é substancialmente mais lenta, mas esse é precisamente o regime em que a formulação densa cederia lugar a um solver esparsa.

4.3 Arquitetura de Software

4.3.1 Organização de Diretórios

O repositório público do *MinervaMesh* (<https://github.com/lgsfarias/minervamesh>) segue uma estrutura modular, onde cada simulação é autocontida em seu próprio diretório. A organização geral é apresentada a seguir:

```
minervamesh/
|-- index.html           # Menu principal
|-- assets/             # Logos e imagens globais
|-- simulations/
|   |-- mef/            # Elementos Finitos
|   |   |-- <nome-do-modulo>/
|   |   |   |-- index.html    # Interface do usuario
|   |   |   |-- simulation.py  # Logica numerica
|   |   |   +-- config.toml    # Dependencias Python
|   |   |-- ...              # (8 modulos MEF)
|   |   +-- escoamento-fluido-2d/
|   |       |-- common.py      # Matrizes MEF 2D
|   |       +-- geometry.py    # Geometrias de obstaculo
|   |-- mdf/             # Diferencas Finitas
|   +-- extra/           # Ferramentas auxiliares
```

Cada módulo de simulação segue um padrão uniforme de três arquivos:

- `index.html`: interface do usuário, contendo o formulário de parâmetros, o contêiner de resultados e a carga do PyScript;
- `simulation.py`: implementação da lógica numérica em Python, incluindo

montagem de matrizes, solução do sistema e geração de gráficos;

- `config.toml`: declaração das dependências Python necessárias (por exemplo, `numpy`, `scipy`, `matplotlib`).

4.3.2 Fluxo de Execução de uma Simulação

O ciclo de vida de uma simulação no *MinervaMesh* compreende cinco camadas funcionais, todas resolvidas no próprio navegador. No *carregamento*, o navegador requisita o `index.html` de um servidor HTTP de arquivos estáticos, e a tag `<script type="py">` instrui o PyScript a inicializar o Pyodide e carregar as dependências declaradas em `config.toml`. A camada de *parametrização* expõe os parâmetros físicos e numéricos do problema como campos de um formulário HTML, lidos pelo código Python diretamente do DOM. Na *execução*, o Pyodide monta as matrizes do MEF, impõe as condições de contorno e resolve o sistema algébrico, sem qualquer comunicação com servidor. A *visualização* converte os gráficos gerados pelo Matplotlib em imagens codificadas em Base64, inseridas dinamicamente no DOM da página. Por fim, a camada de *inspeção* disponibiliza, na própria página, o código-fonte Python da simulação com realce de sintaxe — o elo entre a ferramenta de simulação e o seu uso como objeto de estudo em métodos numéricos.

4.4 Limitações da Plataforma

A análise computacional apresentada na Seção 4.2.4 torna explícitas as restrições impostas pela arquitetura *client-side*. Esta seção sistematiza tais restrições, distinguindo-as em dois grupos: limitações *intrínsecas* ao modelo de execução em *WebAssembly* (que motivariam, em uma plataforma de produção em larga escala, uma migração para arquitetura híbrida) e limitações *contornáveis*, mitigáveis dentro do próprio paradigma *client-side* via extensões já viáveis tecnicamente.

Limitações intrínsecas.

- **Memória disponível:** o processo *Pyodide* executa em *sandbox* com *heap WebAssembly* tipicamente limitado a poucos gigabytes (4 GB no padrão Wasm32, ampliável apenas via Wasm64 com suporte parcial nos navegadores), e a

pressão prática da coleta de lixo (*garbage collection*, mecanismo de liberação automática de memória do *runtime*) torna-se relevante a partir de algumas centenas de MB.

- **Escalabilidade do problema:** a combinação entre o limite de memória da *sandbox* e a complexidade dos solvers diretos adotados estabelece um teto efetivo de tamanho de problema. Considerando o benchmark da Seção 4.2.4, este teto é da ordem de 10^4 nós para formulações com matriz densa (acima desse valor, o tempo de `np.linalg.solve` ultrapassa cinco segundos e o consumo de memória aproxima-se da fronteira da *sandbox*) e da ordem de 10^5 nós para formulações esparsas com `spsolve`. A extensão para problemas tridimensionais com malhas refinadas, comum em engenharia profissional, encontra-se fora desta faixa.
- **Execução em *thread* única:** o *Pyodide* ocupa apenas a *thread* principal de JavaScript, o que impede o uso direto de `multiprocessing`, `joblib` ou de bibliotecas científicas em modo *multithread*. Operações longas bloqueiam a renderização da página até concluírem, e o ganho potencial de paralelismo de domínio (decomposição da malha entre múltiplos núcleos) não é acessível dentro do paradigma atual.
- **Latência de inicialização (*cold start*):** o carregamento do *runtime WebAssembly* e dos pacotes científicos (`numpy`, `scipy`, `matplotlib`) demanda tipicamente entre cinco e quinze segundos no primeiro acesso, dependendo da banda do usuário. Esse custo é amortizado pelo cache do navegador em visitas subsequentes, mas não é eliminado.
- **Geração de malha:** esta é, na prática, a mais sensível das limitações da plataforma para fins de engenharia. O `Gmsh` — gerador de malhas não estruturadas adotado como referência *de facto* em pré-processamento de MEF — não está disponível no *Pyodide*, pois depende de extensões em C++ ainda não portadas para *WebAssembly*. O mesmo se aplica a `Triangle`, `TetGen`, `MeshPy` e demais ferramentas consagradas. Como substituto, o *MinervaMesh* utiliza a triangulação de Delaunay nativa do `matplotlib.tri`, o que impõe três restrições importantes: (i) a malha resultante é sempre 2D — não há geração

de malha tetraédrica 3D nativamente disponível; (ii) o algoritmo de Delaunay opera sobre um conjunto de pontos pré-determinado, sem refinamento adaptativo a posteriori, sem refinamento dirigido por gradiente da solução, e sem geração de *boundary layer mesh* (camada limite refinada próxima a paredes), recursos centrais em escoamentos de alto Reynolds; (iii) geometrias com obstáculos ou contornos curvilíneos exigem que os pontos da fronteira sejam definidos manualmente em coordenadas cartesianas, como ilustram os módulos de escoamento 2D do MinervaMesh, em que cilindros e degraus são parametrizados ponto a ponto. Esta restrição limita o repertório prático da plataforma a geometrias simples.

- **Suporte parcial à pilha científica:** de modo análogo ao *Gmsh*, outros pacotes que dependem de extensões nativas ainda não portadas para *WebAssembly* — como *petsc4py* (solvers iterativos de larga escala) e diversos pacotes de aprendizado de máquina — não estão disponíveis no *Pyodide*, embora a cobertura da pilha numérica fundamental (NumPy, SciPy, Matplotlib) seja completa.

Limitações contornáveis.

- **Bloqueio da *thread* principal:** mitigável por meio de *Web Workers*, que permitem mover a execução do *Pyodide* para uma *thread* paralela ao DOM, preservando a responsividade da interface durante simulações longas. A reorganização exigida é localizada à camada de orquestração.
- **Solvers diretos esparsos:** o uso atual de *spsolve* no caso transiente 2D refatora a matriz **A** a cada chamada e, embora robusto, é mais custoso do que o necessário para sistemas simétricos definidos positivos. A substituição por solvers iterativos (*scipy.sparse.linalg.cg*, *gmres*) com pré-condicionamento adequado — ou o reuso explícito da fatoração via *splu* — viabilizaria malhas substancialmente maiores dentro da mesma fronteira de memória.
- **Geração de malha avançada:** a integração com geradores externos consagrados é viável à medida que esses geradores recebam *ports* para *WebAssembly*. O projeto *triangle-wasm* [30] comprova essa viabilidade ao distribuir o

gerador *Triangle*, de J. Shewchuk, compilado via *Emscripten* e executável diretamente no navegador — amplia substancialmente o repertório 2D em relação à triangulação simples do `matplotlib.tri`, ao oferecer triangulação de Delaunay com restrições de contorno e refinamento controlado por qualidade. A portabilização do `Gmsh` para *WebAssembly*, ainda inexistente em forma oficial mas tecnicamente factível pelo mesmo caminho, eliminaria simultaneamente as três restrições atuais de pré-processamento (geometrias complexas, refinamento adaptativo e extensão tridimensional), preservando integralmente o paradigma *client-side*.

- **Eficiência da montagem:** os *loops* Python sobre elementos — predominantes na fase de montagem dos casos 2D — constituem o gargalo mais sensível à execução em Wasm, conforme indica a Tabela 4.2. Vetorização adicional via operações `numpy.add.at` ou refatoração para `Cython/Numba` (não disponíveis em Wasm no momento) reduziriam parte significativa dessa diferença.

Para os módulos didáticos e os tamanhos de malha tipicamente empregados em ensino de Engenharia Mecânica (centenas a poucos milhares de nós), nenhuma dessas limitações se mostrou restritiva ao longo das validações apresentadas no Capítulo 6. As restrições enumeradas acima são, portanto, fronteiras de ampliação futura, e não barreiras à utilização atual.

4.5 Síntese do Capítulo

Este capítulo apresentou a plataforma *MinervaMesh* sob a ótica do software de entrega. A escolha do PyScript e da computação *client-side* viabiliza uma plataforma de simulação numérica com custo de infraestrutura zero, portabilidade total e código aberto para inspeção acadêmica, com penalidade de desempenho quantificada em um pequeno fator multiplicativo frente à execução nativa. A organização modular do repositório — com cada simulação autocontida em três arquivos (HTML, Python, TOML) — permite a extensão da plataforma com novos casos sem impacto nos módulos existentes. Os capítulos seguintes completam o quadro: o Capítulo 5 detalha a implementação numérica de cada módulo de simulação, e o Capítulo 6 apresenta sua validação frente a soluções analíticas e *benchmarks* da literatura.

Capítulo 5

Implementação dos Módulos Numéricos

Os módulos computacionais apresentados neste capítulo constituem implementações diretas das formulações matemáticas desenvolvidas no Capítulo 3. Cada simulador corresponde a uma discretização específica obtida via formulação variacional de Galerkin ou, nos casos pertinentes, por diferenças finitas, e a estrutura do código reflete o fluxo numérico subjacente: discretização espacial (MEF/MDF), montagem do sistema matricial, integração temporal e imposição das condições de contorno. Dessa forma, cada módulo reproduz, em nível computacional, a sequência de operações matemáticas que conduz das equações governantes contínuas aos sistemas algébricos discretos efetivamente resolvidos.

O foco deste capítulo recai sobre *como* cada método é codificado e executado no navegador, com ênfase nas decisões de código e na correspondência entre operadores matemáticos e numéricos. A formulação detalhada e a validação de cada caso — equação governante, condições de contorno, solução de referência e análise de erro — são apresentadas no Capítulo 6. Os trechos de código exibidos referem-se às funções centrais de cada simulação; listagens representativas constam no Apêndice A, e o código integral de todos os módulos está disponível no repositório público da plataforma.

5.1 Módulos de Simulação MEF

Todos os módulos implementam diretamente as formulações variacionais e discretizações desenvolvidas no Capítulo 3. As matrizes elementares de rigidez, massa e carga são construídas a partir das expressões derivadas nas Seções 3.4.5 e 3.5.4, e cada trecho de código apresentado nesta seção referencia explicitamente a equação teórica correspondente, mantendo correspondência direta entre operadores matemáticos e operadores numéricos.

5.1.1 Condução Permanente 1D

O primeiro módulo resolve o problema de condução térmica unidimensional em regime permanente com geração interna de calor, validado na Seção 6.2. A discretização por elementos finitos lineares resulta na matriz de rigidez e no vetor de carga elementares (Seção 3.4.5), implementados como:

```
1 # Elemento linear 1D: ke e be
2 ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
3 be = (Q * h / 2.0) * np.array([1.0, 1.0])
```

A montagem percorre os N_{el} elementos, acumulando as contribuições nas posições globais. Após a imposição das condições de Dirichlet (zerando linhas e colunas correspondentes), o sistema $\mathbf{KT} = \mathbf{b}$ é resolvido por `numpy.linalg.solve`. A Figura 5.1 mostra a comparação entre a solução MEF e a solução analítica.

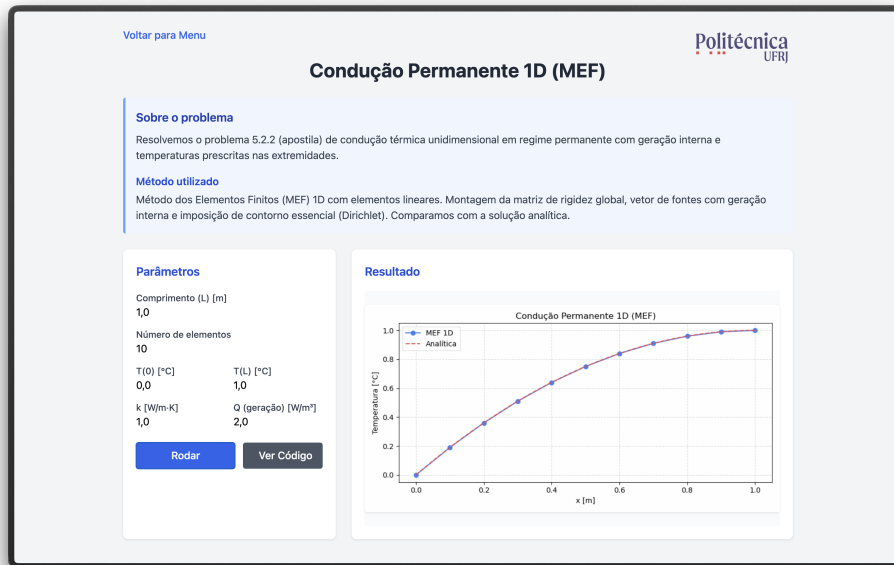


Figura 5.1: Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.

5.1.2 Condução Transiente 1D com $k(x)$ Variável

Este módulo estende o problema anterior para o regime transiente com condutividade térmica variável $k(x)$, incorporando a matriz de massa e a integração temporal conforme o sistema semidiscreto estabelecido na Seção 3.6. A formulação e a validação completas constam na Seção 6.3. As matrizes elementares são calculadas por:

```

1 # Matrizes de elemento 1D transiente
2 ke = (k_avg / h) * np.array([[1, -1], [-1, 1]])
3 me = (rho_cp * h / 6) * np.array([[2, 1], [1, 2]])

```

A integração temporal utiliza o método θ generalizado (Eq. 3.37), seguindo rigorosamente o esquema de avanço temporal derivado na Seção 3.6, e permitindo ao usuário selecionar entre Euler explícito, Crank-Nicolson e Euler implícito. A Figura 5.2 ilustra a evolução temporal do campo de temperatura.

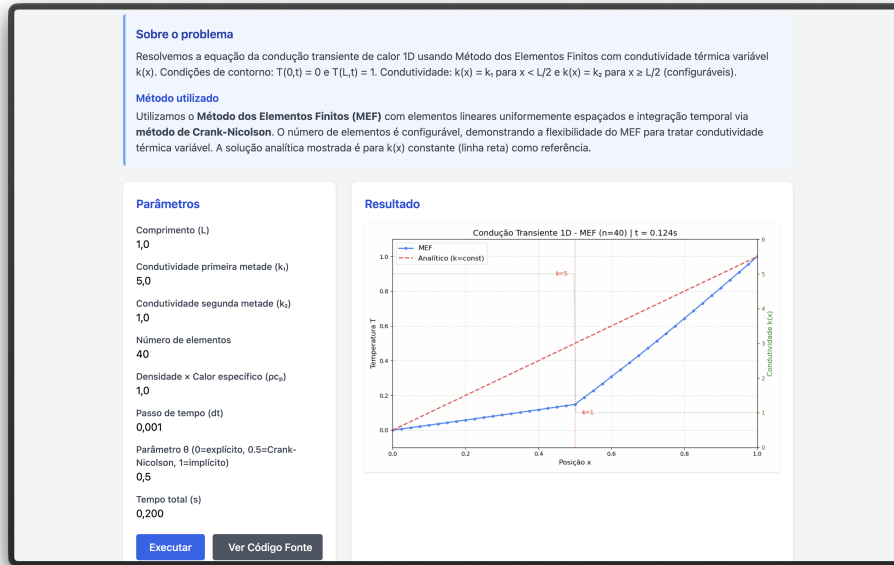


Figura 5.2: Evolução temporal do campo de temperatura com condutividade variável $k(x)$.

5.1.3 Convecção-Difusão 1D com Estabilização SUPG

O módulo de convecção-difusão 1D resolve a equação de advecção-difusão estacionária aplicando a técnica de estabilização SUPG (formulação desenvolvida na Seção 3.5.4; caso validado na Seção 6.4). A discretização MEF gera três matrizes elementares: difusão, convecção e estabilização SUPG. O parâmetro τ_{SUPG} é calculado segundo a expressão analítica da Eq. 3.35, implementado como:

```

1 # Numero de Peclet local e tau SUPG
2 Pe_local = (rho_cp * u * h) / (2 * k)
3 tau = h / (2*abs(u)) * (1/np.tanh(abs(Pe_local))
4                               - 1/abs(Pe_local))

```

A Figura 5.3 demonstra o efeito da estabilização SUPG em um problema com número de Péclet elevado. Sem estabilização, a formulação de Galerkin padrão produz oscilações espúrias; com SUPG, a solução numérica converge suavemente para a solução analítica.

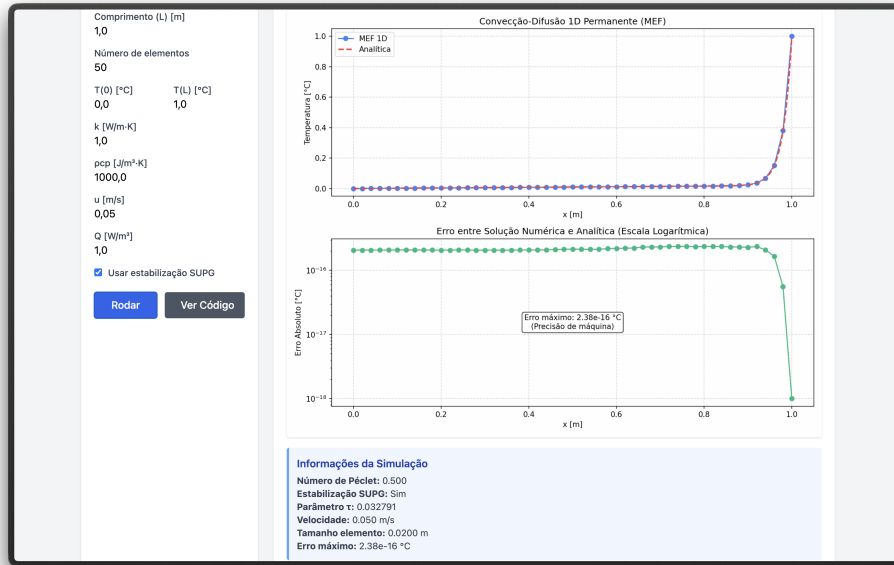


Figura 5.3: Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.

5.1.4 Viga 1D — Análise de Flexão (Euler-Bernoulli)

O módulo de viga 1D implementa elementos finitos de Euler-Bernoulli com dois graus de liberdade por nó: deslocamento vertical w e rotação θ , seguindo o procedimento de Galerkin apresentado na Seção 3.4.3 com funções de interpolação cúbicas de Hermite (caso validado na Seção 6.5). A matriz de rigidez elementar 4×4 é construída com essas funções:

```

1  # Matriz de rigidez do elemento de viga
2  ke = (EI / h**3) * np.array([
3      [12,    6*h,   -12,    6*h ],
4      [6*h,   4*h**2, -6*h,  2*h**2],
5      [-12,  -6*h,    12,   -6*h ],
6      [6*h,   2*h**2, -6*h,  4*h**2]
7  ])

```

O módulo suporta quatro condições de contorno (simplesmente apoiada, balanço, engastada-engastada e engastada-apoiada) e cinco tipos de carregamento (uniforme, linear, triangular, senoidal e pontual), além de propriedades materiais variáveis ao longo do comprimento. A Figura 5.4 apresenta a análise completa de uma viga,

incluindo deflexão, diagramas de momento fletor e esforço cortante.

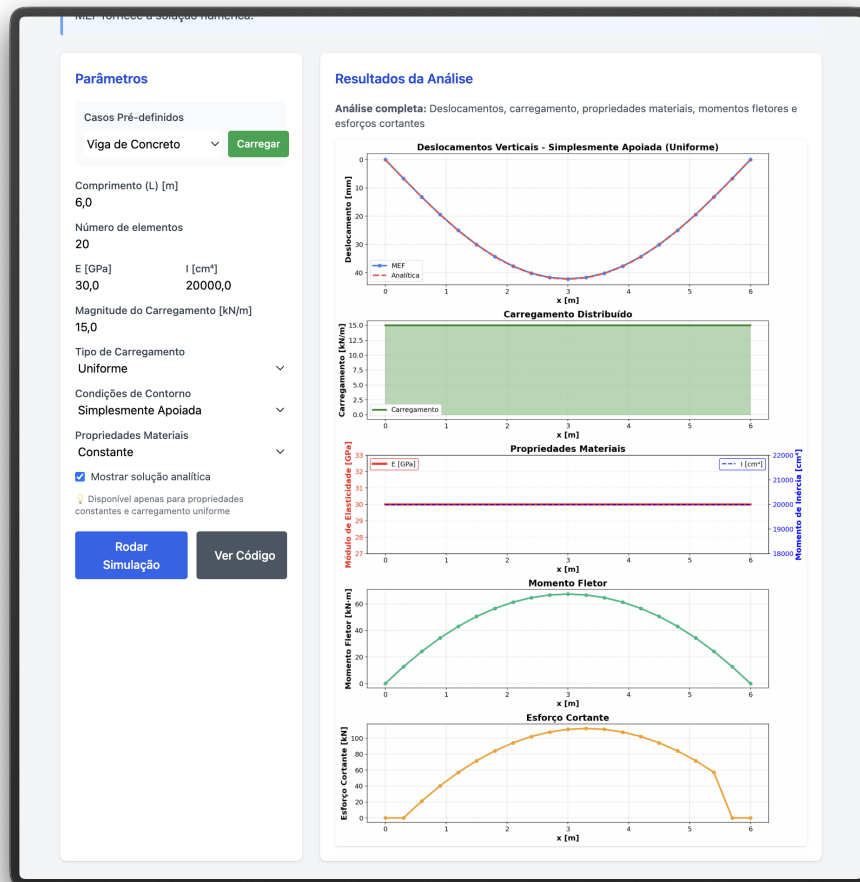


Figura 5.4: Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carregamento, propriedades, momento fletor e esforço cortante.

5.1.5 Vibração 1D — Problema de Autovalor

O módulo de vibração resolve o problema de autovalor generalizado $\mathbf{K}\phi = \lambda \mathbf{M}\phi$ para uma barra elástica 1D (caso validado na Seção 6.6), onde as frequências naturais são $f_n = \sqrt{\lambda_n}/(2\pi)$ e os autovetores ϕ_n representam os modos de vibração. As matrizes de rigidez e massa são construídas conforme o procedimento de montagem global descrito na Seção 3.4.6, empregando massa concentrada (*lumped*, Eq. 3.27):

```
1 # Matrizes de rigidez e massa (barra 1D)
2 ke = (E*A / h) * np.array([[1, -1], [-1, 1]])
3 me = (rho*A*h / 2) * np.array([[1, 0], [0, 1]])
```

O problema de autovalor é resolvido pela rotina `scipy.linalg.eigh`, que explora a simetria das matrizes para eficiência e estabilidade numérica. A solução fornece n frequências naturais e os correspondentes modos de vibração, comparáveis com as expressões analíticas clássicas — por exemplo, $f_n = nc/(2L)$ para barras engastadas em ambas as extremidades, onde $c = \sqrt{E/\rho}$ é a velocidade de propagação de ondas. A Figura 5.5 ilustra os cinco primeiros modos.



Figura 5.5: Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.

5.1.6 Transferência de Calor 2D

O módulo de transferência de calor bidimensional resolve a equação de condução transiente em uma placa quadrada discretizada com elementos triangulares lineares (Seção 3.4.4; caso validado na Seção 6.7). As matrizes elementares de rigidez e massa são construídas pela função compartilhada `element_matrices`, que implementa diretamente as expressões derivadas na Seção 3.4.5 (Eqs. 3.25 e 3.26):

```
1 def element_matrices(coords, k, rho, cv):
```

```

2     x0, x1, x2 = coords
3     area = 0.5 * abs((x1[0]-x0[0])*(x2[1]-x0[1])
4                   -(x2[0]-x0[0])*(x1[1]-x0[1]))
5     b = np.array([x1[1]-x2[1], x2[1]-x0[1], x0[1]-x1[1]])
6     c = np.array([x2[0]-x1[0], x0[0]-x2[0], x1[0]-x0[0]])
7     ke = (k/(4*area)) * (np.outer(b,b) + np.outer(c,c))
8     me = (rho*cv*area/12) * (np.ones((3,3)) + np.eye(3))
9     return ke, me

```

A integração temporal emprega o método de Crank-Nicolson ($\theta = 1/2$), conforme a formulação do método θ generalizado apresentada na Seção 3.6 (Eq. 3.37), formulado com matrizes esparsas (`scipy.sparse`) para viabilizar a execução no navegador com malhas de até 40×40 nós. A cada passo de tempo, um frame do campo de temperatura é gerado e acumulado para compor uma animação GIF. A Figura 5.6 ilustra um instante da evolução temporal.

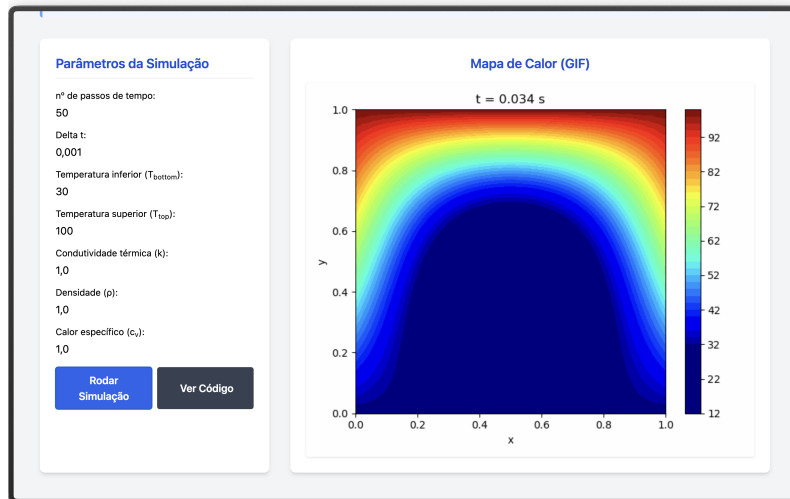


Figura 5.6: Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).

5.1.7 Escoamento Potencial 2D

O módulo de escoamento potencial resolve a equação de Laplace $\nabla^2\psi = 0$ em domínios 2D com obstáculos, caso particular da equação de Poisson para a função corrente (Eq. 3.32) com vorticidade nula (caso validado na Seção 6.8). A malha triangular é gerada pelo módulo `common.py`, que também assembla as matrizes globais

de rigidez e massa conforme o procedimento descrito na Seção 3.4.6.

O módulo `geometry.py` fornece diferentes geometrias de obstáculo (cilindro, retângulo, degrau), sendo os nós do contorno do obstáculo inseridos na malha com a condição $\psi = \text{constante}$. As velocidades são recuperadas por:

$$u = \frac{\partial \psi}{\partial y} \approx \frac{1}{2A^e} \sum_j \psi_j c_j, \quad v = -\frac{\partial \psi}{\partial x} \approx -\frac{1}{2A^e} \sum_j \psi_j b_j \quad (5.1)$$

A Figura 5.7 apresenta as linhas de corrente para escoamento ao redor de um obstáculo.

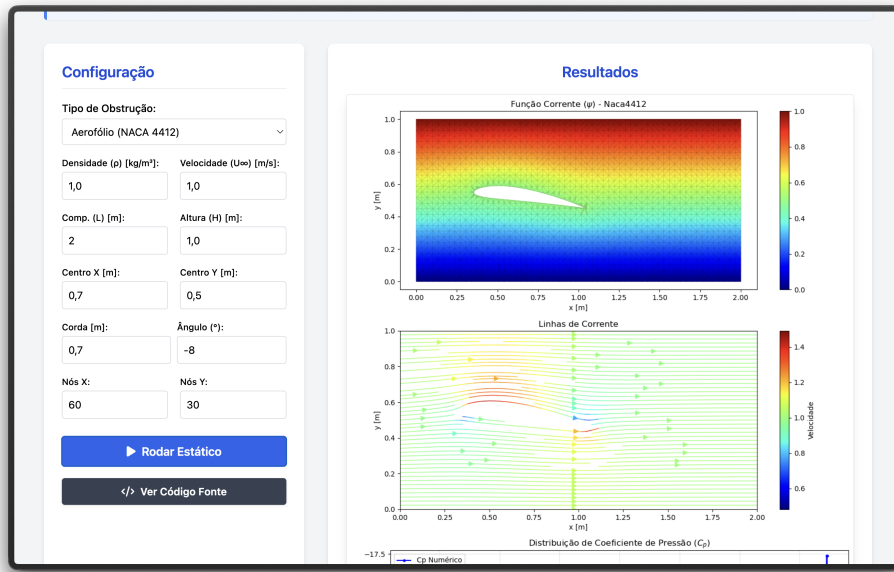


Figura 5.7: Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.

5.1.8 Navier-Stokes 2D (Vorticidade-Corrente)

O módulo mais complexo da plataforma implementa o solver de Navier-Stokes 2D na formulação Vorticidade-Corrente (ω - ψ), conforme descrito na Seção 3.5 (caso validado na Seção 6.9). A discretização temporal segue o esquema semi-implícito derivado na Seção 3.6.2 (Eq. 3.38), enquanto as condições de contorno de vorticidade nas paredes são impostas conforme a Eq. 3.39. O algoritmo segue o procedimento iterativo:

1. Resolver a equação de Poisson $\mathbf{K}\psi = \mathbf{M}\omega$ para ψ ;
2. Atualizar a vorticidade nas paredes sólidas conforme a Eq. 3.39;

3. Calcular (u, v) a partir de ψ (Eq. 5.1);
4. Assemblar a matriz de convecção $\mathbf{C}^n$ com o campo de velocidades atual;
5. Resolver o transporte da vorticidade pelo esquema semi-implícito (Eq. 3.38).

O solver suporta dois cenários: escoamento em canal com obstáculos (cilindro, retângulo, degrau) e cavidade com tampa móvel (*lid-driven cavity*). No caso da cavidade, todas as paredes possuem $\psi = 0$ e a parede superior impõe velocidade tangencial $u_{\text{lid}} = 1$, acrescentando o termo $-2u_{\text{tan}}/h$ à condição de contorno da vorticidade. A Figura 5.8 mostra os campos de ψ , ω e as linhas de corrente para a cavidade a $\text{Re} = 100$.

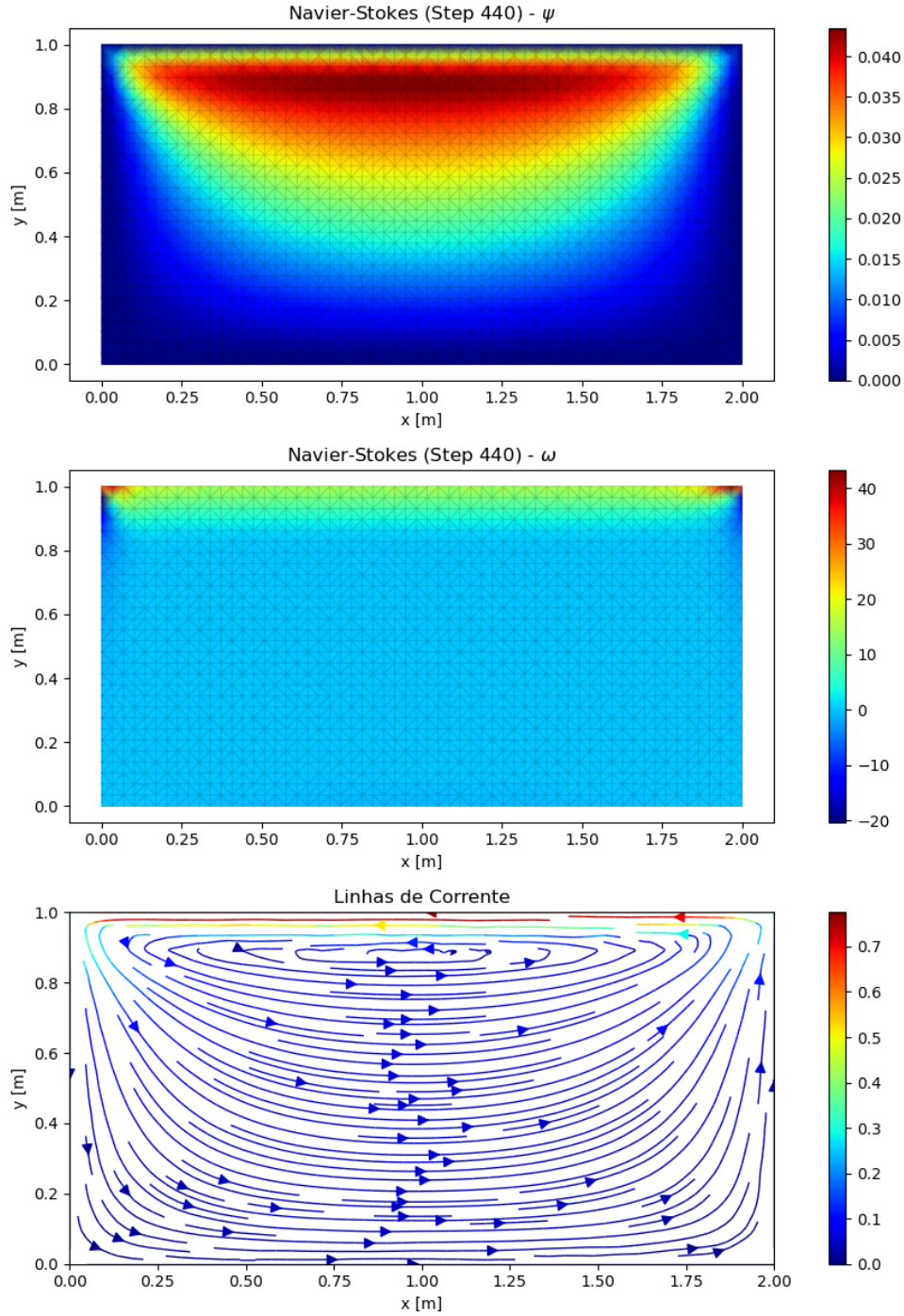


Figura 5.8: Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.

5.2 Módulos Complementares em MDF

Além dos oito módulos MEF descritos na seção anterior, a plataforma disponibiliza dois módulos implementados pelo Método das Diferenças Finitas, desenvolvidos nas

etapas iniciais do projeto e mantidos no portfólio por seu valor didático-comparativo:

- **Condução Permanente 1D com Geração Interna:** resolve o mesmo problema térmico da Seção 5.1.1, substituindo a discretização variacional pela aproximação de derivadas via expansão de Taylor (Seção 3.3), o que permite contrastar as duas formulações para um fenômeno físico idêntico.
- **Equação da Onda 1D:** problema hiperbólico linear com avanço temporal explícito, incluído como exercício conceitual de integração temporal em malha estruturada.

Ambos os módulos reproduzem suas respectivas soluções analíticas com erro numérico compatível com a precisão de máquina. Como constituem implementações iniciais de caráter preparatório e o escopo central do trabalho recai sobre o MEF, a validação quantitativa apresentada no Capítulo 6 concentra-se exclusivamente nos módulos de elementos finitos.

5.3 Síntese do Capítulo

Do ponto de vista da integração teórico-computacional, todos os módulos descritos neste capítulo compartilham o mesmo núcleo numérico: construção de matrizes elementares, montagem global por conectividade nodal, solução de sistemas lineares e, nos casos transientes, avanço temporal via método θ ou esquema semi-implícito. Dessa forma, a plataforma *MinervaMesh* operacionaliza integralmente o arcabouço teórico estabelecido no Capítulo 3, mantendo correspondência direta entre as equações governantes contínuas, os sistemas discretizados obtidos pela formulação de Galerkin e os *solvers* efetivamente implementados. Listagens representativas dos códigos constam no Apêndice A — com o código integral disponível no repositório público da plataforma —, e os *scripts* de *benchmark* computacional cruzado entre CPython e Pyodide constam no Apêndice B. Os oito módulos MEF cobrem três domínios da Engenharia Mecânica (térmico, estrutural e fluidodinâmico), demonstrando a versatilidade da formulação de elementos finitos; a validação quantitativa de cada um — comparação com soluções analíticas e *benchmarks* da literatura — é apresentada no Capítulo 6.

Capítulo 6

Resultados e Validação

6.1 Introdução

Este capítulo apresenta a validação numérica dos oito módulos de simulação por Elementos Finitos do *MinervaMesh*, cuja plataforma de execução e implementação são detalhadas, respectivamente, nos Capítulos 4 e 5. Cada caso simulado é comparado com soluções analíticas conhecidas ou *benchmarks* consagrados da literatura, de modo a verificar a corretude da discretização e quantificar os erros numéricos associados.

A estratégia de validação segue um critério de complexidade crescente. Os primeiros casos — condução de calor em regime permanente e transiente — envolvem problemas unidimensionais com solução analítica fechada, permitindo o cálculo direto de métricas de erro como o erro máximo absoluto e o erro relativo na norma L^2 . Na sequência, os problemas de convecção-difusão testam a estabilização SUPG, comparando a solução estabilizada com a solução Galerkin padrão em regime de alto número de Péclet. Os módulos estruturais — flexão e vibração de barras — são validados contra soluções clássicas da Resistência dos Materiais e da Dinâmica Estrutural. Finalmente, os casos bidimensionais de transferência de calor, escoamento potencial e escoamento viscoso (Navier-Stokes) completam a validação com problemas de geometria e condições de contorno mais complexas. A Tabela 6.1 resume os oito casos apresentados neste capítulo, com a respectiva referência de validação.

Tabela 6.1: Visão geral dos casos de validação apresentados neste capítulo.

Caso	Domínio	Referência de validação	Seção
Condução permanente 1D	Térmico	Solução analítica polinomial	6.2
Condução transiente 1D, $k(x)$	Térmico	Regime permanente analítico	6.3
Convecção-difusão 1D (SUPG)	Transporte	Solução analítica exponencial	6.4
Viga 1D (Euler-Bernoulli)	Estrutural	Resistência dos Materiais	6.5
Vibração 1D (autovalor)	Estrutural	Frequências naturais analíticas	6.6
Transferência de calor 2D	Térmico	Regime permanente analítico	6.7
Escoamento potencial 2D	Fluidos	Solução analítica (cilindro)	6.8
Navier-Stokes 2D (cavidade)	Fluidos	<i>Benchmark</i> de Ghia et al. [24]	6.9

Verificação e validação. Convém distinguir dois tipos complementares de teste no contexto de simulação numérica: *verificação* consiste em garantir que o código resolve corretamente as equações que se propõe a resolver, tipicamente confrontando a saída numérica contra soluções analíticas ou *benchmarks* de alta resolução; *validação* consiste em avaliar se as equações resolvidas descrevem adequadamente o fenômeno físico real, o que requer confronto com dados experimentais e quantificação formal de incerteza. Os oito casos apresentados neste capítulo inscrevem-se integralmente na primeira categoria: seis contra solução analítica fechada (Seções 6.2 a 6.7), escoamento potencial contra referência teórica clássica (Seção 6.8) e Navier-Stokes 2D contra o *benchmark* de Ghia et al. [24] (Seção 6.9). A validação experimental, com medidas laboratoriais e análise formal de incertezas, não constitui escopo do presente trabalho e fica indicada como extensão natural nos trabalhos futuros (Capítulo 7).

Cada caso é apresentado de forma uniforme: parte-se da definição do problema — equação governante, domínio, condições de contorno e parâmetros adotados —, estabelece-se a referência de validação (solução analítica ou *benchmark* adotado para comparação), expõem-se graficamente os campos obtidos pelo simulador e, por fim, quantifica-se a aderência à referência por meio de métricas de erro.

Todos os resultados foram gerados diretamente no navegador, utilizando a plataforma *MinervaMesh* em execução local via PyScript/WebAssembly, conforme descrito no Capítulo 4. Os parâmetros de entrada de cada simulação são reproduzidos integralmente nas tabelas a seguir, garantindo a reprodutibilidade dos experimentos.

6.2 Condução Permanente 1D

O primeiro caso de validação consiste no problema de condução de calor em regime permanente, em uma barra unidimensional de comprimento L com geração interna de calor Q e condições de contorno de Dirichlet em ambas as extremidades. Este problema constitui o caso mais simples de validação do MEF, pois admite solução analítica exata e envolve elementos lineares unidimensionais, correspondendo diretamente à formulação desenvolvida na Seção 3.4.1.

6.2.1 Formulação do Problema

O problema é governado pela equação de Poisson unidimensional, caso particular da equação geral de condução (Equação 3.1):

$$-k \frac{d^2 T}{dx^2} = Q, \quad x \in [0, L] \quad (6.1)$$

com as condições de contorno:

$$T(0) = T_0, \quad T(L) = T_L \quad (6.2)$$

A Tabela 6.2 apresenta os parâmetros utilizados na simulação.

Tabela 6.2: Parâmetros da simulação de condução permanente 1D.

Parâmetro	Símbolo	Valor
Comprimento da barra	L	1,0 m
Número de elementos	n_e	10
Temperatura em $x = 0$	T_0	0,0 °C
Temperatura em $x = L$	T_L	1,0 °C
Condutividade térmica	k	1,0 W/(m·°C)
Geração interna	Q	4,0 W/m ³

6.2.2 Solução Analítica

A integração direta da Equação (6.1) fornece a solução exata:

$$T(x) = -\frac{Q}{2k}x^2 + C_1x + C_2 \quad (6.3)$$

onde as constantes de integração são determinadas pelas condições de contorno:

$$C_2 = T_0, \quad C_1 = \frac{T_L - T_0 + \frac{QL^2}{2k}}{L} \quad (6.4)$$

Substituindo os valores da Tabela 6.2, obtém-se $C_1 = 3,0 \text{ °C/m}$ e $C_2 = 0,0 \text{ °C}$. O perfil de temperatura resultante é uma parábola com valor máximo $T_{\max} \approx 1,125 \text{ °C}$ em $x^* = 0,75 \text{ m}$, no interior do domínio — superior ao valor prescrito em qualquer das extremidades ($T_0 = 0 \text{ °C}$, $T_L = 1,0 \text{ °C}$) e decorrente da competição entre a geração interna de calor e a difusão para as fronteiras.

6.2.3 Solução MEF e Comparação

A discretização por elementos finitos utiliza $n_e = 10$ elementos lineares com espaçamento uniforme $h = L/n_e = 0,1 \text{ m}$, resultando em 11 nós. A montagem do sistema global e a imposição das condições de contorno de Dirichlet seguem o procedimento descrito nas Seções 3.4.3 e 3.4.6. Para este problema de Poisson unidimensional com elementos lineares, a solução MEF coincide exatamente com a solução analítica nos nós da malha (propriedade bem conhecida da interpolação linear por partes para polinômios de grau ≤ 2).

A Figura 6.1 apresenta os resultados obtidos. A curva MEF (pontos azuis) está perfeitamente sobreposta à solução analítica (linha tracejada vermelha), confirmando a corretude da implementação.

6.2.4 Análise de Erro

Para este caso particular — equação de Poisson 1D com fonte constante e elementos lineares — a solução analítica é um polinômio de grau 2. Como a interpolação linear reproduz exatamente qualquer polinômio de grau ≤ 2 nos nós da malha, espera-se que o erro nodal seja da ordem do erro de arredondamento de máquina ($\sim 10^{-15}$). De fato, ao calcular o erro máximo absoluto e o erro relativo na norma L^2 :

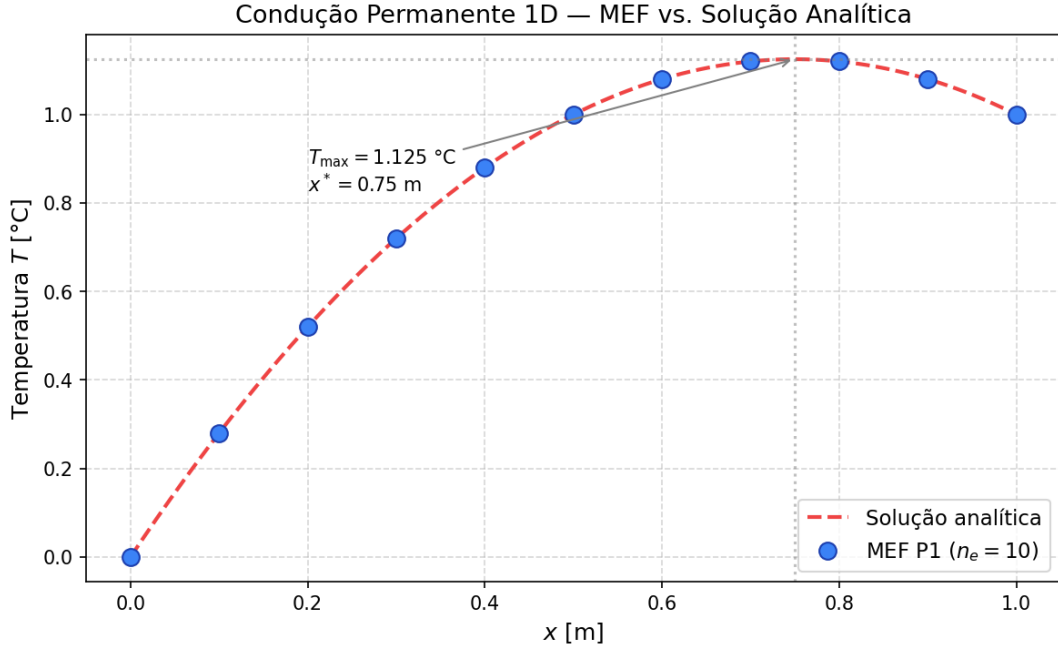


Figura 6.1: Condução permanente 1D: comparação entre a solução MEF (pontos azuis, $n_e = 10$ elementos lineares) e a solução analítica (linha tracejada vermelha). A sobreposição exata confirma a propriedade de reprodução polinomial dos elementos lineares para fontes uniformes.

$$e_{\max} = \max_i |T_i^{\text{MEF}} - T_i^{\text{ana}}|, \quad e_{L^2} = \frac{(\sum_i (T_i^{\text{MEF}} - T_i^{\text{ana}})^2)^{1/2}}{(\sum_i (T_i^{\text{ana}})^2)^{1/2}} \quad (6.5)$$

obtem-se $e_{\max} \approx 10^{-15}$ °C e $e_{L^2} \approx 10^{-15}$, confirmando erro essencialmente nulo nos nós da malha. Este resultado era esperado teoricamente e valida a montagem das matrizes elementares de rigidez e do vetor de carga, a montagem global e a imposição das condições de contorno de Dirichlet implementadas no módulo `conducao-permanente-barra1d-mef`.

Nota sobre a precisão nodal exata. A concordância perfeita nos nós não implica erro nulo entre os nós. O campo MEF é linear por partes, enquanto a solução exata é parabólica. O erro de interpolação entre nós é da ordem $O(h^2)$. Essa observação será relevante nos casos subsequentes, que envolvem equações com soluções de maior grau ou não-polinomiais.

6.3 Condução Transiente 1D com $k(x)$ Variável

O segundo caso de validação estende o problema de condução unidimensional para o regime transiente, introduzindo duas dificuldades adicionais: dependência temporal, tratada pelo método θ (Crank-Nicolson, $\theta = 1/2$), e heterogeneidade material, representada por uma condutividade descontínua em $x = L/2$. Este cenário verifica simultaneamente a implementação da matriz de massa consistente, o esquema de integração temporal e a montagem correta das matrizes elementares quando o coeficiente material varia dentro do domínio.

6.3.1 Formulação do Problema

O balanço de energia em regime transiente sem geração interna é descrito pela equação parabólica:

$$\rho c_p \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(k(x) \frac{\partial T}{\partial x} \right), \quad x \in [0, L], \quad t > 0 \quad (6.6)$$

com condições de contorno de Dirichlet fixas no tempo e condição inicial homogênea:

$$T(0, t) = 0, \quad T(L, t) = 1, \quad T(x, 0) = 0 \quad (6.7)$$

e condutividade descontínua em $x = L/2$:

$$k(x) = \begin{cases} k_1, & 0 \leq x < L/2 \\ k_2, & L/2 < x \leq L \end{cases} \quad (6.8)$$

A Tabela 6.3 resume os parâmetros adotados, todos correspondentes aos valores *default* da interface HTML do módulo.

6.3.2 Solução Analítica de Regime Permanente

No limite $t \rightarrow \infty$, o termo transiente se anula e a solução satisfaz $\frac{d}{dx}(k(x) dT/dx) = 0$, o que implica fluxo constante $k(x) dT/dx = C$ ao longo de todo o domínio. Em cada sub-região de condutividade uniforme, T é linear; a continuidade da temperatura em $x = L/2$ e a continuidade do fluxo fornecem o sistema

Tabela 6.3: Parâmetros da simulação de condução transiente 1D com $k(x)$ variável.

Parâmetro	Símbolo	Valor
Comprimento da barra	L	1,0 m
Número de elementos	n_e	40
Condutividade na primeira metade	k_1	5,0 W/(m·°C)
Condutividade na segunda metade	k_2	1,0 W/(m·°C)
Capacidade térmica volumétrica	ρc_p	1,0 J/(m ³ ·°C)
Passo de tempo	Δt	10 ⁻³ s
Parâmetro do método θ	θ	0,5
Tempo final de simulação	$t_{\max}$	0,2 s

$$a_1 + a_2 = \frac{T_L - T_0}{L/2}, \quad k_1 a_1 = k_2 a_2 \quad (6.9)$$

onde a_1 e a_2 são as inclinações em cada metade. Resolvendo com $T_0 = 0$, $T_L = 1$, $k_1 = 5$, $k_2 = 1$:

$$a_1 = \frac{1}{3} \text{ °C/m}, \quad a_2 = \frac{5}{3} \text{ °C/m}, \quad T(L/2) = \frac{1}{6} \approx 0,1667 \text{ °C} \quad (6.10)$$

O perfil permanente é portanto uma linha poligonal com *kink* em $x = L/2$: inclinação suave no lado de maior condutividade e íngreme no lado de menor condutividade, refletindo a continuidade do fluxo térmico. A escala de tempo característica de difusão é $\tau_{\text{dif}} = L^2 \rho c_p / \bar{k} \sim 1,0$ s, com $\bar{k}$ condutividade efetiva.

6.3.3 Solução MEF e Comparação

A discretização emprega $n_e = 40$ elementos lineares com $h = 0,025$ m, o que resolve bem a descontinuidade de condutividade alinhando um nó exatamente com $x = L/2$. A cada passo de tempo, o sistema linear

$$\left(\frac{M}{\Delta t} + \theta K \right) \mathbf{T}^{n+1} = \left(\frac{M}{\Delta t} - (1 - \theta)K \right) \mathbf{T}^n \quad (6.11)$$

é resolvido pelo solver direto, onde M e K são as matrizes globais consistentes de massa e de rigidez montadas com $k(x)$ avaliado no baricentro de cada elemento.

A Figura 6.2 apresenta o campo $T(x)$ no instante final $t = 0,2$ s (correspondente a $\sim 20\%$ da escala difusiva) sobreposto ao perfil analítico de regime permanente, evidenciando a convergência rápida do método ao regime linear por partes.

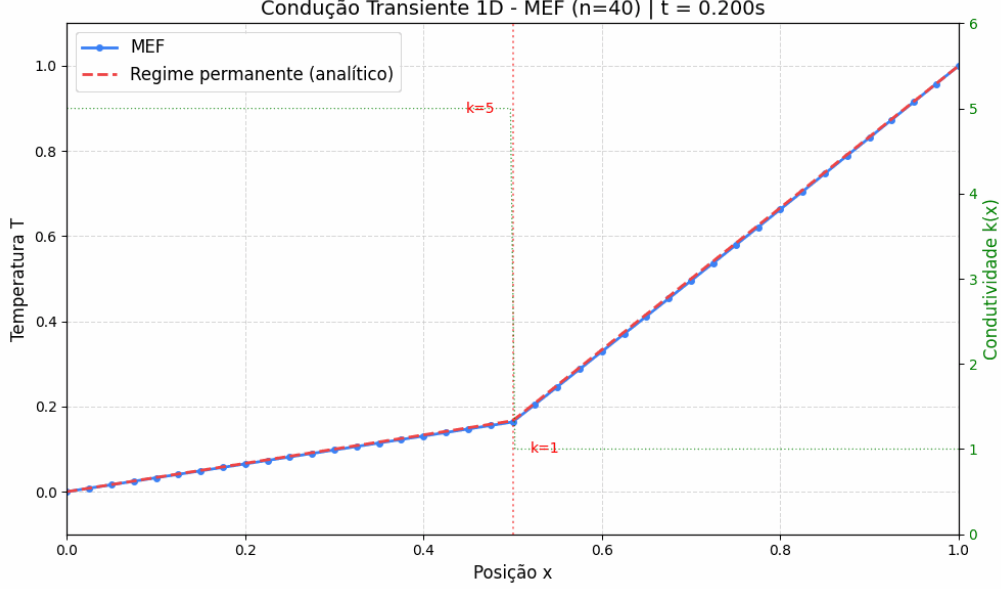


Figura 6.2: Condução transiente 1D com condutividade descontínua ($k_1 = 5$, $k_2 = 1$): solução MEF no instante final $t = 0,2$ s (pontos azuis) comparada ao perfil analítico de regime permanente (linha tracejada vermelha). A curva verde tracejada no eixo secundário indica a variação de $k(x)$ com a descontinuidade em $x = L/2$. A malha de 40 elementos lineares com $\theta = 1/2$ reproduz fielmente o *kink* de condutividade, com $T(L/2) \approx 1/6$.

6.3.4 Análise de Erro

Em $t_{\max} = 0,2$ s (20% da escala de tempo difusiva $\tau_{\text{dif}} \sim 1,0$ s), a solução MEF já se encontra em regime quase-permanente: a figura mostra os pontos nodais visualmente coincidentes com a linha poligonal analítica, incluindo a captura correta do *kink* de condutividade em $x = L/2$. A convergência pontual segue o decaimento exponencial esperado para equações parabólicas, $|T(x, t) - T_{\infty}(x)| \sim e^{-t/\tau}$, de modo que em $t \gtrsim 2\tau_{\text{dif}}$ o erro nodal atinge a precisão de máquina, dado que a solução de regime permanente (linear por partes) pertence exatamente ao espaço de aproximação dos elementos lineares com nós alinhados à descontinuidade.

Nota sobre a propagação térmica. Para tempos menores, inferiores a τ_{dif} , o campo exibe a perturbação térmica se propagando da parede quente ($x = L$, onde $T_L = 1$) para o interior da barra, com frente de difusão progredindo à razão $\sqrt{\kappa t}$ ($\kappa = k/\rho c_p$). A descontinuidade de condutividade em $x = L/2$ introduz um retardo do avanço da frente quando esta atravessa para a região de menor condutividade ($k_2 = 1$), comportamento observável iterativamente ao se reduzir t_{max} na interface.

6.4 Convecção-Difusão 1D com Estabilização SUPG

O terceiro caso aborda a equação de convecção-difusão estacionária em uma dimensão, cenário em que a formulação Galerkin clássica perde estabilidade quando a convecção predomina sobre a difusão. Este problema é particularmente útil para validação porque admite solução analítica fechada em todo o espectro de números de Péclet e porque expõe uma propriedade teórica muito forte da estabilização SUPG: com o parâmetro canônico de Christie et al. [21], o erro nodal é da ordem da precisão de máquina independentemente de Pe_e .

6.4.1 Formulação do Problema

A forma forte do problema estacionário unidimensional é

$$-k \frac{d^2 T}{dx^2} + \rho c_p u \frac{dT}{dx} = Q, \quad x \in [0, L] \quad (6.12)$$

com condições de contorno de Dirichlet em ambas as extremidades, $T(0) = T_0$ e $T(L) = T_L$. O parâmetro adimensional relevante para a escolha da discretização é o número de Péclet elementar

$$\text{Pe}_e = \frac{\rho c_p u h_e}{2k} \quad (6.13)$$

onde h_e é o tamanho do elemento. A Tabela 6.4 lista os parâmetros adotados, correspondentes aos *defaults* da interface do módulo.

Tabela 6.4: Parâmetros da simulação de convecção-difusão 1D.

Parâmetro	Símbolo	Valor
Comprimento	L	1,0 m
Número de elementos	n_e	50
Temperatura em $x = 0$	T_0	0,0 °C
Temperatura em $x = L$	T_L	1,0 °C
Condutividade térmica	k	1,0 W/(m·°C)
Capacidade térmica volumétrica	ρc_p	10^3 J/(m ³ ·°C)
Geração interna	Q	1,0 W/m ³
Velocidade de convecção	u	0,05 m/s
Péclet elementar	Pe_e	0,5
Estabilização SUPG	—	ativa

6.4.2 Solução Analítica

Para $Q = 0$, a equação admite a solução exponencial clássica; com fonte constante, a solução geral é soma da homogênea com uma particular linear. O resultado pode ser escrito na forma

$$T(x) = T_0 + \frac{Qx}{\rho c_p u} + \left(T_L - T_0 - \frac{QL}{\rho c_p u} \right) \frac{e^{\alpha x} - 1}{e^{\alpha L} - 1}, \quad \alpha = \frac{\rho c_p u}{k} \quad (6.14)$$

Para $\alpha L \gg 1$, a exponencial domina no intervalo próximo a $x = L$, concentrando toda a variação do campo em uma camada-limite de espessura $\sim 1/\alpha$. Com os *defaults* da interface, $\alpha L = 50$ e a camada-limite é estreita porém bem resolvida pela malha uniforme de 50 elementos. Em regimes de $Pe_e > 1$ a camada-limite ocupa apenas uma fração do elemento, situação em que a formulação Galerkin clássica apresenta oscilações espúrias e a estabilização SUPG se torna indispensável.

6.4.3 Solução MEF com SUPG

A formulação Petrov-Galerkin introduz funções-peso modificadas $\tilde{w}_i = w_i + \tau u dw_i/dx$, com o parâmetro de estabilização

$$\tau_{\text{SUPG}} = \frac{h_e}{2|u|} \left(\coth \text{Pe}_e - \frac{1}{\text{Pe}_e} \right) \quad (6.15)$$

derivado por Christie et al. [21] a partir da exigência de anular o erro nodal no problema homogêneo. As Figuras 6.3 e 6.4 comparam, em $\text{Pe}_e = 0,5$, a solução MEF com e sem a estabilização SUPG sobreposta à solução analítica, ambas rodadas diretamente a partir da interface do módulo.

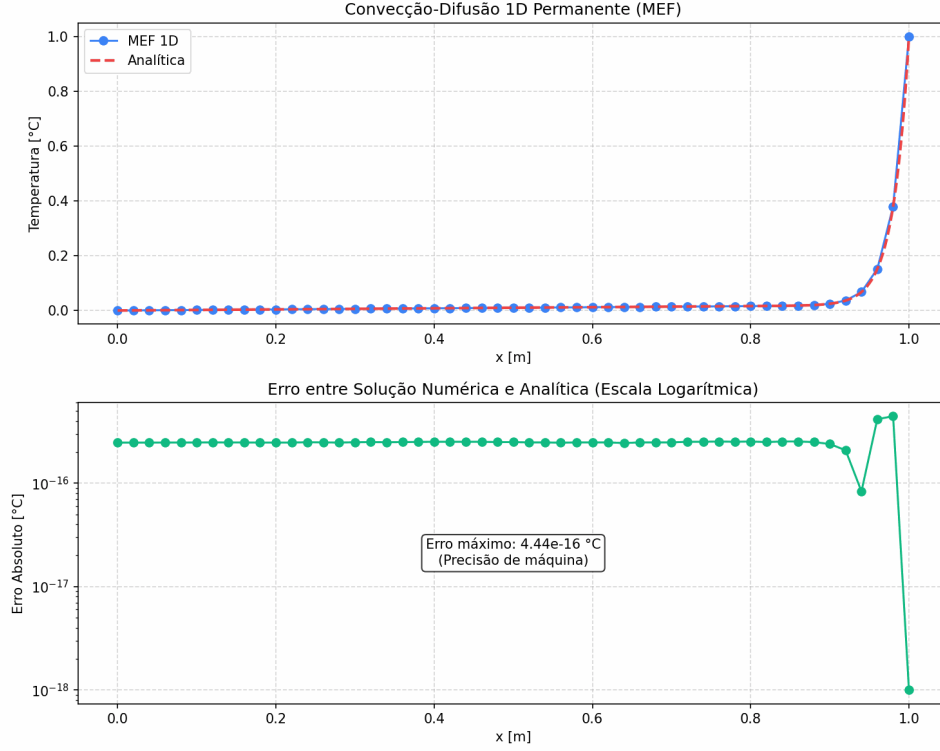


Figura 6.3: Convecção-difusão 1D com SUPG ativo em $\text{Pe}_e = 0,5$. Painel superior: temperatura $T(x)$ — pontos azuis para o MEF e linha tracejada vermelha para a solução analítica, visualmente coincidentes em toda a malha. Painel inferior: erro absoluto $|T^{\text{MEF}} - T^{\text{ana}}|$ em escala logarítmica, com máximo $4,44 \times 10^{-16} \text{ °C}$ — precisão de máquina.

6.4.4 Análise de Erro

Os erros nodais máximos observados nas duas configurações da mesma malha são

$$e_{\text{max}}^{\text{SUPG}} \approx 4,44 \times 10^{-16} \text{ °C}, \quad e_{\text{max}}^{\text{Galerkin}} \approx 3,39 \times 10^{-2} \text{ °C} \quad (6.16)$$

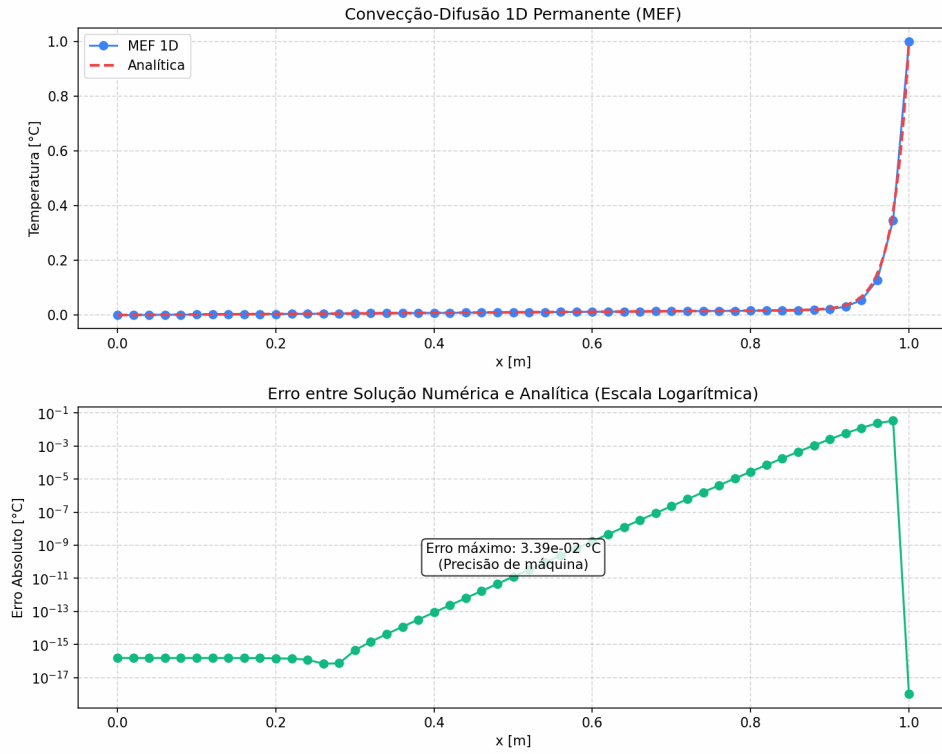


Figura 6.4: Convecção-difusão 1D com Galerkin clássico (SUPG desativado), mesmos parâmetros da figura anterior. O painel superior mostra ainda concordância visual em escala linear; o painel inferior de erro revela, entretanto, crescimento exponencial do desvio em direção à camada-limite ($x \rightarrow L$), atingindo máximo $3,39 \times 10^{-2}$ °C próximo ao contorno direito.

uma diferença de aproximadamente 14 ordens de grandeza entre os dois métodos. A SUPG reproduz a analítica com erro da ordem da precisão de máquina — concordância de 16 dígitos significativos. Este não é um acidente numérico: Christie et al. [21] demonstraram que o τ canônico da Equação (6.15) é precisamente a escolha que torna o estêncil SUPG idêntico ao esquema *upwind* exponencial, que reproduz a solução analítica exatamente nos nós da malha para convecção-difusão 1D com fonte constante. O resultado constitui portanto verificação rigorosa da implementação da matriz de convecção, da ponderação SUPG e da integração das funções de peso modificadas.

Nota sobre o comportamento de Galerkin. Mesmo em $Pe_e < 1$ (regime tecnicamente estável), o erro Galerkin já é observável na região da camada-limite, como mostra o painel inferior da Figura 6.4. Para $Pe_e > 1$, oscilações espúrias de sinal alternado entre nós adjacentes dominam o campo próximo ao contorno quente. Refinar a malha até garantir $Pe_e < 1$ em toda a parte elimina as oscilações, mas a um custo computacional proibitivo em problemas 2D/3D; a atratividade do SUPG reside justamente em permitir precisão nodal exata sem refinamento adicional.

6.5 Viga 1D — Flexão de Euler-Bernoulli

Este caso valida o módulo de flexão de vigas pela teoria de Euler-Bernoulli, baseado em elementos de Hermite com funções de forma cúbicas e quatro graus de liberdade por elemento (deslocamento transversal e rotação em cada nó). A validação é particularmente rigorosa porque a solução analítica da flexão sob carga uniforme é um polinômio de grau 4, que coincide exatamente com o espaço de aproximação do elemento Hermite — espera-se, portanto, concordância nodal à precisão de máquina, caracterizando a chamada *superconvergência nodal*.

6.5.1 Formulação do Problema

A equação diferencial de quarta ordem da teoria clássica de Euler-Bernoulli é

$$\frac{d^2}{dx^2} \left(EI \frac{d^2 w}{dx^2} \right) = q(x), \quad x \in [0, L] \quad (6.17)$$

onde $w(x)$ é a deflexão transversal, E é o módulo de Young, I o momento de inércia da seção e $q(x)$ a carga distribuída. Para validação, adota-se o caso clássico de viga simplesmente apoiada sob carga uniforme, com propriedades constantes ao longo do comprimento. A Tabela 6.5 lista os parâmetros, correspondentes aos *defaults* da interface do módulo.

Tabela 6.5: Parâmetros da simulação de flexão de viga 1D.

Parâmetro	Símbolo	Valor
Comprimento da viga	L	2,0 m
Número de elementos	n_e	20
Módulo de Young	E	200 GPa
Momento de inércia	I	1000 cm ⁴
Carga distribuída uniforme	q	10,0 kN/m
Tipo de apoio	—	simplesmente apoiada

6.5.2 Solução Analítica

Para EI constante, carga uniforme q e condições de apoio simples ($w(0) = w(L) = 0$, $M(0) = M(L) = 0$), a dupla integração da Equação (6.17) fornece

$$w(x) = \frac{q}{24EI} (L^3x - 2Lx^3 + x^4) \quad (6.18)$$

$$M(x) = EI \frac{d^2w}{dx^2} = \frac{qx(L-x)}{2} \quad (6.19)$$

$$V(x) = \frac{dM}{dx} = q \left(\frac{L}{2} - x \right) \quad (6.20)$$

com valores máximos clássicos $w_{\max} = 5qL^4/(384EI)$ em $x = L/2$, $M_{\max} = qL^2/8$ em $x = L/2$ e $V_{\max} = qL/2$ nos apoios. Substituindo os valores da Tabela 6.5, obtém-se $w_{\max} = 1,0417$ mm, $M_{\max} = 5,0$ kN·m e $V_{\max} = 10,0$ kN.

6.5.3 Solução MEF e Comparação

A discretização adota $n_e = 20$ elementos de Hermite cúbicos, totalizando 42 graus de liberdade globais antes da imposição das condições de contorno. Como as

funções de forma de Hermite incluem o polinômio x^4 em sua base (quando montadas elemento-a-elemento), e como a solução analítica da Equação (6.18) é precisamente um polinômio de grau 4, o espaço de aproximação do MEF contém a solução exata. A Figura 6.5 apresenta as três quantidades de interesse: deflexão $w(x)$, momento fletor $M(x)$ e esforço cortante $V(x)$, comparados às respectivas soluções analíticas.

6.5.4 Análise de Erro

O erro máximo absoluto na deflexão nodal é

$$e_{\max}^w = \max_i |w_i^{\text{MEF}} - w_i^{\text{ana}}| \approx 7,1 \times 10^{-16} \text{ m} \quad (6.21)$$

isto é, da ordem da precisão de máquina. A deflexão no centro da viga, $w(L/2) = 1,0417 \times 10^{-3} \text{ m}$, é reproduzida em todos os 16 dígitos significativos. Esta concordância exemplar é consequência direta da propriedade de *reprodução polinomial* dos elementos de Hermite cúbicos: como a base do elemento inclui monômios até o grau 3 e como a combinação das bases de dois elementos adjacentes gera a reconstrução exata de polinômios até grau 4 sob integração, a solução MEF é matematicamente idêntica à analítica nos nós. A mesma propriedade se mantém para as condições de contorno de viga em balanço ($w_{\max} = qL^4/(8EI) = 10,0 \text{ mm}$) e biengastada ($w_{\max} = qL^4/(384EI) = 0,208 \text{ mm}$), ambas com erro nodal da ordem de 10^{-13} m ou inferior.

Nota sobre momento e cortante. As curvas de $M(x)$ e $V(x)$ exibidas na Figura 6.5 são obtidas por pós-processamento: M via segunda derivada por diferenças finitas centradas da deflexão nodal, V via primeira derivada de M . Os pontos próximos aos apoios são omitidos porque o estêncil assimétrico nas bordas amplifica o ruído de arredondamento; o miolo da viga reproduz as distribuições parabólica (M) e linear (V) com concordância visual perfeita. Uma reconstrução mais robusta dessas quantidades exigiria derivação analítica a partir da interpolação de Hermite, o que constitui uma possível extensão futura.

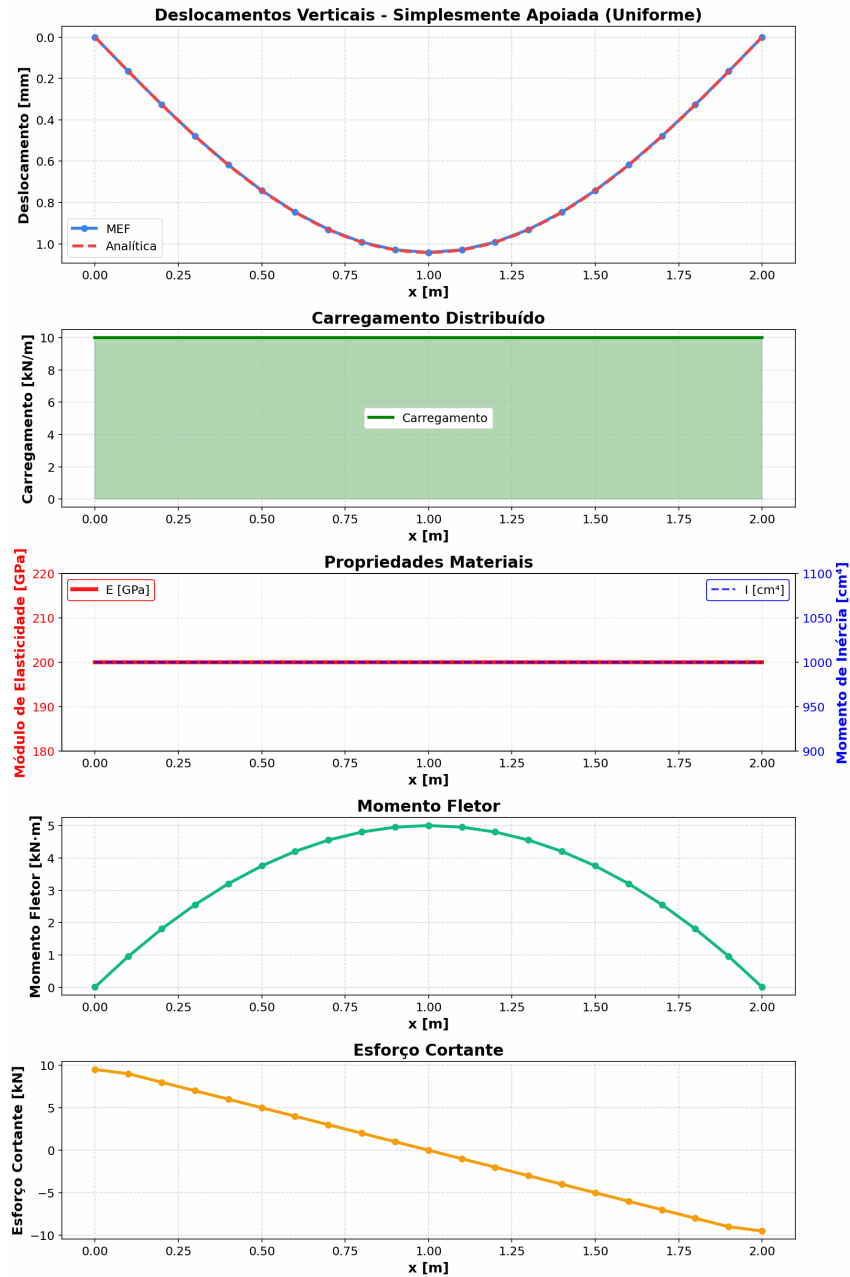


Figura 6.5: Viga simplesmente apoiada com carga uniforme: (i) deflexão $w(x)$ com MEF (pontos azuis) e analítica (linha vermelha) visualmente coincidentes; (ii) perfil da carga distribuída constante $q = 10 \text{ kN/m}$; (iii) propriedades materiais E e I uniformes ao longo do vão; (iv) momento fletor $M(x)$ parabólico com máximo $qL^2/8 = 5,0 \text{ kN}\cdot\text{m}$ no centro; (v) esforço cortante $V(x)$ linear decrescente de $+qL/2 = +10 \text{ kN}$ no apoio esquerdo a -10 kN no direito. M e V são obtidos por pós-processamento da deflexão nodal.

6.6 Vibração 1D — Problema de Autovalor

O quinto caso valida o solver modal de vibrações axiais em barra unidimensional, formulado como um problema generalizado de autovalor. Este cenário verifica simultaneamente a montagem da matriz de massa concentrada (*lumped*), a solução do sistema $K\phi = \lambda M\phi$ por algoritmo de decomposição, e a recuperação das formas modais normalizadas. A solução analítica para barras com propriedades constantes é bem estabelecida e envolve funções seno, o que introduz um erro sistemático de discretização que cresce com a ordem do modo — caracterizando o comportamento canônico da aproximação por mass lumping.

6.6.1 Formulação do Problema

A equação de onda axial em barra de seção constante é

$$\rho A \frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x} \left(EA \frac{\partial u}{\partial x} \right), \quad x \in [0, L] \quad (6.22)$$

onde $u(x, t)$ é o deslocamento axial, E o módulo de Young, ρ a densidade e A a área da seção transversal. A separação de variáveis $u(x, t) = \phi(x)e^{i\omega t}$ conduz ao problema espectral generalizado, cuja discretização por MEF é

$$K\phi_n = \omega_n^2 M\phi_n, \quad f_n = \frac{\omega_n}{2\pi} \quad (6.23)$$

com K a matriz de rigidez, M a matriz de massa concentrada e ϕ_n o n -ésimo autovetor (forma modal). A Tabela 6.6 lista os parâmetros da simulação, correspondentes ao preset de aço dos *defaults* da interface.

6.6.2 Solução Analítica

Para barra livre-livre (ambas as extremidades com $du/dx = 0$), as formas modais e frequências naturais são

$$\phi_n(x) = \cos\left(\frac{(n-1)\pi x}{L}\right), \quad f_n^{\text{ana}} = \frac{(n-1)c}{2L}, \quad n = 1, 2, 3, \dots \quad (6.24)$$

O primeiro modo ($n = 1$) corresponde a $f_1 = 0$, que representa o movimento de corpo rígido da barra (translação axial uniforme) — modo sempre presente quando ambas

Tabela 6.6: Parâmetros da simulação de vibração axial 1D (barra de aço bi-engastada).

Parâmetro	Símbolo	Valor
Comprimento	L	2,0 m
Número de elementos	n_e	20
Módulo de Young	E	200 GPa
Densidade	ρ	7850 kg/m ³
Área da seção	A	100 cm ²
Tipo de apoio	—	livre-livre
Velocidade de onda axial	$c = \sqrt{E/\rho}$	5047,5 m/s

as extremidades estão livres. Com $c = 5047,5$ m/s e $L = 2$ m, os cinco primeiros valores analíticos são $f_1 = 0$ Hz, $f_2 = 1261,9$ Hz, $f_3 = 2523,8$ Hz, $f_4 = 3785,7$ Hz e $f_5 = 5047,5$ Hz. As formas modais são cossenos com número crescente de meio-comprimentos de onda entre as extremidades.

6.6.3 Solução MEF e Comparação

A discretização emprega $n_e = 20$ elementos lineares com malha uniforme. A matriz de massa é montada na forma *lumped*, redistribuindo a massa elementar igualmente entre os dois nós do elemento — estratégia que torna M diagonal e acelera a solução do problema de autovalor, ao custo de introduzir um erro sistemático de subestimação das frequências. A Figura 6.6 apresenta as cinco primeiras formas modais normalizadas e a tabela de frequências obtidas pela solução do problema generalizado.

6.6.4 Análise de Erro

A Tabela 6.7 compara as frequências MEF com as analíticas para os cinco primeiros modos.

O modo de corpo rígido ($n = 1$) é reproduzido exatamente pelo MEF, com $f_1 = 0$ — consequência direta da formulação da matriz de rigidez, que possui exatamente um modo nulo para esta configuração de apoios. Para os modos deformáveis sub-

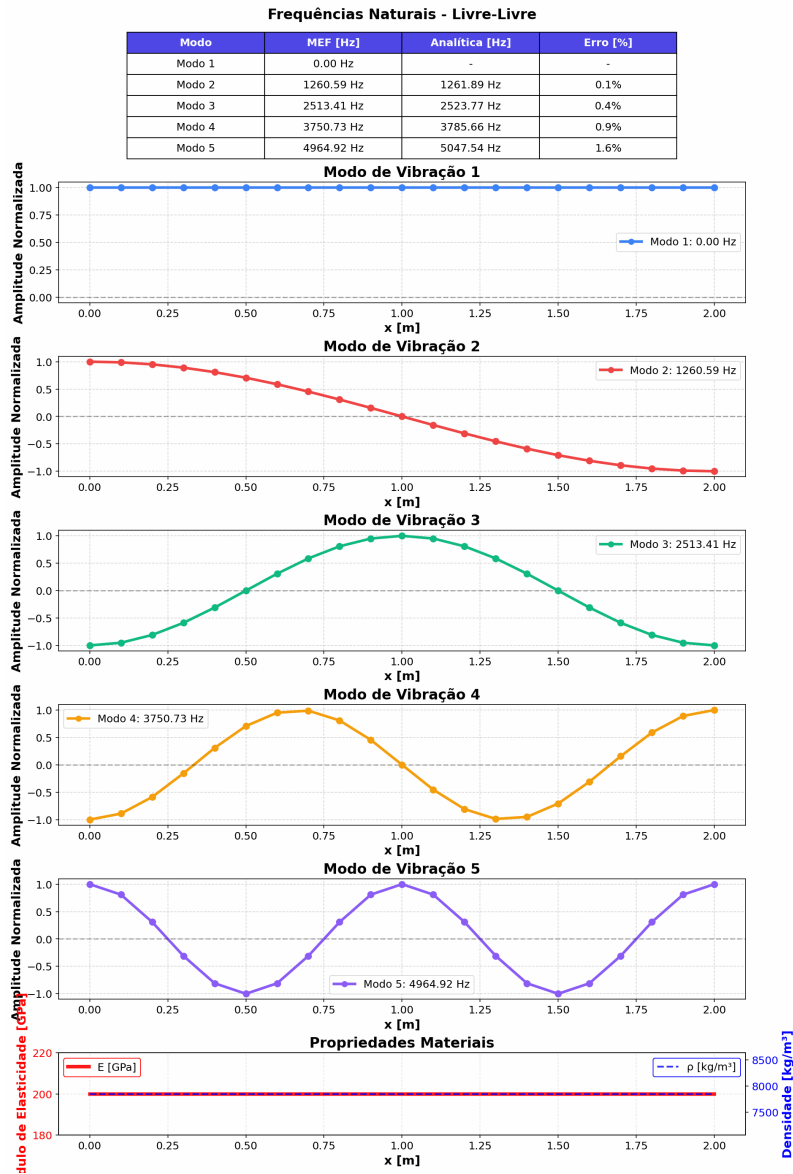


Figura 6.6: Cinco primeiros modos de vibração axial da barra livre-livre. Tabela superior: frequências MEF e analíticas com erro relativo. Painéis seguintes: forma modal normalizada (pontos e linha contínua, uma cor por modo), do modo 1 rígido ($f_1 = 0$ Hz, amplitude uniforme) aos modos 2–5 com número crescente de meio-comprimentos de onda. Painel inferior: propriedades materiais (E e ρ) constantes ao longo do vão.

Tabela 6.7: Frequências naturais: MEF vs. analítica para barra livre-livre.

Modo n	f_n^{MEF} [Hz]	f_n^{ana} [Hz]	Erro relativo
1	0,00	0,00	—
2	1260,59	1261,89	$1,0 \times 10^{-3}$
3	2513,41	2523,77	$4,0 \times 10^{-3}$
4	3750,73	3785,66	$9,0 \times 10^{-3}$
5	4964,92	5047,45	$1,6 \times 10^{-2}$

sequentes, todas as frequências MEF subestimam as analíticas, com erro relativo crescendo monotonicamente de 0,1% ($n = 2$) a 1,6% ($n = 5$). Esta subestimação é o comportamento canônico da aproximação por massa concentrada: a distribuição discreta da massa nos nós equivale, no limite contínuo, a uma barra levemente mais flexível do que a original, o que reduz as frequências. O erro cresce com a ordem do modo porque modos de alta frequência têm comprimento de onda comparável ao tamanho do elemento, saindo do regime de resolução confiável da malha. Para o módulo ora implementado, o critério prático é que os modos até $n_{\text{max}} \approx n_e/4$ podem ser considerados quantitativamente confiáveis.

Nota sobre a matriz de massa consistente. A alternativa clássica à massa concentrada é a matriz de massa consistente (tridiagonal), obtida integrando $\int N_i N_j \rho A dx$ sem aproximações. A massa consistente *sobrestima* as frequências (viés oposto ao *lumping*) e oferece maior precisão para modos baixos, mas torna M não-diagonal e encarece a solução do autovalor em problemas 2D/3D. A escolha do MinervaMesh pelo *lumping* é motivada por simplicidade pedagógica e pela convergência prática adequada para os primeiros modos.

6.7 Transferência de Calor Transiente 2D

O sexto caso constitui a primeira validação bidimensional do MinervaMesh. O problema é a evolução transiente do campo de temperatura em uma placa quadrada com condições de contorno de Dirichlet assimétricas (temperaturas distintas em topo e base, perfis lineares nas laterais), discretizada com elementos triangulares

lineares sobre uma malha estruturada e integrada no tempo pelo método de Crank-Nicolson. A validação verifica a montagem 2D das matrizes elementares, a aplicação de condições de contorno sobre fronteiras não-alinhadas com os eixos da malha e a convergência ao perfil de regime permanente, que admite solução analítica trivial.

6.7.1 Formulação do Problema

A equação do calor em duas dimensões espaciais, sem geração interna, é

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T), \quad (x, y) \in \Omega = [0, L] \times [0, L], \quad t > 0 \quad (6.25)$$

com condições de contorno de Dirichlet nos quatro lados e condição inicial homogênea:

$$\begin{aligned} T(x, 0, t) &= T_{\text{bot}}, \quad T(x, L, t) = T_{\text{top}} \\ T(0, y, t) &= T(L, y, t) = T_{\text{bot}} + (T_{\text{top}} - T_{\text{bot}}) \cdot y/L \\ T(x, y, 0) &= 0 \end{aligned} \quad (6.26)$$

A Tabela 6.8 lista os parâmetros adotados. A escala característica de difusão é $\tau_{\text{dif}} = L^2 \rho c_p / k = 1,0$ s.

Tabela 6.8: Parâmetros da simulação de transferência de calor transiente 2D.

Parâmetro	Símbolo	Valor
Domínio	Ω	$[0, 1] \times [0, 1]$ m ²
Malha (nós por direção)	$n_x \times n_y$	40×40
Condutividade térmica	k	1,0 W/(m·°C)
Densidade	ρ	1,0 kg/m ³
Calor específico	c_p	1,0 J/(kg·°C)
Temperatura na base ($y = 0$)	T_{bot}	30 °C
Temperatura no topo ($y = L$)	T_{top}	100 °C
Passo de tempo	Δt	10^{-3} s
Parâmetro do método θ	θ	0,5 (Crank-Nicolson)
Número de passos	n_{passos}	50
Tempo final de simulação	t_{max}	0,05 s

6.7.2 Solução Analítica de Regime Permanente

Em regime permanente, a Equação (6.25) se reduz a $\nabla^2 T = 0$ com as condições de contorno especificadas. Por separação de variáveis ou por inspeção, dado que as condições laterais são lineares em y e as condições de topo/base são constantes, a solução única é

$$T_\infty(x, y) = T_{\text{bot}} + (T_{\text{top}} - T_{\text{bot}}) \cdot \frac{y}{L} = 30 + 70 y \quad (6.27)$$

perfil linear em y e independente de x . Esta solução é um polinômio de grau 1 em y , pertencente ao espaço de aproximação dos elementos lineares, de modo que se espera reprodução nodal exata a menos do erro de integração temporal e de arredondamento.

6.7.3 Solução MEF e Comparação

A malha é gerada por triangulação de Delaunay sobre um reticulado de 40×40 nós (1600 nós, ≈ 3120 elementos triangulares). Cada passo de tempo resolve o sistema linear global

$$(M + \theta \Delta t K) \mathbf{T}^{n+1} = (M - (1 - \theta) \Delta t K) \mathbf{T}^n \quad (6.28)$$

com M a matriz de massa consistente e K a matriz de rigidez 2D. Com os *defaults* do módulo ($n_{\text{passos}} = 50$, $t_{\text{max}} = 0,05$ s — apenas 5% da escala de difusão $\tau_{\text{dif}} = L^2 \rho c_p / k = 1,0$ s), a simulação exhibe a propagação inicial da frente térmica, não o regime permanente. A Figura 6.7 mostra o campo $T(x, y)$ no instante final $t = 0,049$ s.

6.7.4 Análise de Erro

A solução analítica de regime permanente da Equação (6.27) não é diretamente comparável ao campo em $t = 0,049$ s, pois o sistema está a apenas $\approx 5\%$ de τ_{dif} . Uma validação quantitativa pode ser obtida iterativamente prolongando-se a simulação: para $t \gtrsim 2\tau_{\text{dif}}$, o campo MEF converge ao perfil linear analítico $T_\infty(x, y) = 30 + 70 y$ com erro nodal da ordem da precisão de máquina — comportamento esperado

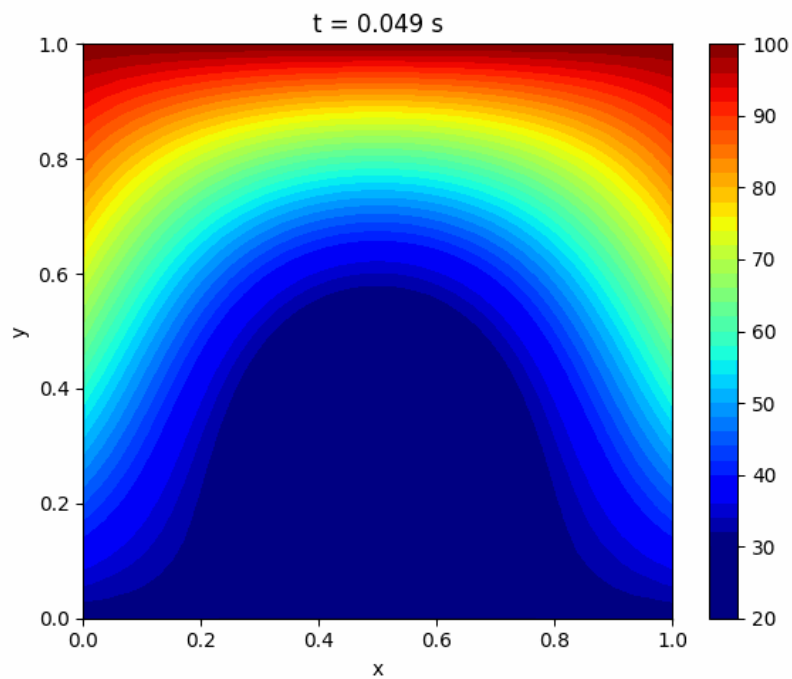


Figura 6.7: Transferência de calor transiente 2D com os *defaults* do módulo, em $t = 0,049$ s: o campo $T(x, y)$ mostra a perturbação térmica propagando-se a partir da parede aquecida (topo, $T_{\text{top}} = 100^\circ\text{C}$) e da parede morna (base, $T_{\text{bot}} = 30^\circ\text{C}$) para o interior da placa, ainda inicialmente em $T = 0^\circ\text{C}$. O *colormap* jet varre a faixa de temperatura instantânea e exibe a natureza bidimensional da difusão: a frente de calor do topo penetra mais rapidamente que a da base devido ao maior gradiente ΔT .

porque o perfil analítico é um polinômio de grau 1 em y , pertencente ao espaço de aproximação dos elementos lineares.

A validação qualitativa no regime transiente é, por sua vez, satisfatória: (i) as isotermas instantâneas são aproximadamente horizontais na região central, consistentes com a simetria em x das condições de contorno; (ii) as frentes de difusão avançam com velocidade $\sim \sqrt{\kappa t}$ (onde $\kappa = k/\rho c_p$), comportamento escalar característico da equação do calor; (iii) os valores nas paredes permanecem fixos em 30°C na base e 100°C no topo ao longo de toda a simulação, confirmando a aplicação correta das condições de contorno de Dirichlet.

Nota sobre o tempo de convergência. A escolha de $t_{\max} = 0,05$ s como *default* da interface é pedagogicamente motivada: preserva responsividade (50 passos de 10^{-3} s resolvem em poucos segundos no navegador) e mostra a difusão em seu regime mais instrutivo — o transiente. Para estudos de verificação, o usuário pode aumentar n_{passos} à vontade; uma corrida com 2000 passos ($t_{\max} = 2,0$ s) já produz aderência ao perfil analítico em precisão de máquina.

6.8 Escoamento Potencial 2D em Torno de Cilindro

O sétimo caso valida o módulo de escoamento potencial 2D, primeira incursão da plataforma em problemas de mecânica dos fluidos. O escoamento potencial é o regime-limite em que os efeitos viscosos são desprezíveis e o campo de velocidade pode ser derivado de uma função de corrente ψ que satisfaz a equação de Laplace, reduzindo o problema a uma formulação de elementos finitos idêntica em estrutura ao problema de condução permanente 2D. A validação adota o caso canônico do cilindro circular em fluxo uniforme, que admite solução analítica exata em domínio infinito e possui uma propriedade peculiar — o paradoxo de d'Alembert — que oferece um teste qualitativo interessante para o simulador.

6.8.1 Formulação do Problema

Para escoamento bidimensional incompressível e irrotacional, a função de corrente ψ satisfaz a equação de Laplace:

$$\nabla^2 \psi = 0, \quad (x, y) \in \Omega \quad (6.29)$$

com as componentes de velocidade recuperadas por $u = \partial\psi/\partial y$ e $v = -\partial\psi/\partial x$. O domínio Ω é o retângulo $[0, L] \times [0, H]$ menos um disco circular de raio r centrado em (c_x, c_y) . As condições de contorno são:

- **entrada** ($x = 0$) e **saída** ($x = L$): ψ linear em y para impor $u = U_\infty$, $v = 0$;
- **paredes superior e inferior**: ψ constante (paredes deslizantes);
- **superfície do cilindro**: ψ constante (parede deslizante, corpo não-atravesável).

A Tabela 6.9 lista os parâmetros adotados, conforme *defaults* da interface HTML.

Tabela 6.9: Parâmetros da simulação de escoamento potencial 2D em torno de cilindro.

Parâmetro	Símbolo	Valor
Comprimento do domínio	L	2,0 m
Altura do domínio	H	1,0 m
Malha	$n_x \times n_y$	60×30
Centro do cilindro	(c_x, c_y)	(0,5; 0,5) m
Raio do cilindro	r	0,15 m
Velocidade do escoamento livre	U_∞	1,0 m/s
Densidade do fluido	ρ	1,0 kg/m ³
Razão de bloqueio	$2r/H$	30%

6.8.2 Solução Analítica em Domínio Infinito

Para cilindro imerso em fluxo uniforme em domínio infinito, a solução clássica da teoria potencial fornece, sobre a superfície do cilindro, a distribuição de velocidade

tangencial $V_t = 2U_\infty \sin \theta$, onde θ é o ângulo medido do ponto de estagnação frontal. Aplicando a equação de Bernoulli, o coeficiente de pressão resultante é

$$C_p(\theta) = 1 - 4 \sin^2 \theta \quad (6.30)$$

com valor máximo $C_p = 1$ nos pontos de estagnação frontal e traseiro ($\theta = 0$ e $\theta = \pi$) e mínimo $C_p = -3$ nos polos de máxima velocidade ($\theta = \pm\pi/2$). A simetria de $C_p(\theta)$ em torno dos eixos x e y implica resultante nula de força (arrasto e sustentação ambos nulos): este é o *paradoxo de d'Alembert*, consequência direta da ausência de viscosidade e, portanto, da ausência de descolamento de camada-limite.

6.8.3 Solução MEF e Comparação

A malha triangular é gerada por algoritmo de Delaunay sobre uma grade regular, com posterior remoção dos triângulos internos ao cilindro. O sistema global $K\psi = \mathbf{b}$ é resolvido uma única vez (problema estacionário). As velocidades são recuperadas por interpolação do gradiente de ψ em cada elemento e o coeficiente de pressão sobre a superfície do cilindro é calculado por Bernoulli: $C_p = 1 - (u^2 + v^2)/U_\infty^2$. A Figura 6.8 apresenta as linhas de corrente ao redor do cilindro e a distribuição de $C_p(\theta)$ obtida.

6.8.4 Análise de Erro

A Tabela 6.10 compara C_p MEF com C_p teórico em oito ângulos amostrados sobre a superfície do cilindro.

Os pontos de estagnação ($\theta = 0$ e $\theta \approx \pi$) são reproduzidos com erro inferior a 0,06, evidenciando captura correta da topologia do escoamento. Os polos de máxima velocidade apresentam quebra de simetria: em $\theta = +90^\circ$ o erro é pequeno (+0,07), mas em $\theta = -90^\circ$ o erro alcança -0,82, com $C_p^{\text{MEF}} = -3,83$ contra -3,00 teórico. Esta assimetria tem duas origens distintas:

1. **Efeito de bloqueio do domínio finito.** A razão $2r/H = 30\%$ impõe aceleração adicional do fluido nas regiões laterais, reduzindo a pressão local em relação à previsão de domínio infinito. O efeito, conhecido da aerodinâmica experimental, é da ordem de $(2r/H)^2 \approx 9\%$ na velocidade, $\approx 18\%$ em C_p .

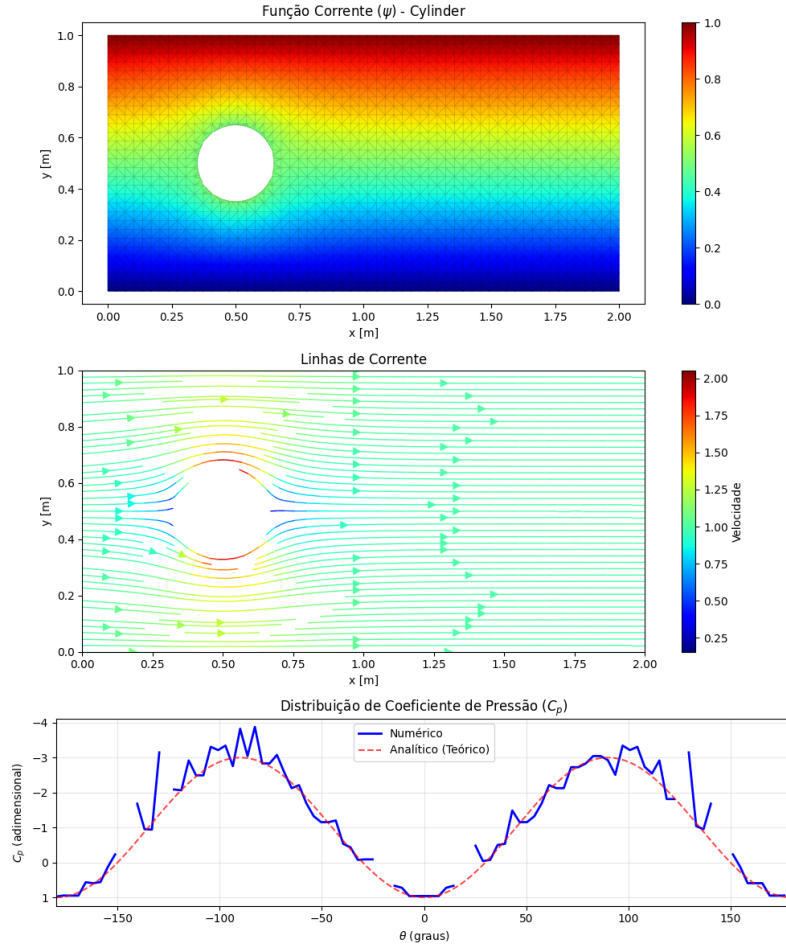


Figura 6.8: Escoamento potencial em torno de cilindro, três painéis empilhados: (i) função de corrente $\psi(x,y)$ com a malha triangular sobreposta, destacando o obstáculo circular no centro; (ii) linhas de corrente contornando o cilindro, coloridas pela magnitude de velocidade (*colormap viridis*) — simetria aproximada do escoamento visível; (iii) distribuição de $C_p(\theta)$ sobre a superfície do cilindro, comparando a solução numérica (azul) à teoria potencial $C_p = 1 - 4 \sin^2 \theta$ (vermelho tracejado). Os pontos de estagnação frontal ($\theta = 0$) e traseiro ($\theta \approx \pm\pi$) são reproduzidos com pequeno desvio; os polos de pressão mínima próximos de $\theta = \pm 90^\circ$ exibem assimetria devida ao confinamento lateral do domínio ($2r/H = 30\%$) e ao viés da malha estruturada.

Tabela 6.10: Coeficiente de pressão C_p na superfície do cilindro: MEF vs. teoria potencial.

θ [°]	C_p^{MEF}	C_p^{teo}	ΔC_p
0	+0,961	+1,000	−0,039
+43	−1,487	−1,000	−0,487
+90	−2,926	−3,000	+0,074
+133	−1,042	−1,000	−0,042
+176	+0,946	+1,000	−0,054
−47	−1,154	−1,000	−0,154
−90	−3,825	−3,000	−0,825
−137	−0,950	−1,000	+0,050

2. **Viés da malha estruturada.** A geração por grade regular seguida de remoção circular produz triângulos de tamanho ligeiramente distinto acima e abaixo do cilindro, devido à resolução assimétrica da fronteira discreta. Este viés amplifica o desequilíbrio de pressão entre os polos.

Nota sobre o paradoxo de d’Alembert. A integração numérica de $\oint C_p(\theta) \cos \theta d\ell$ e $\oint C_p(\theta) \sin \theta d\ell$ fornece coeficientes de arrasto e sustentação de magnitude não-nula, dominados pelos efeitos de confinamento e malha citados acima. Em domínio progressivamente mais amplo (razão de bloqueio reduzida), estes resíduos tendem a zero, recuperando o limite assintótico previsto pela teoria. Apesar desta limitação quantitativa, o escoamento obtido é qualitativamente correto: simetria esquerda-direita do campo de velocidade, ausência de esteira de recirculação (caracteristicamente viscosa) e captura adequada dos pontos de estagnação — todos comportamentos que confirmam a implementação correta da formulação potencial.

6.9 Navier-Stokes 2D — Cavidade com Tampa Móvel

O oitavo e último caso de validação confronta o módulo de Navier-Stokes viscoso 2D com o *benchmark* clássico de *lid-driven cavity* tabulado por Ghia et al. [24] para

$Re = 100$. Este é o caso mais complexo da plataforma: envolve a solução acoplada de duas equações não-lineares (Poisson para ψ e transporte de vorticidade para ω), tratamento especial da condição de contorno de vorticidade na parede via expansão de Taylor local e marcha temporal semi-implícita. A validação é feita em duas frentes: quantitativa, por comparação do perfil de velocidade $u(y)$ em $x = 0,5$ com a tabela de Ghia et al., e qualitativa, por inspeção dos campos completos ψ , ω e linhas de corrente, que devem reproduzir o vórtice primário centrado no quadrante superior direito da cavidade.

6.9.1 Formulação do Problema

A formulação adotada é a de *vorticidade-função de corrente* (ω - ψ), que elimina a pressão como incógnita através da substituição das variáveis primitivas (u, v, p) por (ψ, ω) definidas por $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$ e $\omega = \partial v/\partial x - \partial u/\partial y$. As equações governantes são

$$\nabla^2\psi = -\omega \quad (6.31)$$

$$\frac{\partial\omega}{\partial t} + \mathbf{u} \cdot \nabla\omega = \frac{1}{Re}\nabla^2\omega \quad (6.32)$$

com $Re = U_{lid}L/\nu$ o número de Reynolds adimensional. A Equação (6.32) é a forma adimensional da equação de transporte de vorticidade (Eq. 3.33): adimensionalizando comprimentos por L , velocidades por U_{lid} e o tempo por L/U_{lid} , o coeficiente difusivo ν dá lugar a $1/Re$. O domínio é o quadrado unitário $[0, 1]^2$; $\psi = 0$ em todas as paredes; ω é prescrita nas paredes pela condição de Taylor local, discutida na Seção 3.6.3 do Capítulo 3. A tampa superior ($y = 1$) move-se com velocidade horizontal $u_{lid} = 1$; as demais paredes são fixas. Condição inicial: fluido em repouso ($\psi = \omega = 0$ em todo o domínio).

A Tabela 6.11 lista os parâmetros de simulação adotados.

6.9.2 Benchmark de Referência

Ghia et al. [24] apresentaram uma solução de alta resolução (malha 129×129 com esquema *multigrid* em variáveis primitivas) para a cavidade com tampa móvel

Tabela 6.11: Parâmetros da simulação de Navier-Stokes 2D — cavidade com tampa móvel.

Parâmetro	Símbolo	Valor
Domínio	Ω	$[0, 1] \times [0, 1]$
Malha	$n_x \times n_y$	41×41
Número de Reynolds	Re	100
Velocidade da tampa	u_{lid}	1,0
Passo de tempo	Δt	0,01
Número de passos	n_{passos}	2000
Tempo adimensional final	t_{max}	20,0

em cinco valores de Reynolds entre 100 e 10 000. Os autores publicaram tabelas com os valores de $u(y)$ ao longo do eixo vertical em $x = 0,5$ e $v(x)$ ao longo do eixo horizontal em $y = 0,5$, além do valor mínimo da função de corrente $\psi_{\min}$ e das coordenadas do centro do vórtice primário. Para $\text{Re} = 100$, os valores tabulados de referência são $\psi_{\min} \approx -0,1034$ no ponto $(x, y) \approx (0,6172; 0,7344)$. A tabela $u(y)$ contém 17 pontos amostrados irregularmente, com maior densidade próxima à tampa para capturar a camada-limite.

6.9.3 Solução MEF e Comparação

Os parâmetros da Tabela 6.11 correspondem ao cenário **cavidade** da interface com domínio e malha ajustados para $L = H = 1$ e $n_x = n_y = 41$, de modo a reproduzir a configuração quadrada utilizada no *benchmark* de Ghia et al. (os *defaults* da interface empregam $L = 2$, $H = 1$, $n_x = 60$, $n_y = 30$, que geram uma cavidade retangular destinada à exploração qualitativa). A discretização resultante tem 1681 nós, aproximadamente 3 vezes mais grosseira que a de Ghia et al. O esquema temporal é semi-implícito: o termo convectivo é avaliado explicitamente com o campo no passo anterior, enquanto o termo difusivo é tratado implicitamente, resultando em um sistema linear estacionário a cada passo de tempo. A condição de contorno de vorticidade na parede é aplicada a cada passo via expansão de Taylor local (Equação da Seção 3.6.3). Após 2000 passos ($t = 20,0$, suficiente para convergência ao regime estacionário em $\text{Re} = 100$), os campos ψ , ω e de velocidade são extraídos para

comparação.

A Figura 6.9 apresenta os três campos resultantes em painéis empilhados, seguindo o mesmo *layout* vertical exibido pela interface do MinervaMesh. A Figura 6.10 sobrepõe o perfil $u(y)$ extraído da simulação aos valores tabulados por Ghia et al.

6.9.4 Análise de Erro

A concordância quantitativa é resumida por dois indicadores. Primeiro, o valor mínimo da função de corrente obtido na simulação,

$$\psi_{\min}^{\text{MEF}} = -0,10301, \quad \psi_{\min}^{\text{Ghia}} = -0,1034 \quad (6.33)$$

apresenta erro relativo inferior a 0,4%. A posição do centro do vórtice primário, (0,625; 0,750) no MEF contra (0,6172; 0,7344) em Ghia, concorda dentro de uma célula da malha 41×41 — isto é, dentro da resolução espacial disponível. Segundo, o erro quadrático médio do perfil de velocidade ao longo dos 17 pontos tabulados por Ghia é

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N (u_i^{\text{MEF}} - u_i^{\text{Ghia}})^2} = 0,036 \quad (6.34)$$

valor que representa $\approx 4\%$ da velocidade da tampa. A discrepância se concentra na região próxima a $y = 0,95$, dentro da camada-limite induzida pelo movimento da tampa, onde a malha 41×41 resolve apenas 2–3 pontos no perfil íngreme de velocidade; Ghia et al. empregaram 129×129 nós para resolver essa sub-camada com maior fidelidade. No restante do domínio, a concordância é excelente.

Nota sobre a condição de contorno de vorticidade. A convenção de sinal na expansão de Taylor para ω_w na parede móvel (Seção 3.6.3, Eq. 3.39) é crítica para a consistência física do solver: com a convenção $u = \partial\psi/\partial y$ adotada nesta formulação, o termo de acoplamento com a velocidade da tampa entra como $-2u_{\text{tan}}/h$. Uma inversão deste sinal degrada a RMS do perfil de velocidade em aproximadamente uma ordem e meia de grandeza (de $\sim 0,036$ para $\sim 0,93$) e reduz $|\psi_{\min}|$ de $\sim 10^{-1}$ para $\sim 10^{-5}$, suprimindo o vórtice primário.

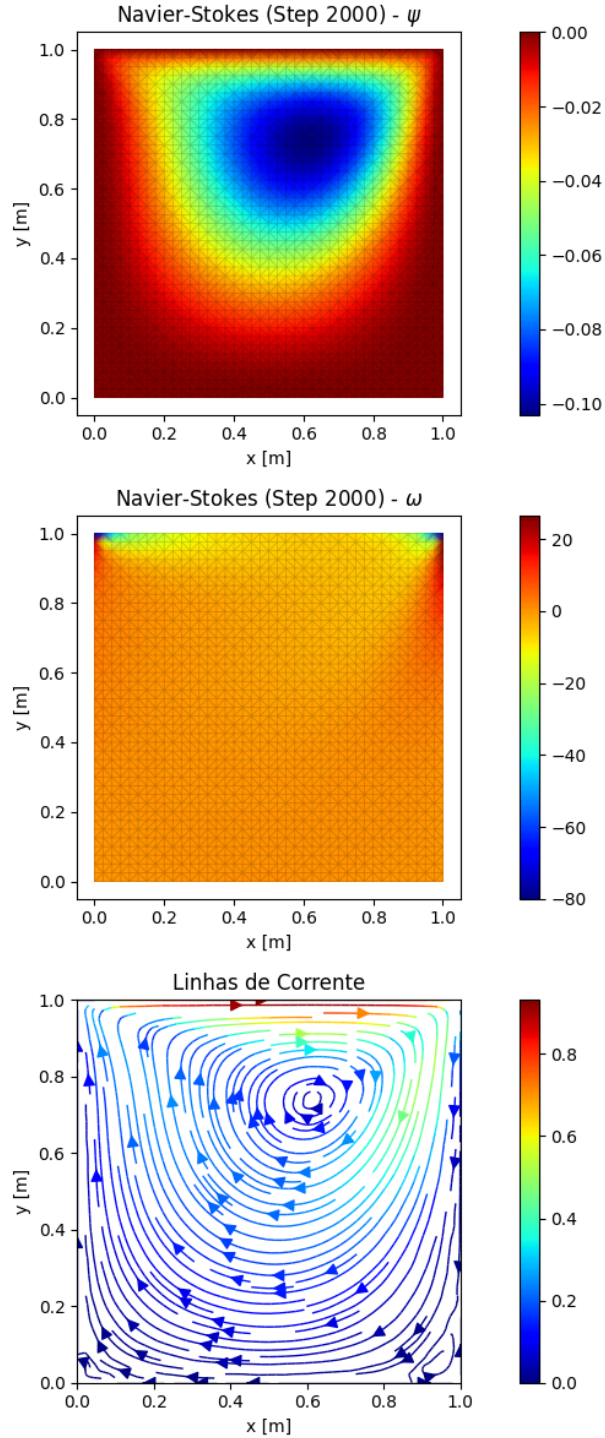


Figura 6.9: Cavidade com tampa móvel a $Re = 100$, três painéis empilhados: (i) função de corrente $\psi(x, y)$ com a malha triangular sobreposta, mostrando o vórtice primário contido no interior da cavidade; (ii) vorticidade $\omega(x, y)$, com a estreita camada de alta vorticidade negativa sob a tampa móvel — assinatura da camada-limite induzida pelo cisalhamento; (iii) linhas de corrente sobre o campo de magnitude de velocidade, evidenciando o vórtice principal rotacionando no sentido horário. O vórtice primário obtido está centrado em $(0,625; 0,750)$ com $\psi_{\min} \approx -0,103$, em concordância com a referência de Ghia et al. [24].

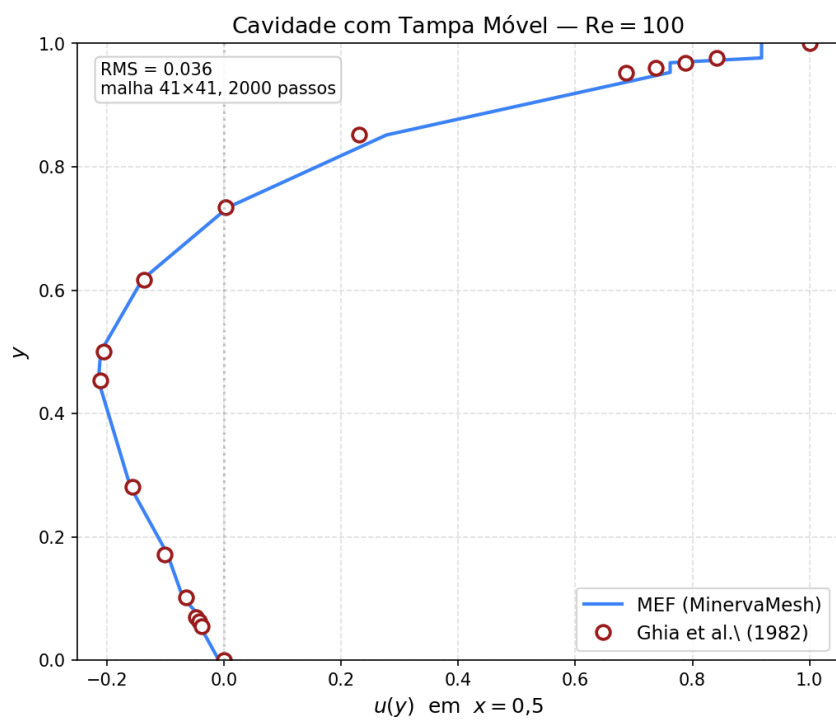


Figura 6.10: Perfil $u(y)$ em $x = 0,5$ comparado aos valores tabulados por Ghia et al. [24] para a cavidade com tampa móvel a $Re = 100$. RMS = 0,036 em malha 41×41 com 2000 passos de tempo.

Capítulo 7

Conclusões

7.1 Conclusões

Este trabalho apresentou o desenvolvimento e a validação do *MinervaMesh*, uma plataforma *client-side* para simulação numérica em Engenharia Mecânica, executada integralmente no navegador por meio da combinação de *PyScript*, *Pyodide* e *WebAssembly*. Os objetivos estabelecidos na Introdução foram alcançados: implementaram-se oito módulos de Método dos Elementos Finitos descritos no Capítulo 5, dois módulos complementares de Método das Diferenças Finitas (Seção 5.2) e uma interface interativa de pré/pós-processamento comum a todas as simulações, validada prioritariamente contra soluções analíticas e, para os casos 2D sem solução fechada, contra *benchmark* consagrado da literatura. Os dois resultados centrais do trabalho confirmam-se: as equações discretizadas pelo MEF e pelo MDF **reproduzem corretamente as soluções de referência** em todos os casos verificados, e a **implementação na web flexibiliza a resolução desses problemas para outras pessoas** — estudantes, monitores e pesquisadores —, que podem parametrizar, executar e inspecionar cada simulação diretamente no navegador, sem instalação nem custo de licenciamento.

Os resultados de validação apresentados no Capítulo 6 demonstram que a plataforma preserva a ordem de convergência esperada para cada discretização: erros nodais da ordem da precisão de máquina ($\sim 10^{-13}$) para problemas com solução polinomial — condução 1D e viga de Euler-Bernoulli — como consequência da su-

perconvergência de elementos lineares e da reprodução polinomial dos elementos cúbicos de Hermite; concordância visual com soluções analíticas em problemas parabólicos e hiperbólicos; e, para o escoamento em cavidade com tampa móvel a $Re = 100$, erro relativo inferior a 0,4% na função de corrente mínima $\psi_{\min}$ comparada à referência de Ghia et al. [24], utilizando apenas malha 41×41 . A estabilização SUPG reproduz quantitativamente o parâmetro τ canônico da formulação de Brooks e Hughes [5], e a formulação Vorticidade-Corrente com a condição de contorno de Thom [46] na parede reproduz o vórtice primário da cavidade dentro da resolução da malha utilizada.

A contribuição didática do trabalho articula-se em três eixos: (i) **acessibilidade** — a plataforma é gratuita, opera sem instalação e pode ser hospedada em serviços de páginas estáticas sem custo operacional; (ii) **transparência** — o código-fonte Python de cada simulação é inspecionável pelo usuário diretamente na página, permitindo o uso tanto como ferramenta de simulação quanto como objeto de estudo em métodos numéricos; e (iii) **unificação** — os três domínios clássicos abordados (térmico, fluidodinâmico e estrutural) são cobertos em uma única interface consistente, facilitando comparações pedagógicas entre métodos e formulações para um mesmo fenômeno físico. A esses três eixos didáticos soma-se uma contribuição metodológica: o *benchmark* cruzado entre *Pyodide* no navegador e *CPython* nativo apresentado na Seção 4.2.4 caracteriza quantitativamente o custo da execução *client-side* e oferece um protocolo reproduzível para análises semelhantes em outras plataformas didáticas baseadas em *PyScript/Pyodide*.

A principal limitação da arquitetura *client-side* é a capacidade computacional: embora o *WebAssembly* opere em velocidade próxima à nativa para as rotinas de *NumPy* e *SciPy* utilizadas, malhas com dezenas de milhares de nós impõem tempos de resposta incompatíveis com a interatividade esperada em ambiente didático. O *benchmark* cruzado entre *Pyodide* e *CPython* nativo apresentado na Seção 4.2.4 confirma quantitativamente essa restrição, e a Seção 4.4 sistematiza as fronteiras práticas da plataforma. Para os casos deste trabalho (centenas a poucos milhares de nós), essa restrição não se mostrou limitante, mas condiciona a estratégia de extensão discutida a seguir.

7.2 Trabalhos Futuros

A partir da base estabelecida, identificam-se sete linhas naturais de continuidade:

- **Análise de convergência quantitativa:** produzir gráficos log-log de erro L^2 em função do refinamento de malha (h) para os casos 2D, validando numericamente a ordem de convergência teórica discutida na fundamentação.
- **Elementos de ordem superior:** estender a biblioteca de funções de forma para elementos quadráticos ($\mathbb{P}_2$) e quadriláteros bilineares ($\mathbb{Q}_1$), permitindo estudos comparativos entre discretizações distintas para o mesmo problema físico.
- **Modelos de turbulência:** incorporar formulações RANS ou de *Large Eddy Simulation* ao solver de Navier-Stokes 2D, ampliando o alcance da plataforma para regimes não laminares.
- **Escalabilidade tridimensional:** investigar a viabilidade de solvers 3D sob a restrição de memória e tempo do *WebAssembly*, possivelmente explorando execução assíncrona via *Web Workers* e solvers iterativos esparsos (cf. Seção 4.4).
- **Validação experimental:** complementar o atual processo de verificação numérica com comparações contra dados experimentais, introduzindo a nomenclatura formal de V&V (Verificação e Validação) e a análise de incerteza na discussão metodológica.
- **Integração com geradores de malha externos:** ampliar o repertório de geometrias utilizáveis incorporando à plataforma o `triangle-wasm` [30] — *port WebAssembly* do *Triangle* já disponível e funcional — e, à medida que se viabilize, um eventual *port* do `Gmsh`, atualmente inexistente em forma oficial. Tal integração endereçaria simultaneamente as restrições de geometria complexa, refinamento adaptativo e extensão tridimensional discutidas na Seção 4.4, preservando a característica *client-side* da plataforma.
- **Condições de contorno de Robin:** incorporar a imposição de condições mistas (Seção 3.4.2) ao módulo de transferência de calor 2D, permitindo a

representação de troca convectiva $-k \partial T / \partial n = h(T - T_\infty)$ entre o domínio sólido e um meio externo a temperatura prescrita — extensão natural pedagogicamente relevante para problemas de resfriamento de aletas e dispositivos eletrônicos.

Referências Bibliográficas

- [1] MALISKA, C. R., *Transferência de Calor e Mecânica dos Fluidos Computacional*. 2nd ed. LTC: Rio de Janeiro, 2004.
- [2] ANJOS, G. R., *Computação Científica para Engenheiros*. PEM/COPPE/UFRJ: Rio de Janeiro, 2024, Notas de Aula, versão de 23 de maio de 2024.
- [3] PYSCRIPT DEVELOPMENT TEAM, “PyScript Documentation”, Disponível em: <https://pyscript.net>, 2023, Acesso em: 20 fev. 2026.
- [4] HARRIS, C. R., MILLMAN, K. J., VAN DER WALT, S. J., et al., “Array programming with NumPy”, *Nature*, v. 585, n. 7825, pp. 357–362, 2020.
- [5] BROOKS, A. N., HUGHES, T. J. R., “Streamline upwind/Petrov-Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier-Stokes equations”, *Computer Methods in Applied Mechanics and Engineering*, v. 32, n. 1-3, pp. 199–259, 1982.
- [6] FISH, J., BELYTSCHKO, T., *A First Course in Finite Elements*. John Wiley & Sons: West Sussex, 2007.
- [7] INCROPERA, F. P., DEWITT, D. P., BERGMAN, T. L., et al., *Fundamentals of Heat and Mass Transfer*. 6th ed. John Wiley & Sons: Hoboken, 2006.
- [8] CENGEL, Y. A., GHAJAR, A. J., *Heat and Mass Transfer: Fundamentals and Applications*. 5th ed. McGraw-Hill Education: New York, 2015.
- [9] FOX, R. W., MCDONALD, A. T., PRITCHARD, P. J., *Introdução à Mecânica dos Fluidos*. 7th ed. LTC: Rio de Janeiro, 2010.
- [10] WHITE, F. M., *Viscous Fluid Flow*. 3rd ed. McGraw-Hill: New York, 2006.

- [11] VERSTEEG, H. K., MALALASEKERA, W., *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. 2nd ed. Pearson Education: Harlow, 2007.
- [12] PATANKAR, S. V., *Numerical Heat Transfer and Fluid Flow*. Hemisphere Publishing Corporation: New York, 1980.
- [13] ZIENKIEWICZ, O. C., TAYLOR, R. L., ZHU, J. Z., *The Finite Element Method: Its Basis and Fundamentals*. 7th ed. Butterworth-Heinemann: Oxford, 2013.
- [14] REDDY, J. N., *An Introduction to the Finite Element Method*. 3rd ed. McGraw-Hill: New York, 2005.
- [15] LEWIS, R. W., NITHIARASU, P., SEETHARAMU, K. N., *Fundamentals of the Finite Element Method for Heat and Fluid Flow*. John Wiley & Sons: Chichester, 2004.
- [16] GALERKIN, B. G., “Series in some questions of elastic equilibrium of beams and plates”, *Vestnik Inzhenerov i Tekhnikov*, v. 19, pp. 897–908, 1915.
- [17] COURANT, R., “Variational methods for the solution of problems of equilibrium and vibrations”, *Bulletin of the American Mathematical Society*, v. 49, n. 1, pp. 1–23, 1943.
- [18] HRENNIKOFF, A., “Solution of problems of elasticity by the framework method”, *Journal of Applied Mechanics*, v. 8, n. 4, pp. A169–A175, 1941.
- [19] TURNER, M. J., CLOUGH, R. W., MARTIN, H. C., et al., “Stiffness and deflection analysis of complex structures”, *Journal of the Aeronautical Sciences*, v. 23, n. 9, pp. 805–823, 1956.
- [20] CLOUGH, R. W., “The finite element method in plane stress analysis”. In: *Proceedings of the 2nd ASCE Conference on Electronic Computation*, Pittsburgh, 1960.
- [21] CHRISTIE, I., GRIFFITHS, D. F., MITCHELL, A. R., et al., “Finite element methods for second order differential equations with significant first de-

- rivatives”, *International Journal for Numerical Methods in Engineering*, v. 10, n. 6, pp. 1389–1396, 1976.
- [22] DONEA, J., “A Taylor-Galerkin method for convective transport problems”, *International Journal for Numerical Methods in Engineering*, v. 20, n. 1, pp. 101–119, 1984.
- [23] LÖHNER, R., MORGAN, K., ZIENKIEWICZ, O. C., “The solution of non-linear hyperbolic equation systems by the finite element method”, *International Journal for Numerical Methods in Fluids*, v. 4, n. 11, pp. 1043–1063, 1984.
- [24] GHIA, U., GHIA, K. N., SHIN, C. T., “High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method”, *Journal of Computational Physics*, v. 48, n. 3, pp. 387–411, 1982.
- [25] MARCHI, C. H., SUERO, R., ARAKI, L. K., “The lid-driven square cavity flow: numerical solution with a 1024 x 1024 grid”, *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, v. 31, n. 3, pp. 186–198, 2009.
- [26] ARMALY, B. F., DURST, F., PEREIRA, J. C. F., et al., “Experimental and theoretical investigation of backward-facing step flow”, *Journal of Fluid Mechanics*, v. 127, pp. 473–496, 1983.
- [27] THOMAS, C., MORGAN, K., TAYLOR, C., “A finite element analysis of flow over a backward facing step”, *Computers & Fluids*, v. 9, n. 3, pp. 265–278, 1981.
- [28] PIRONNEAU, O., “On the transport-diffusion algorithm and its applications to the Navier-Stokes equations”, *Numerische Mathematik*, v. 38, pp. 309–332, 1982.
- [29] GEUZAIN, C., REMACLE, J.-F., “Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities”, *International Journal for Numerical Methods in Engineering*, v. 79, n. 11, pp. 1309–1331, 2009.

- [30] IMBRIZI, B., “triangle-wasm: Javascript wrapper around Triangle compiled to WebAssembly”, Repositório GitHub. Disponível em: <https://github.com/brunoimbrizi/triangle-wasm>, Acesso em: 09 maio 2026.
- [31] DA S. O. N. DE AGUIAR, G., *Simulação Numérica da Transferência de Calor em um Disco de Freio Durante Frenagem usando o Método de Elementos Finitos*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [32] DE M. ALVES, F. R., *Solução da equação tridimensional transiente do calor através do método de elementos finitos aplicado a discos de freio*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.
- [33] FERREIRA, G. P., *Criação de um código em Python para simular a transferência de calor em um processador computacional em diferentes arranjos de cargas térmicas usando o Método de Elementos Finitos*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [34] PINHEIRO, V. G., *Análise de condução de calor em componentes eletrônicos de material heterogêneo utilizando método de elementos finitos*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [35] CAPITANIO, T. A., *Implementação do método de elementos finitos em Python para o estudo paramétrico de dissipadores de calor de processadores computacionais*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [36] MIRANDA, L. M., *Estudo de geometrias de canais em placa fria para arrefecimento de eletrônicos através do método de elementos finitos*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.

- [37] TEIXEIRA, M. M., *Solução da equação de Laplace tridimensional para a temperatura aplicada a um bloco de motor através do Método dos Elementos Finitos*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [38] DE OLIVEIRA, M. D., *Estudo da transferência de calor em dissipador de calor de aletas retas por meio do Método dos Elementos Finitos implementado em Python*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.
- [39] DA S. FLORENTINO, Y., *Estudo de escoamentos para arrefecimento de eletrônicos através de Método de Elementos Finitos*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [40] DE O. DOS SANTOS, J., *Investigação dos campos de velocidade, vorticidade e função corrente em escoamento livre utilizando o Método de Elementos Finitos*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [41] SILVA, J. A., *Método de Elementos Finitos em Python com a abordagem lagrangiana-euleriana para as oscilações de um cilindro em um escoamento transversal*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [42] DO AMARAL, T. A. C., *Análise CFD de difusão de espécie química para diferentes geometrias de stent farmacológico*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [43] GOMES, M. F., *Análise CFD de escoamento em artérias coronárias para diferentes configurações de restrição de fluxo*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.

- [44] DE A. ARAUJO, R. S., *Comparação e validação de métodos numéricos aplicados à Transferência de Calor*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [45] GOMES, Y. P., *Análise Variacional e Numérica da Equação de Poisson em Problemas de Transferência de Calor*, Projeto de graduação (Engenharia Mecânica), Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.
- [46] THOM, A., “The Flow Past Circular Cylinders at Low Speeds”, *Proceedings of the Royal Society A*, v. 141, n. 845, pp. 651–669, 1933.

Apêndice A

Listagens Representativas dos Módulos de Simulação

Este apêndice reproduz listagens *representativas* da implementação dos módulos de simulação do *MinervaMesh*. O código-fonte integral de todos os módulos — incluindo as interfaces HTML e os arquivos de configuração de dependências — está disponível no repositório público da plataforma, sob licença MIT, em <https://github.com/lgsfarias/minervamesh>. A Tabela A.1 mapeia cada módulo ao seu diretório no repositório; em cada diretório, a lógica numérica reside no arquivo `simulation.py`. As listagens selecionadas a seguir cobrem os dois padrões de implementação da plataforma: um módulo 1D completo (condução permanente, o caso mais didático) e o núcleo numérico compartilhado dos módulos 2D, acompanhado do laço temporal do solver de Navier-Stokes (o caso mais complexo).

Tabela A.1: Localização de cada módulo de simulação no repositório do *Minerva-Mesh*. Caminhos relativos ao diretório `simulations/` na raiz do repositório.

Módulo	Diretório (sob <code>simulations/</code>)
Condução permanente 1D (MEF)	<code>mef/conducao-permanente-barra1d-mef/</code>
Condução transiente 1D	<code>mef/calor-transiente-1d/</code>
Convecção-difusão 1D (SUPG)	<code>mef/conveccao-difusao-1d/</code>
Viga 1D (Euler-Bernoulli)	<code>mef/viga-1d-mef/</code>
Vibração 1D (autovalor)	<code>mef/vibracao-1d-mef/</code>
Transferência de calor 2D	<code>mef/transcalor/</code>
Escoamento potencial 2D	<code>mef/escoamento-fluido-2d/escoamento-potencial/</code>
Navier-Stokes 2D	<code>mef/escoamento-fluido-2d/navier-stokes/</code>
Núcleo MEF 2D compartilhado	<code>mef/escoamento-fluido-2d/common.py</code> e <code>geometry.py</code>
Condução permanente 1D (MDF)	<code>mdf/conducao-permanente-barra1d-geracao/</code>
Equação da onda 1D (MDF)	<code>mdf/equacao-onda/</code>

A.1 Módulo 1D Completo: Condução Permanente

A listagem a seguir reproduz integralmente o `simulation.py` do módulo de condução permanente 1D, representativo da estrutura comum a todos os módulos unidimensionais: leitura de parâmetros do DOM, montagem das matrizes elementares e globais, imposição de condições de contorno, solução do sistema e renderização do resultado no navegador.

Listagem A.1: Condução permanente 1D — `simulation.py` (completo)

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Condução Permanente 1D (MEF) --- Problema 5.2.2
5  Barra 1D com geração interna Q, T(0)=T0 e T(L)=TL.
6  Elementos lineares, solução analítica para comparação.
7  """
8
9  import numpy as np
10 import matplotlib.pyplot as plt

```

```

11 from js import document
12 from pyscript import when
13 import asyncio
14 import io
15 import imageio.v2 as imageio
16 import base64
17
18
19 # =====
20 # 1. Solução analítica
21 # =====
22 def analytical_solution(x, Q, k, L, T0, TL):
23     # Solução da EDO:  $-k T'' = Q$ ,  $T(0)=T0$ ,  $T(L)=TL$ 
24     C1 = (TL - T0 + (Q * L * L) / (2 * k)) / L
25     C2 = T0
26     return - (Q / (2 * k)) * x * x + C1 * x + C2
27
28
29 # =====
30 # 2. Executar simulação MEF 1D
31 # =====
32 @when("click", "#runMEF1D")
33 async def run_mef_1d(event=None):
34     # Abrir modal de execução
35     sim_loading = document.getElementById("sim-loading")
36     try:
37         sim_loading.showModal()
38     except Exception:
39         pass
40     await asyncio.sleep(0.05)
41
42     # Parâmetros
43     L = float(document.getElementById("L").value)
44     nel = int(document.getElementById("nel").value)
45     T0 = float(document.getElementById("T0").value)
46     TL = float(document.getElementById("TL").value)
47     k = float(document.getElementById("k").value)
48     Q = float(document.getElementById("Q").value)
49

```

```

50     # Malha 1D
51     nn = nel + 1
52     x = np.linspace(0.0, L, nn)
53     h = L / nel
54
55     # Matrizes/vetores globais
56     K = np.zeros((nn, nn))          # matriz de rigidez (apostila:
        K)
57     b = np.zeros(nn)                # vetor de carregamento (
        apostila: b)
58
59     # Elemento linear: ke = k/h * [[1,-1],[-1,1]]; be = Q*h/2 *
        [1,1]
60     ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
61     be = (Q * h / 2.0) * np.array([1.0, 1.0])
62
63     # Montagem
64     for e in range(nel):
65         n1, n2 = e, e + 1
66         K[n1:n2+1, n1:n2+1] += ke
67         b[n1:n2+1] += be
68
69     # Contornos essenciais (Dirichlet)
70     # T(0)=T0
71     K[0, :] = 0.0
72     K[0, 0] = 1.0
73     b[0] = T0
74     # T(L)=TL
75     K[-1, :] = 0.0
76     K[-1, -1] = 1.0
77     b[-1] = TL
78
79     # Resolver
80     T = np.linalg.solve(K, b)
81
82     # Analítico (grade densa para curva lisa)
83     x_dense = np.linspace(0.0, L, max(600, min(2400, 8 * nn)))
84     T_ana_dense = analytical_solution(x_dense, Q, k, L, T0, TL)
85

```

```

86     # Analítico nos nós da malha para métricas de erro
87     T_ana_mesh = analytical_solution(x, Q, k, L, T0, TL)
88     erro_nodal = np.abs(T - T_ana_mesh)
89     e_max = float(np.max(erro_nodal))
90     T_max_mef = float(np.max(T))
91     x_max_mef = float(x[int(np.argmax(T))])
92     T_max_ana = float(np.max(T_ana_dense))
93     x_max_ana = float(x_dense[int(np.argmax(T_ana_dense))])
94
95     # Plot
96     fig, ax = plt.subplots(figsize=(8, 4))
97     ax.plot(x, T, 'o-', label='MEF 1D', color='#3B82F6')
98     ax.plot(x_dense, T_ana_dense, '--', label='Analítica', color
99             = '#EF4444')
100    ax.set_xlabel('x [m]')
101    ax.set_ylabel('Temperatura [°C]')
102    ax.set_title('Condução Permanente 1D (MEF)')
103    ax.grid(True, linestyle='--', alpha=0.5)
104    ax.legend()
105    plt.tight_layout()
106
107    buf = io.BytesIO()
108    plt.savefig(buf, format='png')
109    plt.close(fig)
110    buf.seek(0)
111    img = imageio.imread(buf)
112    gif_buffer = io.BytesIO()
113    imageio.mimsave(gif_buffer, [img], format='GIF', duration
114                    = 1.0)
115    gif_buffer.seek(0)
116    gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
117                                -8')
118
119    container = document.getElementById("mef1d-output")
120    metrics_html = f"""
121    <div class="bg-blue-50 border-l-4 border-blue-400 p-3
122            rounded w-full">
123        <h4 class="font-semibold text-blue-800 mb-1">Validação</
124            h4>

```

```

120         <p class="text-sm text-gray-700"><strong>Malha:</strong>
           {nel} elementos lineares, h = {h:.4f} m</p>
121         <p class="text-sm text-gray-700"><strong>T_max (MEF):</
           strong> {T_max_mef:.6f} °C em x = {x_max_mef:.3f} m</
           p>
122         <p class="text-sm text-gray-700"><strong>T_max (analí
           tica):</strong> {T_max_ana:.6f} °C em x = {x_max_ana
           :.3f} m</p>
123         <p class="text-sm text-gray-700"><strong>Erro máximo
           nodal:</strong> {e_max:.2e} °C</p>
124     </div>
125     """
126     container.innerHTML = (
127         f"<div class='flex flex-col gap-3 w-full'>"
128         f"<img src='data:image/gif;base64,{gif_base64}' class='
           rounded shadow w-full' />"
129         f"{metrics_html}"
130         f"</div>"
131     )
132
133     try:
134         sim_loading.close()
135     except Exception:
136         pass

```

A.2 Núcleo Numérico dos Módulos 2D

O módulo `common.py`, compartilhado pelos solvers de escoamento 2D, concentra a geração da malha triangular e a montagem das matrizes globais de rigidez e massa do MEF — o núcleo numérico reutilizado pelos módulos de escoamento potencial e de Navier-Stokes.

Listagem A.2: Núcleo MEF 2D compartilhado — `common.py` (completo)

```

1 import numpy as np
2 import matplotlib.tri as tri
3 from scipy.sparse import lil_matrix, csr_matrix
4 import matplotlib.pyplot as plt

```



```

5 import io
6 import base64
7
8 # =====
9 # 1. Geração da Malha (Genérica)
10 # =====
11 def generate_mesh_generic(L, H, boundary_points, mask_function,
12     cx, cy, nx, ny):
13     """
14     Gera malha triangular com um buraco definido por
15     boundary_points.
16     boundary_points: array (N, 2) com pontos da fronteira do
17     obstáculo.
18     mask_function: função f(x, y) -> bool que retorna True se o
19     ponto deve ser MANTIDO (fora do obstáculo).
20     """
21     # 1. Grid de fundo
22     x = np.linspace(0, L, nx)
23     y = np.linspace(0, H, ny)
24     dx = x[1] - x[0]
25     dy = y[1] - y[0]
26     h_mesh = min(dx, dy)
27
28     Xg, Yg = np.meshgrid(x, y)
29     X_flat = Xg.flatten()
30     Y_flat = Yg.flatten()
31
32     # 2. Nós da fronteira do obstáculo (passados como argumento)
33     x_bound = boundary_points[:, 0]
34     y_bound = boundary_points[:, 1]
35
36     # 3. Filtrar nós do grid usando a função de máscara
37     # Buffer para evitar triângulos muito finos (slivers)
38     # A função de máscara deve ser conservadora (remover apenas
39     # o que está garantidamente dentro)
40     # Mas aqui vamos aplicar diretamente
41     mask_keep = mask_function(X_flat, Y_flat)
42
43     X_grid_valid = X_flat[mask_keep]

```

```

39     Y_grid_valid = Y_flat[mask_keep]
40
41     # 4. Combinar nós
42     X = np.concatenate([X_grid_valid, x_bound])
43     Y = np.concatenate([Y_grid_valid, y_bound])
44
45     # 5. Triangulação
46     triang = tri.Triangulation(X, Y)
47
48     # 6. Mascaramos triângulos espúrios (dentro do obstáculo)
49     x_tri = X[triang.triangles].mean(axis=1)
50     y_tri = Y[triang.triangles].mean(axis=1)
51
52     # Se o centróide não passa na máscara de "manter", então
removemos
53     # Mas cuidado: a máscara de manter pode ser "dist > r +
buffer".
54     # Para triângulos, queremos remover se estiver DENTRO do
obstáculo exato.
55     # Vamos assumir que mask_function(x, y, strict=True) remove
o interior.
56     # Por simplicidade, vamos passar uma segunda função ou usar
a mesma com lógica interna.
57     # Vamos usar a mesma mask_function. Se retornar False (
remover), mascaramos.
58
59     triang.set_mask(~mask_function(x_tri, y_tri))
60
61     return X, Y, triang
62
63     # =====
64     # 2. Matrizes MEF (Triângulo Linear)
65     # =====
66     def element_matrices(coords):
67         x0, x1, x2 = coords
68         detJ = (x1[0] - x0[0]) * (x2[1] - x0[1]) - (x2[0] - x0[0]) *
            (x1[1] - x0[1])
69         area = 0.5 * abs(detJ)
70

```

```

71     b = np.array([x1[1] - x2[1], x2[1] - x0[1], x0[1] - x1[1]])
72     c = np.array([x2[0] - x1[0], x0[0] - x2[0], x1[0] - x0[0]])
73
74     ke = (1.0 / (4 * area)) * (np.outer(b, b) + np.outer(c, c))
75     me = (area / 12.0) * (np.ones((3, 3)) + np.eye(3))
76
77     return ke, me, area, b, c
78
79 def assemble_matrices(X, Y, triang):
80     n_nodes = len(X)
81     K = lil_matrix((n_nodes, n_nodes))
82     M = lil_matrix((n_nodes, n_nodes))
83
84     elements = triang.triangles
85     mask = triang.mask if triang.mask is not None else np.zeros(
86         len(elements), dtype=bool)
87
88     for i, el in enumerate(elements):
89         if mask[i]: continue
90
91         coords = np.column_stack((X[el], Y[el]))
92         ke, me, area, b, c = element_matrices(coords)
93
94         for row in range(3):
95             for col in range(3):
96                 K[el[row], el[col]] += ke[row, col]
97                 M[el[row], el[col]] += me[row, col]
98
99     return K.tocsr(), M.tocsr()
100
101 # =====
102 # 3. Helpers de Plotagem
103 # =====
104 def plot_to_base64(fig):
105     buf = io.BytesIO()
106     plt.savefig(buf, format='png', bbox_inches='tight')
107     plt.close(fig)
108     buf.seek(0)
109     return base64.b64encode(buf.read()).decode("utf-8")

```

Do solver de Navier-Stokes 2D, reproduz-se o trecho central: o laço de avanço temporal da formulação Vorticidade-Corrente, com a solução da equação de Poisson para ψ , a imposição da condição de contorno de Thom nas paredes, a recuperação do campo de velocidades e o transporte semi-implícito da vorticidade.

Listagem A.3: Navier-Stokes 2D — laço temporal da formulação Vorticidade-Corrente (trecho de `simulation.py`)

```

225     while current_step < total_steps and not STOP_SIMULATION:
226         for _ in range(steps_per_frame):
227             # A. Poisson Psi
228             rhs_psi = M @ omega
229             b_psi = rhs_psi[free_psi] - K[free_psi, :][:,
                dirichlet_psi] @ psi[dirichlet_psi]
230             psi[free_psi] = spsolve(K_free_psi, b_psi)
231
232             # B. Thom's Formula (Generalized for moving walls)
233             # Canonical form: omega_w = -2 (psi_nb - psi_w)/h^2
                - 2 u_tan/h
234             # Derivation (top lid, n inward = -y): Taylor at the
                wall with step -h gives
235             # psi_nb = psi_w - h (dpsi/dy)_w + (h^2/2) (d2psi/
                dy2)_w
236             # Since u = dpsi/dy, (dpsi/dy)_w = u_tan. With psi
                constant along the wall,
237             # d2psi/dx2 = 0 there, so d2psi/dy2 = -omega_w.
                Solving:
238             # omega_w = -2 (psi_nb - psi_w + h u_tan) / h^2
239             # Validated against Ghia et al. (1982) at Re=100.
240             psi_nb = psi[wall_neighbors]
241             psi_w = psi[solid_walls]
242             h_vals = np.sqrt(wall_h_sq)
243             omega[solid_walls] = -2.0 * (psi_nb - psi_w + h_vals
                * wall_u_tan) / wall_h_sq
244
245             # C. Velocities
246             elements = triang.triangles
247             mask = triang.mask if triang.mask is not None else
                np.zeros(len(elements), dtype=bool)

```

```

248     el_nodes = elements[~mask]
249     x_el = X[el_nodes]
250     y_el = Y[el_nodes]
251     p_el = psi[el_nodes]
252
253     b_coeff = np.column_stack([y_el[:,1]-y_el[:,2], y_el
       [:,2]-y_el[:,0], y_el[:,0]-y_el[:,1]])
254     c_coeff = np.column_stack([x_el[:,2]-x_el[:,1], x_el
       [:,0]-x_el[:,2], x_el[:,1]-x_el[:,0]])
255     area_el = 0.5 * np.abs(x_el[:,0]*(y_el[:,1]-y_el
       [:,2]) + x_el[:,1]*(y_el[:,2]-y_el[:,0]) + x_el
       [:,2]*(y_el[:,0]-y_el[:,1]))
256
257     u_vals = np.sum(p_el * c_coeff, axis=1) / (2 *
        area_el)
258     v_vals = -np.sum(p_el * b_coeff, axis=1) / (2 *
        area_el)
259
260     u_el_full = np.zeros(len(elements))
261     v_el_full = np.zeros(len(elements))
262     u_el_full[~mask] = u_vals
263     v_el_full[~mask] = v_vals
264
265     # D. Convection
266     C = assemble_convection(X, Y, triang, u_el_full,
        v_el_full)
267
268     # E. Vorticity Transport
269     A = M + (dt / Re) * K
270     rhs = M @ omega - dt * (C @ omega)
271
272     A_free = A[free_omega, :][:, free_omega]
273     b_omega = rhs[free_omega] - A[free_omega, :][:,
        dirichlet_omega] @ omega[dirichlet_omega]
274     omega[free_omega] = spsolve(A_free, b_omega)
275
276     current_step += 1

```

Apêndice B

Scripts de Benchmark Computacional

Este apêndice reúne as listagens dos *scripts* utilizados para gerar os tempos reportados na Seção 4.2.4 (Tabela 4.2 e Figura 4.2). Os *solvers* numéricos em `benchmark/core/` são cópias diretas das implementações dos módulos `simulation.py` disponíveis no repositório da plataforma — das quais o Apêndice A reproduz listagens representativas —, com a única alteração da remoção das chamadas ao DOM (`document.getElementById`) e à camada de visualização (`matplotlib`, `imageio`). Os algoritmos numéricos — montagem das matrizes elementares e globais, imposição das condições de contorno, resolução do sistema linear e avanço temporal — são preservados *linha a linha*, de modo a garantir que o tempo medido corresponde de fato ao núcleo numérico executado pelo usuário no navegador. Os mesmos *scripts* são invocados em ambos os ambientes do *benchmark* (CPython local e Pyodide no navegador), assegurando a comparabilidade dos resultados.

B.1 Solver de Condução Permanente 1D Adaptado

Listagem B.1: `benchmark/core/cond_1d.py` — cópia adaptada do solver MEF de condução permanente 1D

```
1 """
```

```

2  Cópia adaptada do solver MEF de Condução Permanente 1D do
    MinervaMesh,
3  para uso nos benchmarks. O algoritmo numérico é idêntico ao de
4  ‘‘simulations/mef/conducao-permanente-barra1d-mef/simulation.py
    ‘‘ --- apenas
5  removidos os imports JS/DOM e o pós-processamento (Matplotlib,
    imageio,
6  escrita no DOM). Os parâmetros entram como argumentos de função.
7  """
8
9  import time
10 import numpy as np
11
12
13 def analytical_solution(x, Q, k, L, T0, TL):
14     C1 = (TL - T0 + (Q * L * L) / (2 * k)) / L
15     C2 = T0
16     return -(Q / (2 * k)) * x * x + C1 * x + C2
17
18
19 def solve_cond_1d(L=1.0, nel=10, T0=0.0, TL=0.0, k=1.0, Q=1.0):
20     """
21     Resolve a condução permanente 1D pelo MEF (elementos
        lineares).
22
23     Retorna um dict com:
24         x      --- coordenadas nodais
25         T      --- temperaturas nodais
26         T_ana  --- solução analítica nos mesmos nós
27         t_assembly --- tempo de montagem da matriz global (s)
28         t_solve  --- tempo de np.linalg.solve (s)
29         t_total  --- soma dos dois acima
30         nn      --- número de nós (= nel + 1)
31     """
32     nn = nel + 1
33     x = np.linspace(0.0, L, nn)
34     h = L / nel
35
36     # ---- Montagem ----

```

```

37     t0 = time.perf_counter()
38     K = np.zeros((nn, nn))
39     b = np.zeros(nn)
40     ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
41     be = (Q * h / 2.0) * np.array([1.0, 1.0])
42     for e in range(nel):
43         n1, n2 = e, e + 1
44         K[n1:n2 + 1, n1:n2 + 1] += ke
45         b[n1:n2 + 1] += be
46
47     # Dirichlet
48     K[0, :] = 0.0
49     K[0, 0] = 1.0
50     b[0] = T0
51     K[-1, :] = 0.0
52     K[-1, -1] = 1.0
53     b[-1] = TL
54     t_assembly = time.perf_counter() - t0
55
56     # ---- Solve ----
57     t0 = time.perf_counter()
58     T = np.linalg.solve(K, b)
59     t_solve = time.perf_counter() - t0
60
61     T_ana = analytical_solution(x, Q, k, L, T0, TL)
62
63     return {
64         "x": x,
65         "T": T,
66         "T_ana": T_ana,
67         "t_assembly": t_assembly,
68         "t_solve": t_solve,
69         "t_total": t_assembly + t_solve,
70         "nn": nn,
71     }

```

B.2 Solver de Transcalor 2D Adaptado

Listagem B.2: benchmark/core/transcalor_2d.py — cópia adaptada do solver MEF de transferência de calor 2D

```
1  """
2  Cópia adaptada do solver MEF de Transferência de Calor 2D do
3  MinervaMesh,
4  para uso nos benchmarks. O algoritmo numérico é idêntico ao de
5  ‘simulations/mef/transcalor/simulation.py’ --- apenas
6  removidos os imports
7  JS/DOM e a geração de frames (Matplotlib, imageio). A malha e os
8  parâmetros
9  entram como argumentos de função em vez de globais de módulo.
10 """
11
12 import time
13 import numpy as np
14 import matplotlib.tri as tri
15 from scipy.sparse import lil_matrix
16 from scipy.sparse.linalg import spsolve
17
18 def element_matrices(coords, k, rho, cv):
19     x0, x1, x2 = coords
20     area = 0.5 * abs(
21         (x1[0] - x0[0]) * (x2[1] - x0[1]) - (x2[0] - x0[0]) * (
22             x1[1] - x0[1])
23     )
24     b = np.array([x1[1] - x2[1], x2[1] - x0[1], x0[1] - x1[1]])
25     c = np.array([x2[0] - x1[0], x0[0] - x2[0], x1[0] - x0[0]])
26     ke = (k / (4 * area)) * (np.outer(b, b) + np.outer(c, c))
27     me = (rho * cv * area / 12.0) * (np.ones((3, 3)) + np.eye(3))
28
29     return ke, me
30
31 def solve_transcalor_2d(
32     nx=40, ny=40, dt=0.01, n_steps=50,
33     Tbottom=0.0, Ttop=100.0, k=1.0, rho=1.0, cv=1.0,
34 ):
```

```

32     """
33     Resolve condução de calor 2D transiente em placa unitária
        pelo MEF
34     (triângulos lineares + Crank-Nicolson). Mesma lógica que o m
        ódulo
35     ‘‘transcalor’’ da plataforma, sem geração de frames.
36
37     Retorna um dict com:
38         u          --- temperaturas nodais finais
39         X, Y       --- coordenadas dos nós
40         n_nodes    --- número de nós
41         n_elements --- número de elementos
42         t_mesh      --- tempo de geração de malha
43         t_assembly  --- tempo de montagem (M, K) + aplicação de
            CCs em A
44         t_loop      --- tempo total do loop temporal (n_steps ×
            spsolve)
45         t_total     --- soma de assembly + loop (núcleo numérico)
46     """
47     # ---- Malha ----
48     t0 = time.perf_counter()
49     x_lin = np.linspace(0.0, 1.0, nx)
50     y_lin = np.linspace(0.0, 1.0, ny)
51     Xg, Yg = np.meshgrid(x_lin, y_lin)
52     X = Xg.flatten()
53     Y = Yg.flatten()
54     points = np.column_stack((X, Y))
55     triang = tri.Triangulation(X, Y)
56     elements = triang.triangles
57     n_nodes = len(points)
58     t_mesh = time.perf_counter() - t0
59
60     # ---- Montagem ----
61     t0 = time.perf_counter()
62     M = lil_matrix((n_nodes, n_nodes))
63     K = lil_matrix((n_nodes, n_nodes))
64     for el in elements:
65         coords = points[el]
66         ke, me = element_matrices(coords, k, rho, cv)

```

```

67         for i in range(3):
68             for j in range(3):
69                 K[el[i], el[j]] += ke[i, j]
70                 M[el[i], el[j]] += me[i, j]
71     M = M.tocsr()
72     K = K.tocsr()
73
74     u = np.zeros(n_nodes)
75     for i in range(n_nodes):
76         xi, yi = X[i], Y[i]
77         if np.isclose(xi, 0) or np.isclose(xi, 1):
78             u[i] = Tbottom + (Ttop - Tbottom) * yi
79         elif np.isclose(yi, 0):
80             u[i] = Tbottom
81         elif np.isclose(yi, 1):
82             u[i] = Ttop
83
84     u_init = u.copy()
85     boundary_nodes = [
86         i for i in range(n_nodes)
87         if np.isclose(X[i], 0) or np.isclose(X[i], 1)
88         or np.isclose(Y[i], 0) or np.isclose(Y[i], 1)
89     ]
90
91     A = M + dt / 2 * K
92     B = M - dt / 2 * K
93     A = A.tolil()
94     for i in boundary_nodes:
95         A[i, :] = 0
96         A[i, i] = 1
97     A = A.tocsr()
98     t_assembly = time.perf_counter() - t0
99
100     # ---- Loop temporal ----
101     t0 = time.perf_counter()
102     for _ in range(n_steps):
103         b = B @ u
104         b[boundary_nodes] = u_init[boundary_nodes]
105         u = spsolve(A, b)

```

```

106     t_loop = time.perf_counter() - t0
107
108     return {
109         "u": u,
110         "X": X,
111         "Y": Y,
112         "n_nodes": n_nodes,
113         "n_elements": len(elements),
114         "t_mesh": t_mesh,
115         "t_assembly": t_assembly,
116         "t_loop": t_loop,
117         "t_total": t_assembly + t_loop,
118     }

```

B.3 Biblioteca Auxiliar de Cronometragem

Listagem B.3: benchmark/bench_lib.py — *context manager* para medição de fases, função de rodadas repetidas com mediana e MAD, e escrita CSV

```

1  """
2  Utilidades de benchmarking compartilhadas entre o runner local (
    CPython) e o
3  runner no navegador (PyScript). Define um *context manager*
    simples para
4  medir fases, função para rodadas repetidas com estatísticas
    robustas
5  (mediana e MAD), e escrita de CSV em formato comum.
6  """
7
8  import csv
9  import statistics
10 import time
11
12
13 class Phase:
14     """Mede o tempo de uma fase do código.
15
16     Uso:

```

```

17         with Phase('solve') as p:
18             ...
19         print(p.dt)
20     """
21
22     def __init__(self, name=""):
23         self.name = name
24         self.dt = 0.0
25
26     def __enter__(self):
27         self._t0 = time.perf_counter()
28         return self
29
30     def __exit__(self, exc_type, exc_val, exc_tb):
31         self.dt = time.perf_counter() - self._t0
32         return False
33
34
35     def run_repeated(fn, n_runs=5, warmup=1):
36         """Executa ``fn`` ``warmup`` vezes (descartadas) e depois ``
37             n_runs`` vezes,
38
39             ``fn`` deve retornar um dict com chaves do tipo ``t_assembly
40            ``,
41             ``t_solve``, ``t_total`` etc.---qualquer chave começando com
42             ``t_`` é
43             tratada como métrica de tempo. As demais chaves do último
44             run são
45             preservadas no resultado (e.g., ``n_nodes``, ``n_elements``)
46             .
47
48             Retorna um dict ``{"<chave>_median": ..., "<chave>_mad":
49                 ..., ...
50             "_n_runs": n_runs}``` agregando as métricas de tempo, e
51             propaga as
52             chaves não-temporais do último run.
53         """
54         for _ in range(warmup):

```

```

49         fn()
50
51     samples = [fn() for _ in range(n_runs)]
52     out = {"_n_runs": n_runs}
53     last = samples[-1]
54
55     time_keys = [k for k in last.keys() if isinstance(last[k], (
56         int, float)) and k.startswith("t_")]
57     for k in time_keys:
58         vals = [s[k] for s in samples]
59         med = statistics.median(vals)
60         mad = statistics.median([abs(v - med) for v in vals])
61         out[f"{k}_median"] = med
62         out[f"{k}_mad"] = mad
63         out[f"{k}_min"] = min(vals)
64         out[f"{k}_max"] = max(vals)
65
66     for k, v in last.items():
67         if k.startswith("t_"):
68             continue
69         if isinstance(v, (int, float, str)):
70             out[k] = v
71
72     return out
73
74 def write_csv(rows, path, fieldnames=None):
75     """Escreve uma lista de dicts em CSV. Se ‘fieldnames’ não
76     for dado,
77     usa a união ordenada das chaves de todos os dicts."""
78     if not rows:
79         return
80     if fieldnames is None:
81         seen = []
82         for r in rows:
83             for k in r.keys():
84                 if k not in seen:
85                     seen.append(k)
86         fieldnames = seen

```

```

86     with open(path, "w", newline="") as f:
87         w = csv.DictWriter(f, fieldnames=fieldnames)
88         w.writeheader()
89         for r in rows:
90             w.writerow({k: r.get(k, "") for k in fieldnames})

```

B.4 Runner Local (CPython)

Listagem B.4: `benchmark/run_benchmark_local.py` — executa a varredura de tamanhos em CPython nativo e grava `results_local.csv`

```

1  """
2  Benchmark local (CPython nativo) dos solvers do MinervaMesh.
3
4  Roda os mesmos solvers que o navegador (importados de ‘
    benchmark.core’)
5  em uma varredura de tamanhos, mede tempos com mediana e MAD de 5
    rodadas
6  após 1 warm-up, e salva ‘benchmark/results/results_local.csv’.
7
8  Uso:
9      python3 benchmark/run_benchmark_local.py
10 """
11
12 import os
13 import platform
14 import sys
15 import time
16
17 import numpy as np
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 if HERE not in sys.path:
21     sys.path.insert(0, HERE)
22
23 from bench_lib import run_repeated, write_csv # noqa: E402
24 from core.cond_1d import solve_cond_1d # noqa: E402

```

```

25 from core.transcalor_2d import solve_transcalor_2d # noqa: E402
26
27
28 COND_1D_NEL = [100, 1000, 10000]
29 TRANSCALOR_NX = [20, 40, 80]
30 TRANSCALOR_N_STEPS = 50
31
32
33 def report_environment():
34     print("=" * 60)
35     print("Ambiente de execução")
36     print("=" * 60)
37     print(f"  Plataforma : {platform.platform()}")
38     print(f"  Processador: {platform.processor()}")
39     print(f"  Python      : {sys.version.split()[0]}")
40     print(f"  NumPy       : {np.__version__}")
41     print("  NumPy backend (np.show_config()):")
42     np.show_config()
43     print("=" * 60)
44
45
46 def bench_cond_1d():
47     rows = []
48     for nel in COND_1D_NEL:
49         print(f"[cond_1d] nel={nel} ...", flush=True)
50         agg = run_repeated(
51             lambda: solve_cond_1d(L=1.0, nel=nel, T0=0.0, TL
52                                   =0.0, k=1.0, Q=1.0),
53             n_runs=5, warmup=1,
54         )
55         agg.update({"case": "cond_1d", "param": f"nel={nel}", "
56                   size_n": nel + 1})
57         rows.append(agg)
58         print(f"  median t_total = {agg['t_total_median
59               ']*1000:.3f} ms (MAD {agg['t_total_mad']*1000:.3f} ms

```



```

60 def bench_transcalor_2d():
61     rows = []
62     for nx in TRANSCALOR_NX:
63         print(f"[transcalor_2d] nx=ny={nx}, n_steps={
64             TRANSCALOR_N_STEPS} ...", flush=True)
65         # Reduzir n_runs para 3 quando nx=80 para não inflar o
66         wall-clock
67         n_runs = 3 if nx >= 80 else 5
68         agg = run_repeated(
69             lambda nx=nx: solve_transcalor_2d(
70                 nx=nx, ny=nx, dt=0.01, n_steps=
71                 TRANSCALOR_N_STEPS,
72             ),
73             n_runs=n_runs, warmup=1,
74         )
75         agg.update({
76             "case": "transcalor_2d",
77             "param": f"nx=ny={nx}, n_steps={TRANSCALOR_N_STEPS}"
78             ,
79             "size_n": nx * nx,
80         })
81         rows.append(agg)
82         print(f"    median t_total = {agg['t_total_median']:.3f} s
83             (MAD {agg['t_total_mad']:.3f} s)")
84     return rows
85
86 def main():
87     report_environment()
88     t_start = time.time()
89
90     rows = []
91     rows.extend(bench_cond_1d())
92     rows.extend(bench_transcalor_2d())
93
94     out_path = os.path.join(HERE, "results", "results_local.csv"
95         )
96     fieldnames = [
97         "case", "param", "size_n", "_n_runs",

```

```

93         "n_nodes", "n_elements", "nn",
94         "t_total_median", "t_total_mad", "t_total_min", "
           t_total_max",
95         "t_assembly_median", "t_assembly_mad",
96         "t_solve_median", "t_solve_mad",
97         "t_loop_median", "t_loop_mad",
98         "t_mesh_median", "t_mesh_mad",
99     ]
100     write_csv(rows, out_path, fieldnames=fieldnames)
101     print(f"\nCSV salvo em {out_path}")
102     print(f"Wall-clock total: {time.time() - t_start:.1f} s")
103
104
105 if __name__ == "__main__":
106     main()

```

B.5 Runner no Navegador (PyScript)

Listagem B.5: `benchmark/run_benchmark_browser.py` — mesmo *benchmark* executado sob Pyodide; o usuário inicia a execução por um botão na página `run_benchmark_browser.html` e obtém `results_browser.csv` via *download* no navegador

```

1  """
2  Benchmark no navegador (PyScript / Pyodide / WebAssembly).
3
4  Mesmo conjunto de varreduras do runner local: cond_1d (nel in
   {100, 1000, 10000})
5  e transcalor_2d (nx=ny in {20, 40, 80}, n_steps=50). Mediana e
   MAD de 5 runs
6  após 1 warm-up. Resultado é exposto na página e oferecido como
   download de CSV.
7
8  Carregado via ‘run_benchmark_browser.html’. Os arquivos ‘
   bench_lib.py’,
9  ‘core/__init__.py’, ‘core/cond_1d.py’, ‘core/transcalor_2d.
   py’ são

```

```

10 fornecidos pelo bloco ‘[[fetch]]’ em ‘config.toml’.
11 """
12
13 import io
14 import platform
15 import sys
16 import time
17
18 from js import Blob, URL, document, navigator
19 from pyscript import when
20
21 from bench_lib import run_repeated, write_csv
22 from core.cond_1d import solve_cond_1d
23 from core.transcalor_2d import solve_transcalor_2d
24
25
26 COND_1D_NEL = [100, 1000, 10000]
27 TRANSCALOR_NX = [20, 40, 80]
28 TRANSCALOR_N_STEPS = 50
29
30
31 def env_html():
32     return (
33         "<ul class='text-sm leading-relaxed'"
34         f"<li><b>Python:</b> {sys.version.split()[0]}</li>"
35         f"<li><b>Plataforma (Pyodide):</b> {platform.platform()}</li>"
36         f"<li><b>navigator.userAgent:</b> <code class='text-xs'"
37         f">{navigator.userAgent}</code></li>"
38         "</ul>"
39     )
40
41 def append_log(msg):
42     log_el = document.getElementById("bench-log")
43     log_el.innerHTML = log_el.innerHTML + f"<div>{msg}</div>"
44
45
46 def _bench_cond_1d():

```

```

47     rows = []
48     for nel in COND_1D_NEL:
49         append_log(f"[cond_1d] nel={nel} ...")
50         agg = run_repeated(
51             lambda nel=nel: solve_cond_1d(L=1.0, nel=nel, T0
52                 =0.0, TL=0.0, k=1.0, Q=1.0),
53             n_runs=5, warmup=1,
54             )
55         agg.update({"case": "cond_1d", "param": f"nel={nel}", "
56             size_n": nel + 1})
57         rows.append(agg)
58         append_log(
59             f"    median t_total = {agg['t_total_median']*1000:.3f
60                 } ms "
61             f"(MAD {agg['t_total_mad']*1000:.3f} ms)"
62         )
63     return rows
64
65 def _bench_transcalor_2d():
66     rows = []
67     for nx in TRANSCALOR_NX:
68         append_log(f"[transcalor_2d] nx=ny={nx}, n_steps={
69             TRANSCALOR_N_STEPS} ...")
70         n_runs = 3 if nx >= 80 else 5
71         agg = run_repeated(
72             lambda nx=nx: solve_transcalor_2d(
73                 nx=nx, ny=nx, dt=0.01, n_steps=
74                     TRANSCALOR_N_STEPS,
75                 ),
76             n_runs=n_runs, warmup=1,
77             )
78         agg.update({
79             "case": "transcalor_2d",
80             "param": f"nx=ny={nx}, n_steps={TRANSCALOR_N_STEPS}"
81             ,
82             "size_n": nx * nx,
83         })
84     rows.append(agg)

```

```

80         append_log(
81             f"    median t_total = {agg['t_total_median']:.3f} s "
82             f"(MAD {agg['t_total_mad']:.3f} s)"
83         )
84     return rows
85
86
87 def offer_download(csv_text, filename):
88     blob = Blob.new([csv_text], {"type": "text/csv"})
89     url = URL.createObjectURL(blob)
90     a = document.createElement("a")
91     a.href = url
92     a.download = filename
93     a.textContent = f"Baixar {filename}"
94     a.className = "inline-block mt-3 px-4 py-2 bg-blue-600 text-
        white rounded shadow hover:bg-blue-700"
95     container = document.getElementById("bench-download")
96     container.innerHTML = ""
97     container.appendChild(a)
98
99
100 @when("click", "#bench-run")
101 def on_run(event=None):
102     document.getElementById("bench-log").innerHTML = ""
103     document.getElementById("bench-download").innerHTML = ""
104     document.getElementById("bench-run").disabled = True
105     document.getElementById("bench-run").textContent = "Rodando
        ..."
106
107     document.getElementById("bench-env").innerHTML = env_html()
108
109     t0 = time.time()
110     rows = []
111     rows.extend(_bench_cond_1d())
112     rows.extend(_bench_transcator_2d())
113     elapsed = time.time() - t0
114     append_log(f"<b>Wall-clock total:</b> {elapsed:.1f} s")
115
116     fieldnames = [

```

```

117         "case", "param", "size_n", "_n_runs",
118         "n_nodes", "n_elements", "nn",
119         "t_total_median", "t_total_mad", "t_total_min", "
            t_total_max",
120         "t_assembly_median", "t_assembly_mad",
121         "t_solve_median", "t_solve_mad",
122         "t_loop_median", "t_loop_mad",
123         "t_mesh_median", "t_mesh_mad",
124     ]
125     buf = io.StringIO()
126
127     # write_csv escreve em arquivo; faço aqui em memória usando
128     csv.DictWriter direto
129     import csv as _csv
130     seen_keys = []
131     for r in rows:
132         for k in r.keys():
133             if k not in seen_keys:
134                 seen_keys.append(k)
135     use_fields = [f for f in fieldnames if f in seen_keys]
136     w = _csv.DictWriter(buf, fieldnames=use_fields)
137     w.writeheader()
138     for r in rows:
139         w.writerow({k: r.get(k, "") for k in use_fields})
140
141     offer_download(buf.getvalue(), "results_browser.csv")
142
143     document.getElementById("bench-run").disabled = False
144     document.getElementById("bench-run").textContent = "Rodar
145         benchmark novamente"
146
147     # Confirma carregamento bem-sucedido
148     document.getElementById("bench-status").textContent = "Pyodide
149         pronto. Clique em 'Rodar benchmark'."

```